

UNIVERSITY OF THESSALY
SCHOOL OF ENGINEERING
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

**Energy-Efficient Hardware Implementation of a Scalable
FFT Processor**

Diploma Thesis

Christodoulos Zerdalis

Supervisor: Georgios Karakonstantis

Volos, September 2026



UNIVERSITY OF THESSALY
SCHOOL OF ENGINEERING
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

**Energy-Efficient Hardware Implementation of a Scalable
FFT Processor**

Diploma Thesis

Christodoulos Zerdalis

Supervisor: Georgios Karakonstantis

Volos, September 2026



ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΙΑΣ

ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ

ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

**Υλοποίηση Ενεργειακά Αποδοτικού Επεξεργαστή FFT σε
Υλικό με Δυνατότητα Κλιμάκωσης**

Διπλωματική Εργασία

Χριστόδουλος Ζερδαλής

Επιβλέπων: Γεώργιος Καρακωνσταντής

Βόλος, Σεπτέμβριος 2026

Approved by the Examination Committee:

Supervisor **Georgios Karakonstantis**

Associate Professor, Department of Electrical and Computer Engineering, University of Thessaly

Member **Georgios Stamoulis**

Professor, Department of Electrical and Computer Engineering, University of Thessaly

Member **Nestoras Evmorfopoulos**

Professor, Department of Electrical and Computer Engineering, University of Thessaly

Acknowledgements

I would first like to thank Professor Georgios Karakonstantis for giving me the opportunity to work on a diploma thesis project that I really enjoyed. I am grateful for his valuable advice, his expertise, and our great collaboration. Not only did I manage to successfully complete my thesis, but I also learned many new things that will be very useful for my future career. I would also like to thank Professor Georgios Stamoulis and Professor Nestoras Evmorfopoulos, with whom I have worked in past semesters, for serving on the examination committee and reviewing this thesis.

I would also like to deeply thank my family for supporting me both financially and emotionally throughout these five years. I owe the successful completion of my studies and all the knowledge I gained along the way to their unconditional support.

Lastly, I would like to thank all my friends, my fellow students, and the people close to me. You made life during these past years special and helped me grow as a person in invaluable ways.

DISCLAIMER ON ACADEMIC ETHICS AND INTELLECTUAL PROPERTY RIGHTS

«Being fully aware of the implications of copyright laws, I expressly state that this diploma thesis, as well as the electronic files and source codes developed or modified in the course of this thesis, are solely the product of my personal work and do not infringe any rights of intellectual property, personality and personal data of third parties, do not contain work / contributions of third parties for which the permission of the authors / beneficiaries is required and are not a product of partial or complete plagiarism, while the sources used are limited to the bibliographic references only and meet the rules of scientific citing. The points where I have used ideas, text, files and / or sources of other authors are clearly mentioned in the text with the appropriate citation and the relevant complete reference is included in the bibliographic references section. I also declare that the results of the work have not been used to obtain another degree. I fully, individually and personally undertake all legal and administrative consequences that may arise in the event that it is proven, in the course of time, that this thesis or part of it does not belong to me because it is a product of plagiarism».

The declarant



Christodoulos Zerdalis

Diploma Thesis

Energy-Efficient Hardware Implementation of a Scalable FFT Processor

Christodoulos Zerdalis

Abstract

This thesis lies in the field of digital hardware design for signal processing, and specifically in the fixed-point implementation of the Fast Fourier Transform on FPGA devices. Its subject is the design, verification and comparative evaluation of scalable pipelined low area and power FFT architectures, together with the arithmetic and resource trade-offs that determine their cost, speed and numerical accuracy.

Three decompositions, radix-2, radix-4 and split-radix, were implemented in parameterisable RTL for transform sizes of 256, 1024 and 4096 points. All follow the multi-path delay feedback (MDF) architecture, which restricts each FFT stage to a single compute unit, and share stage-specific twiddle memories and a configurable complex multiplication scheme. Implementations combining different memory and complex multiplication architectures, based on DSP blocks, Block RAM and LUTRAM, are compared with one another and against logic-only implementations built on two custom multipliers: a pipelined carry-save array and a Computation Sharing Multiplier (CSHM). Functional accuracy was verified against a floating-point golden model at twiddle widths from 8 to 16 bits, and the designs were synthesised for an Artix-7 XC7A200T device at widths of 9, 13 and 16 bits. Both flows are fully automated by Python scripts that can simulate and implement any transform size for every combination of the parameters above. Finally, the CSHM design space itself was explored in the context of the FFT, in order to establish which hardware configurations are best suited to twiddle-factor multiplication.

Accuracy improves by approximately 6 dB per additional twiddle bit, with split-radix the most accurate at every operating point, its structural complexity, however, makes it the most expensive in area, leaving radix-2 and radix-4 as the more balanced alternatives. Reducing the twiddle width and moving storage into Block RAM both decrease area drastically and improve timing, and the radix-4 design with Block RAM proves the least expensive and most efficient configuration, combining the lowest latency with a throughput above 1400

MSamples/s. Among the custom multipliers, the CSHM occupies less area than the carry-save array and reaches the lowest overall latency, at the cost of a slightly longer critical path.

Keywords:

Fast Fourier Transform, FPGA, Pipelined architecture, Radix-2, Radix-4, Split-radix, Fixed-point arithmetic, Quantization noise, BRAM, DSP, Twiddle factors, CSHM, Carry-save multiplier, High throughput, Low latency, Low energy, Area optimization.

Διπλωματική Εργασία

Υλοποίηση Ενεργειακά Αποδοτικού Επεξεργαστή FFT σε Υλικό με

Δυνατότητα Κλιμάκωσης

Χριστόδουλος Ζερδαλής

Περίληψη

Η παρούσα διπλωματική εργασία εντάσσεται στο πεδίο της σχεδίασης ψηφιακών κυκλωμάτων για επεξεργασία σημάτων και, ειδικότερα, στην υλοποίηση του Ταχέος Μετασχηματισμού Fourier με αριθμητική σταθερής υποδιαστολής σε διατάξεις FPGA. Αντικείμενό της είναι η σχεδίαση, η επαλήθευση και η συγκριτική αξιολόγηση κλιμακώσιμων αρχιτεκτονικών FFT με διοχέτευση, χαμηλής επιφάνειας και κατανάλωσης ισχύος, καθώς και οι συμβιβασμοί αριθμητικής και πόρων που καθορίζουν το κόστος, την ταχύτητα και την αριθμητική τους ακρίβεια.

Υλοποιήθηκαν τρεις αλγόριθμοι αποσύνθεσης, radix-2, radix-4 και split-radix, σε παραμετροποιήσιμη περιγραφή RTL, για μεγέθη μετασχηματισμού 256, 1024 και 4096 σημείων. Όλοι ακολουθούν την αρχιτεκτονική multi-path delay feedback (MDF), η οποία περιορίζει κάθε βαθμίδα του FFT σε μία μόνο μονάδα υπολογισμού, και μοιράζονται μνήμες συντελεστών twiddle ανά βαθμίδα καθώς και ένα παραμετροποιήσιμο σχήμα μιγαδικού πολλαπλασιασμού. Υλοποιήσεις που συνδυάζουν διαφορετικές αρχιτεκτονικές μνήμης και μιγαδικού πολλαπλασιασμού, βασισμένες σε μονάδες DSP, Block RAM και LUTRAM, συγκρίνονται μεταξύ τους αλλά και με υλοποιήσεις αποκλειστικά σε λογική, οι οποίες στηρίζονται σε δύο εξειδικευμένους πολλαπλασιαστές: έναν διοχετευμένο πολλαπλασιαστή carry-save και έναν πολλαπλασιαστή διαμοιρασμού υπολογισμών (CSHM). Η λειτουργική ακρίβεια επαληθεύτηκε ως προς πρότυπο αναφοράς κινητής υποδιαστολής, για εύρη συντελεστών twiddle από 8 έως 16 bits, ενώ η υλοποίηση έγινε στη διάταξη Artix-7 XC7A200T για εύρη 9, 13 και 16 bits. Και οι δύο ροές είναι πλήρως αυτοματοποιημένες μέσω Python scripts, τα οποία μπορούν να προσομοιώσουν και να υλοποιήσουν οποιοδήποτε μέγεθος μετασχηματισμού για κάθε συνδυασμό των παραπάνω παραμέτρων. Τέλος, διερευνήθηκε ο χώρος σχεδίασης του CSHM στο πλαίσιο του FFT, ώστε να προσδιοριστεί ποιες επιλογές υλικού είναι καταλληλότερες για τον πολλαπλασιασμό με συντελεστές twiddle.

Η ακρίβεια βελτιώνεται κατά περίπου 6 dB για κάθε επιπλέον bit συντελεστή twiddle, με

τον αλγόριθμο split-radix να επιτυγχάνει την υψηλότερη ακρίβεια σε κάθε σημείο λειτουργίας. Ωστόσο, η δομική του πολυπλοκότητα τον καθιστά τον πιο δαπανηρό σε επιφάνεια, καθιστώντας τις αρχιτεκτονικές radix-2 και radix-4 πιο αποδοτικές. Η μείωση του εύρους των συντελεστών twiddle και η μεταφορά της αποθήκευσης σε Block RAM μειώνουν δραστικά την επιφάνεια και βελτιώνουν τον χρονισμό, ενώ η υλοποίηση radix-4 με Block RAM αποδεικνύεται η λιγότερο δαπανηρή και αποδοτικότερη διαμόρφωση, συνδυάζοντας τη χαμηλότερη καθυστέρηση με ρυθμό απόδοσης άνω των 1400 MSamples/s. Μεταξύ των εξειδικευμένων πολλαπλασιαστών, ο CSHM καταλαμβάνει λιγότερη επιφάνεια από τον carry-save και επιτυγχάνει τη χαμηλότερη συνολική καθυστέρηση, με τίμημα μια ελαφρώς χαμηλότερη συχνότητα λειτουργίας.

Λέξεις-κλειδιά:

Ταχύς Μετασχηματισμός Fourier, FPGA, αρχιτεκτονική διοχέτευσης, radix-2, radix-4, split-radix, αριθμητική σταθερής υποδιαστολής, θόρυβος κβαντισμού, BRAM, DSP, συντελεστές twiddle, CSHM, πολλαπλασιαστής carry-save, υψηλός ρυθμός απόδοσης, χαμηλή καθυστέρηση, χαμηλή κατανάλωση, βελτιστοποίηση επιφάνειας.

Table of contents

Acknowledgements	ix
Abstract	xii
Περίληψη	xiv
Table of contents	xvii
List of figures	xxi
List of tables	xxv
Abbreviations	xxvii
1 Introduction	1
1.1 Problem Statement	1
1.1.1 Research Objectives and Contributions	2
1.2 Thesis Structure	2
2 Background and Theoretical Foundations	5
2.1 Introduction to Fourier Transform	5
2.2 Fast Fourier Transform in General	6
2.3 FFT Decomposition	7
2.3.1 Radix-2	7
2.3.2 Radix-4	9
2.3.3 Split-Radix	9
2.4 Unrolled and Single-Path Delay Feedback Approaches	10
2.5 Fixed-Point Arithmetic and Quantization Noise	11

2.6	Custom Multipliers	13
2.6.1	Carry-Save Array Multiplier	13
2.6.2	Computation-Sharing Hardware Multiplier (CSHM)	14
2.7	FPGA Hardware Resources	15
3	Hardware Implementation of the Proposed Algorithms	17
3.1	Introduction	17
3.2	Scalability	17
3.3	Shared Architectural Elements	17
3.3.1	Twiddle Factors	18
3.3.2	Datapaths	20
3.3.3	Datatypes	20
3.3.4	Complex Multiplication Techniques	20
3.4	Radix-2 Implementation	21
3.4.1	Review of Prior Implementation	21
3.4.2	Overview of Proposed Implementation	22
3.4.3	Butterfly Radix-2	22
3.4.4	First Stage	24
3.4.5	Control Unit	24
3.4.6	Delay Registers and Final Design	25
3.4.7	Implementation Results using an ISO Clock	26
3.5	Radix-4 Implementation	29
3.5.1	Overview of Proposed Implementation	29
3.5.2	Butterfly Radix-4	29
3.5.3	First Stage	29
3.5.4	Control Unit	31
3.5.5	Initial Delay Register Architecture	31
3.5.6	Proposed BRAM-Compatible Memory Structure	33
3.5.7	Final Design and Results using an ISO Clock	35
3.6	Split-Radix Implementation	39
3.6.1	Overview of Proposed Implementation	39
3.6.2	Butterfly Split-Radix and Complex Multipliers	41
3.6.3	First Stage	42

3.6.4	Second Stage	44
3.6.5	Third Stage	47
3.6.6	Penultimate Stage and Last Stage	50
3.6.7	Implementation Results using an ISO Clock	51
3.7	Timing and Critical Path Optimization	54
3.7.1	DSP Optimizations	55
3.7.2	Memory and Bram Optimization	58
4	Functional Simulation and Algorithmic Verification	59
4.1	Overview	59
4.2	Stimulus Generation	60
4.3	Simulation	61
4.4	Comparison with Golden Model	61
5	Custom Multipliers	65
5.1	Carry-Save Array Multiplier Architecture	65
5.2	CSHM Architecture	66
5.2.1	Architecture Choices	67
5.2.2	Hardware Resource Comparison	68
5.2.3	Exploration of Alphabets and Set Combinations	69
5.2.4	Simulation Accuracy Results	73
6	Implementation Results	75
6.1	9-Bit Unoptimized vs. Optimized Design	76
6.2	9-Bit Carry-Save vs. CSHM Multipliers	78
6.3	13-Bit and 16-Bit Implementations	80
6.3.1	DSP and BRAM Optimized Versions	81
6.3.2	Carry-Save and CSHM Versions	84
6.4	Latency and Throughput Results	86
6.4.1	DSP and BRAM Optimized Versions	86
6.4.2	Carry-Save and CSHM Versions	87
6.5	Evaluation Against Existing Architectures	88

7	Conclusions and Future Work	91
7.1	Summary of Results	91
7.2	Future Work and Extensions	92
	Bibliography	93

List of figures

2.1	Representation of a time-domain signal decomposed into multiple signals of different frequencies.	5
2.2	Comparison of Radix-2, Radix-4 and Split-Radix dataflow structures. . . .	7
2.3	Recursive logic of the Radix-2 FFT, demonstrating the combination of even and odd sequences from previous stages.	8
2.4	Comparison of unrolled architectures.	11
2.5	Simplified diagram of the SDF units discussed in this chapter.	11
2.6	Formulas for the signal power and Signal-to-Quantization-Noise Ratio (SQNR). P_{x^ν} represents the average power of the input signal. ν represents the number of bits (resolution).	12
2.7	Top diagram of the Carry-Save Array Multiplier discussed in this chapter .	13
2.8	Top diagram of the CSHM Multiplier discussed in this chapter	14
3.1	Twiddle factor visualization.	18
3.2	Radix-2 first stage top module.	24
3.3	Operation flow of the counters managing the pairs of data points.	25
3.4	Radix-2 general stage unit top module.	26
3.5	Utilization comparison for Radix-2 FFT implementations across sizes 256, 1024, and 4096 under ISO clock constraints (16-bit twiddle width).	27
3.6	Utilization comparison for Radix-2 FFT implementations across sizes 256, 1024, and 4096 under ISO clock constraints (8-bit twiddle width).	28
3.7	Radix-4 first stage top module.	29
3.8	Operation flow of the counters managing the sets of four (quads) data points.	31
3.9	Top diagram Radix-4 stage unit delay registers.	32

3.10 Utilization comparison for Radix-4 FFT implementations across sizes under ISO clock constraints.	32
3.11 Memory module general Organization.	33
3.12 Memory module organization and access pattern.	34
3.13 Top diagram Radix-4 stage unit with new memory.	35
3.14 Resource utilization for Radix-4 FFT with new memory at $N = 256$ (16-bit).	35
3.15 Resource utilization for Radix-4 FFT with new memory at $N = 1024$ (16-bit).	36
3.16 Resource utilization for Radix-4 FFT with new memory at $N = 4096$ (16-bit).	36
3.17 Resource utilization for Radix-4 FFT with new memory at $N = 256$ (8-bit).	37
3.18 Resource utilization for Radix-4 FFT with new memory at $N = 1024$ (8-bit).	37
3.19 Resource utilization for Radix-4 FFT with new memory at $N = 4096$ (8-bit).	38
3.20 New Split-Radix Dataflow structure, example for an FFT size of 32.	40
3.21 Split-Radix stages, example for an FFT size of 32.	41
3.22 Split Radix butterfly.	42
3.23 First Stage Split-Radix.	43
3.24 Memory pattern First Stage Split-Radix.	43
3.25 Inverse mapping formula for digit reshuffling, p is the output index, N is the size of the FFT and k the real index	44
3.26 Input-Output flow of Split-Radix second stage.	45
3.27 Memory pattern second stage split radix.	45
3.28 Second Stage Unit Top Diagram.	46
3.29 Third stage Dataflow	47
3.30 Third stage Dataflow sequence	48
3.31 Third stage Control Unit	48
3.32 More detailed diagram for the processing state	49
3.33 Third Stage Unit Top Diagram.	50
3.34 Penultimate Stage Unit Top Diagram.	50
3.35 Last Stage Unit Top Diagram.	51
3.36 Resource utilization for Split-Radix FFT with new memory at $N = 256$ (16-bit).	51
3.37 Resource utilization for Split-Radix FFT with new memory at $N = 1024$ (16-bit).	52

3.38	Resource utilization for Split-Radix FFT with new memory at $N = 4096$ (16-bit).	52
3.39	Resource utilization for Split-Radix FFT with new memory at $N = 256$ (8-bit).	53
3.40	Resource utilization for Split-Radix FFT with new memory at $N = 1024$ (8-bit).	53
3.41	Resource utilization for Split-Radix FFT with new memory at $N = 4096$ (8-bit).	54
3.42	Critical path of unoptimized DSPs.	55
3.43	Comparison of DSP multiplication sequences.	56
3.44	Register positions inside the DSP48E1 schematic.	57
3.45	Register positions inside the RAMB36E1/RAMB18E1 schematic.	58
4.1	Automated verification flow implemented in Python, showing the data path from data generation to hardware simulation and final numerical analysis.	60
4.2	FFT output accuracy - low frequency input (averaged over SimpleMult / CheapMult)	61
4.3	FFT output accuracy - high frequency input (averaged over SimpleMult / CheapMult)	62
4.4	FFT output accuracy - random frequency input (averaged over SimpleMult / CheapMult)	62
5.1	Top diagram of the general CSHM Multiplier	67
5.2	Top diagram of the CSHM Architecture used	68
5.3	Utilization comparison for different custom multipliers.	68
5.5	Alphabet needed vs twiddle width ($N = 256$, 2 slots)	70
5.4	Slot and alphabet search implemented. For every slot count the alphabet is enlarged until the reachable set covers all quantised twiddle factors of the stage within the tolerance window.	71
5.6	Alphabet needed vs twiddle width ($N = 256$, 3 slots)	72
5.7	CSHM alphabet exploration with no tolerance.	72
5.8	CSHM key ROM.	73
5.9	Different CSHM modules and their accuracy.	74
6.1	FPGA results 9-bit Unopt vs Opt ($N = 256$).	77

6.2	FPGA results 9-bit Unopt vs Opt ($N = 1024$).	77
6.3	FPGA results 9-bit Unopt vs Opt ($N = 4096$).	78
6.4	FPGA resources 9-Bit Carry-Save vs CSHM ($N = 256$).	79
6.5	FPGA resources 9-Bit Carry-Save vs CSHM ($N = 1024$).	79
6.6	FPGA resources 9-Bit Carry-Save vs CSHM ($N = 4096$).	80
6.7	FPGA resources 13-bit Opt ($N = 256$).	81
6.8	FPGA resources 13-bit Opt ($N = 1024$).	81
6.9	FPGA resources 13-bit Opt ($N = 4096$).	82
6.10	FPGA resources 16-bit Opt ($N = 256$).	82
6.11	FPGA resources 16-bit Opt ($N = 1024$).	83
6.12	FPGA resources 16-bit Opt ($N = 4096$).	83
6.13	FPGA resources 13-Bit Carry-Save vs CSHM ($N = 256$).	84
6.14	FPGA resources 13-Bit Carry-Save vs CSHM ($N = 1024$).	85
6.15	FPGA resources 13-Bit Carry-Save vs CSHM ($N = 4096$).	85

List of tables

2.1	Maximum theoretical SQNR for various hardware bit depths (assuming full-scale square wave input).	12
3.1	Comparison of twiddle factor memory footprint between a naive duplicated storage approach and the proposed stage-specific optimized generation for an 8-stage Radix-2 FFT (DIT).	19
3.2	Synthesis results and latency for the baseline 256-point FFT architecture. .	22
3.3	FIFO Radix-2 Latency.	28
3.4	FIFO Radix-4 Latency.	38
3.5	FIFO Split-Radix Latency.	54
6.1	Hardware specifications of the AMD Xilinx Artix-7 XC7A200T-2FBG676 FPGA used for all implementations.	76
6.2	Performance metrics for $N = 256$ (9-bit Opt)	86
6.3	Performance metrics for $N = 1024$ (9-bit Opt)	86
6.4	Performance metrics for $N = 4096$ (9-bit Opt)	87
6.5	Performance metrics for $N = 256$ Radix-2 (9-bit)	87
6.6	Performance metrics for $N = 256$ Radix-2 (13-bit)	88
6.7	Comparison of FFT Processors	89

Abbreviations

BRAM	Block Random Access Memory
CSA	Carry-Save Adder
CSAM	Carry-Save Array Multiplier
CSHM	Computation-Sharing Hardware Multiplier
dB	Decibel
DFT	Discrete Fourier Transform
DSP	Digital Signal Processing
FFT	Fast Fourier Transform
FPGA	Field-Programmable Gate Array
FT	Fourier Transform
LUT	Look-Up Table
LUTRAM	Look-Up Table Random Access Memory
MAC	Multiply-Accumulate
MDF	Multi-Path Delay Feedback
SDF	Single-Path Delay Feedback
SQNR	Signal-to-Quantization-Noise Ratio
SRFFT	Split-Radix Fast Fourier Transform
DIT	Decimation-In-Time
DIF	Decimation-In-Frequency
FIFO	First-In, First-Out
FSM	Finite State Machine
MAC	Multiply-Accumulate
MSB	Most Significant Bit
SRL	Shift Register Look-Up Table
RTL	Register Transfer Level

Chapter 1

Introduction

Frequency-domain analysis sits at the core of almost every modern signal processing system, from wireless communication and radar receivers to audio and image coding, medical imaging and scientific instrumentation, and the transform that makes it practical is in most cases not one computation among many but the dominant one, repeated continuously on a stream of incoming samples at a rate the system cannot choose. Increasingly this processing happens on embedded and edge platforms, where area, power and thermal budgets are limited and where the transform must share a device with the rest of the signal chain rather than occupy it alone; reconfigurable hardware fits this position naturally, offering the parallelism and deterministic timing of dedicated logic without the cost and rigidity of a fixed-function circuit. However, the design process is not straightforward, as different architectural choices introduce trade-offs that can adversely impact efficiency, operating speed, and numerical accuracy.

1.1 Problem Statement

This thesis presents the design and hardware implementation of area and power efficient Fast Fourier Transform (FFT) architectures on an FPGA platform. To facilitate real-time computation, the proposed designs are optimized to maximize operating frequency while minimizing hardware area and latency. To achieve this, three distinct FFT algorithms—Radix-2, Radix-4, and Split-Radix—are evaluated to exploit their unique architectural characteristics. Dedicated on-chip DSP slices and Block RAMs (BRAMs) are utilized to minimize logic utilization and enhance timing performance. Finally, logic-only implementations utilizing

custom multipliers are realized to evaluate their trade-offs against dedicated FPGA silicon resources.

1.1.1 Research Objectives and Contributions

This thesis provides the following contributions:

- The implementation of three FFT algorithms (Radix-2, Radix-4, and Split-Radix) on FPGA hardware utilizing the multi-path delay feedback (MDF) architecture.
- The mapping of the memory access patterns for Radix-4 and Split-Radix into the minimal number of Block RAMs (BRAMs), alongside a comparative evaluation against distributed-logic storage such as shift registers.
- The integration of the Computation Sharing Multiplier (CSHM) for FFT twiddle-factor multiplication and its comparison against conventional Carry-Save multiplier schemes.
- The exploration of design trade-offs ranging from FFT decomposition type to datapath optimization to establish the optimal configuration for targeted application constraints.

1.2 Thesis Structure

The rest of this thesis is organized as follows:

Chapter 2 provides the necessary theoretical background required for the subsequent implementations. It details the underlying FFT decompositions and algorithms, contrasts the SDF and MDF and unrolled architectures, and covers fixed-point arithmetic, the custom multiplier schemes employed, and the target FPGA primitives.

Chapter 3 details the shared architectural design choices across all proposed algorithms before exploring the specific implementation details of each variant. It examines pipelining strategies, memory configurations, control unit designs, dedicated butterfly computation flows, and synthesis results under an ISO clock constraint of 10 ns. Finally, it presents dedicated optimization techniques targeting on-chip FPGA primitives.

Chapter 4 describes the behavioral verification flow and presents a comprehensive accuracy analysis across all implementations implemented up to this point.

Chapter 5 presents the architectures of the custom multipliers evaluated in this work, specifically the Carry-Save array and the CSHM. It describes the pipelining strategy employed for the Carry-Save multiplier, alongside the design exploration and methodology behind integrating the CSHM with FFT twiddle factors.

Chapter 6 presents the implementation results for all proposed architectures across transform sizes of 256, 1024, and 4096 points, evaluated at twiddle factor widths of 9, 13, and 16 bits. It examines the impact of varied architectural parameters, including the complex multiplication scheme, memory configuration, multiplier architecture, and datapath optimizations. Furthermore, it highlights the operational performance of each design through an evaluation of latency and processing throughput.

Chapter 7 concludes the thesis by summarizing the key findings and contributions of this research, alongside outlining potential future work.

Chapter 2

Background and Theoretical Foundations

2.1 Introduction to Fourier Transform

In mathematics, the Fourier transform (FT) is an integral transform that acts on a function to reveal its frequency components. It takes an input function and generates a new, complex-valued function that details the intensity of different frequencies present in the original. The term “Fourier transform” applies to both the mathematical process itself and the resulting function. To distinguish between the two, the output is often referred to as the “frequency domain” representation.

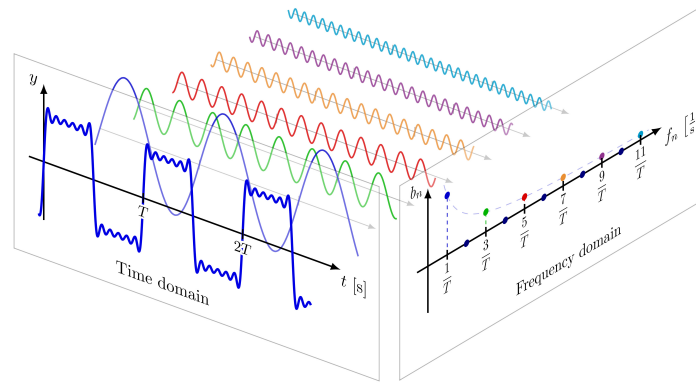


Figure 2.1: Representation of a time-domain signal decomposed into multiple signals of different frequencies.

The Fourier transform, $\hat{f}(\omega)$, of a function $f(t)$ is defined mathematically as:

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(t)e^{-i\omega t} dt \quad (2.1)$$

2.2 Fast Fourier Transform in General

The Fast Fourier Transform (FFT) is an efficient algorithm used to compute the Discrete Fourier Transform (DFT) of a sequence, which transforms a signal from the time domain into the frequency domain. This means it breaks down a complex signal into its constituent sinusoidal components, frequencies and their amplitudes, making it easier to analyze patterns such as pitch in audio, frequencies in communications, or periodicities in data. The FFT drastically reduces the computational complexity of DFT from $O(N^2)$ to $O(N \log(N))$ by cleverly reusing calculations through a divide-and-conquer approach. This algorithm is ideal for real-life applications like audio processing, image analysis, wireless communication, and other biomedical and industrial applications.

The most common FFT algorithm is the **Cooley-Tukey** algorithm [1], which is widely used due to its simplicity and efficiency. It works best when the input size N is a power of 2. The most common versions of the algorithm are:

- **Radix-2:** The most basic and common version. Splits the DFT into two halves (even and odd indices) at each stage. Requires N to be a power of 2.
- **Radix-4:** More efficient than Radix-2 when N is a power of 4. Reduces the number of multiplications compared to Radix-2 by grouping four elements at a time.
- **Split-Radix:** This combines both Radix-2 and Radix-4 versions in order to improve computational efficiency.

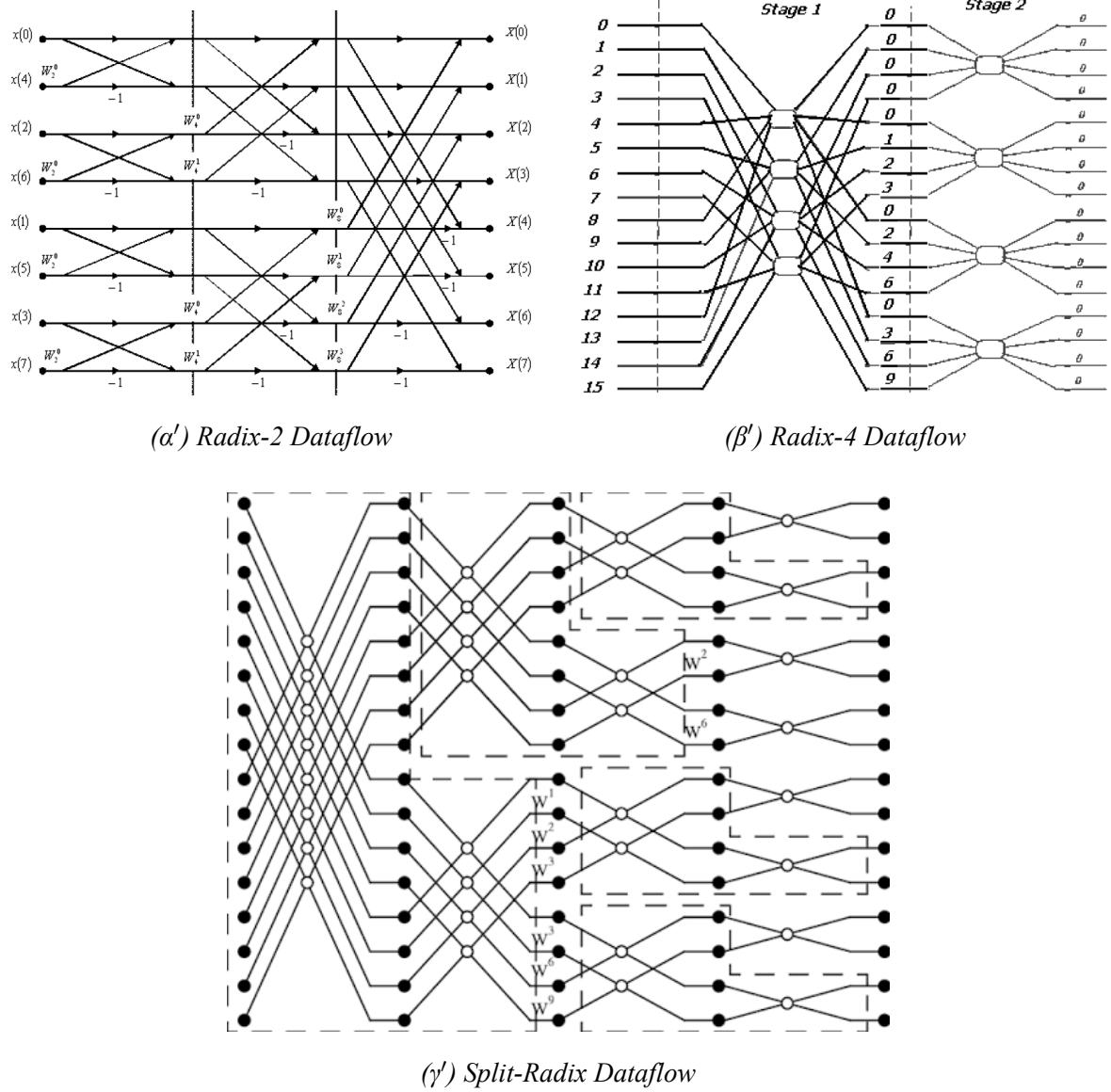


Figure 2.2: Comparison of Radix-2, Radix-4 and Split-Radix dataflow structures.

2.3 FFT Decomposition

2.3.1 Radix-2

Given a sequence $x[n]$ of length N , the Discrete Fourier Transform (DFT) is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot W_N^{kn}, \quad \text{where } W_N = e^{-j\frac{2\pi}{N}} \quad (2.2)$$

Here, W_N represents the **twiddle factor**. The primary advantage of the FFT algorithm is its ability to reorganize this summation, thereby drastically reducing the overall computational complexity.

By separating the input data $x[n]$ into even-indexed ($e[n] = x[2n]$) and odd-indexed ($o[n] = x[2n + 1]$) elements, the DFT can be expressed as:

$$X[k] = \sum_{n=0}^{N/2-1} x[2n] \cdot W_N^{k \cdot 2n} + \sum_{n=0}^{N/2-1} x[2n + 1] \cdot W_N^{k \cdot (2n+1)} \quad (2.3)$$

Applying the substitution $W_N^{2kn} = W_{N/2}^{kn}$, we can factor the equation into two smaller transforms:

$$X[k] = \underbrace{\sum_{n=0}^{N/2-1} e[n] \cdot W_{N/2}^{kn}}_{\text{Even terms } (E[k])} + W_N^k \cdot \underbrace{\sum_{n=0}^{N/2-1} o[n] \cdot W_{N/2}^{kn}}_{\text{Odd terms } (O[k])} \quad (2.4)$$

Let $E[k]$ and $O[k]$ represent the DFTs of the even and odd sequences, respectively. This reduces the main equation to:

$$X[k] = E[k] + W_N^k \cdot O[k] \quad (2.5)$$

Exploiting the N -periodicity of the DFT, the complete range for $k = 0, 1, \dots, N - 1$ is evaluated using the standard **butterfly operation**:

$$X[k] = E[k] + W_N^k \cdot O[k] \quad (2.6)$$

$$X\left[k + \frac{N}{2}\right] = E[k] - W_N^k \cdot O[k] \quad (2.7)$$

for $k = 0, 1, \dots, N/2 - 1$. For an input sequence where $N = 2^m$, this algorithm completes the transform in $m = \log_2(N)$ stages by recursively applying these butterfly operations.

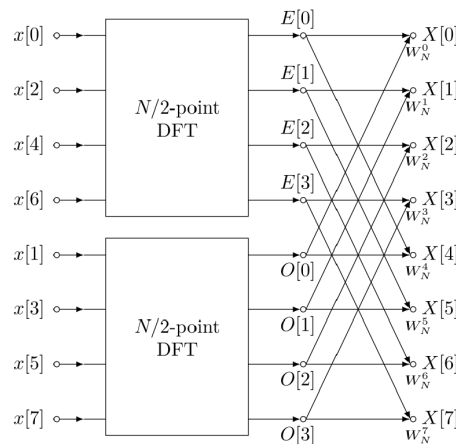


Figure 2.3: Recursive logic of the Radix-2 FFT, demonstrating the combination of even and odd sequences from previous stages.

2.3.2 Radix-4

The Radix-4 FFT builds upon the fundamental principles of Radix-2 by partitioning the DFT into **four smaller DFTs**. This requires the input sequence length N to be a power of 4 (i.e., $N = 4^m$).

The input sequence $x[n]$ is divided into four interleaved subsequences:

$$\begin{aligned} x_0[n] &= x[4n], & x_1[n] &= x[4n + 1], \\ x_2[n] &= x[4n + 2], & x_3[n] &= x[4n + 3] \end{aligned} \quad (2.8)$$

Expanding the DFT with these subsequences yields:

$$X[k] = \sum_{n=0}^{N/4-1} [x[4n] + x[4n + 1]W_N^k + x[4n + 2]W_N^{2k} + x[4n + 3]W_N^{3k}] W_{N/4}^{kn} \quad (2.9)$$

This equation effectively acts as a linear combination of four $N/4$ -point DFTs, denoted as X_0, X_1, X_2 , and X_3 :

$$X[k] = X_0[k] + W_N^k X_1[k] + W_N^{2k} X_2[k] + W_N^{3k} X_3[k] \quad (2.10)$$

To streamline the 4-point butterfly calculation, we define four intermediate variables:

$$A = X_0[k], \quad B = W_N^k X_1[k], \quad C = W_N^{2k} X_2[k], \quad D = W_N^{3k} X_3[k] \quad (2.11)$$

The resulting butterfly outputs are computed as follows:

$$X[k] = A + B + C + D \quad (2.12)$$

$$X\left[k + \frac{N}{4}\right] = A - jB - C + jD \quad (2.13)$$

$$X\left[k + \frac{N}{2}\right] = A - B + C - D \quad (2.14)$$

$$X\left[k + \frac{3N}{4}\right] = A + jB - C - jD \quad (2.15)$$

By leveraging additional symmetries, the Radix-4 algorithm lowers the total number of stages to $\log_4(N)$. This significantly reduces the multiplication count compared to Radix-2, making it highly efficient for hardware implementations when applicable.

2.3.3 Split-Radix

The **Split-Radix FFT (SRFFT)** combines the structures of Radix-2 and Radix-4 to achieve better computational efficiency. It relies on the mathematical observation that the even-indexed

terms of the DFT are best computed using a Radix-2 approach, while the odd-indexed terms are more efficiently handled using a Radix-4 approach. This method yields the lowest number of arithmetic operations for power-of-two FFTs.

The standard DFT is decomposed by applying a Radix-2 split to the even indices, and a Radix-4 split to the odd indices:

$$X[k] = \sum_{n=0}^{N/2-1} x[2n]W_{N/2}^{kn} + \sum_{n=0}^{N/4-1} x[4n+1]W_N^{k(4n+1)} + \sum_{n=0}^{N/4-1} x[4n+3]W_N^{k(4n+3)} \quad (2.16)$$

By factoring out the twiddle factors for the odd terms, the equation simplifies to:

$$X[k] = \sum_{n=0}^{N/2-1} x[2n]W_{N/2}^{kn} + W_N^k \sum_{n=0}^{N/4-1} x[4n+1]W_{N/4}^{kn} + W_N^{3k} \sum_{n=0}^{N/4-1} x[4n+3]W_{N/4}^{kn} \quad (2.17)$$

Let $E[k]$ be the $N/2$ -point DFT of the even elements, and let $O_1[k]$ and $O_3[k]$ represent the $N/4$ -point DFTs of the two interleaved odd sequences. The Split-Radix butterfly operation computes four outputs simultaneously using an L-shaped structure:

$$X[k] = E[k] + (W_N^k O_1[k] + W_N^{3k} O_3[k]) \quad (2.18)$$

$$X\left[k + \frac{N}{2}\right] = E[k] - (W_N^k O_1[k] + W_N^{3k} O_3[k]) \quad (2.19)$$

$$X\left[k + \frac{N}{4}\right] = E\left[k + \frac{N}{4}\right] - j(W_N^k O_1[k] - W_N^{3k} O_3[k]) \quad (2.20)$$

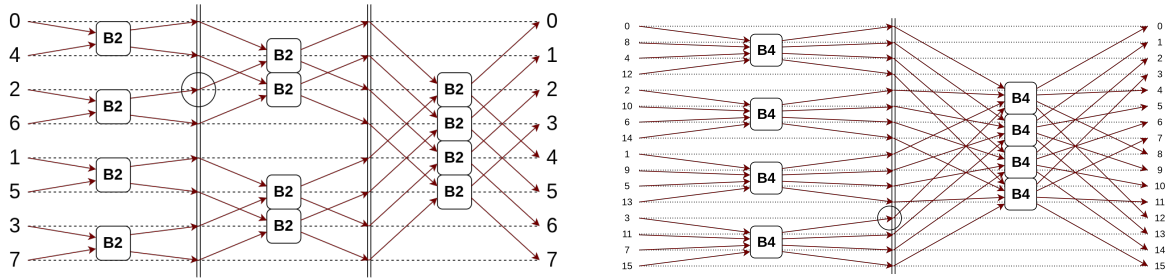
$$X\left[k + \frac{3N}{4}\right] = E\left[k + \frac{N}{4}\right] + j(W_N^k O_1[k] - W_N^{3k} O_3[k]) \quad (2.21)$$

Because it selectively applies the most efficient decomposition step at each stage, the Split-Radix algorithm requires up to one third fewer multiplications than standard Radix-2 [2], without placing the strict $N = 4^m$ size restriction required by pure Radix-4.

2.4 Unrolled and Single-Path Delay Feedback Approaches

There are different approaches to designing the FFT algorithm in hardware. The most intuitive method is the unrolled architecture, which exploits inherent parallelism by unrolling the entire computation. Each butterfly operation is assigned a dedicated hardware unit, resulting in a fully unrolled design where all butterfly computations are executed concurrently. This method is very demanding in resources, especially for smaller FPGAs. Furthermore, it requires a very high data bandwidth, as a 256-point FFT would typically require access to 256 data points in the same clock cycle. For example, if each complex sample is 32 bits

(16bits real and 16bits imaginary) long and the FFT operates at 100 MHz, the system requires a continuous bandwidth of 819.2 Gbps to run without stalling.



(a') Structure/dataflow of the Radix-2 unrolled algorithm (B2 = Butterfly 2).

(b') Structure/dataflow of the Radix-4 unrolled algorithm (B4 = Butterfly 4).

Figure 2.4: Comparison of unrolled architectures.

A highly resource-efficient alternative allocates only a single butterfly unit per FFT stage, employing delay lines or memory buffers to schedule the incoming data streams for computation. This approach is known as Single-Path Delay Feedback (SDF) when the datapath processes a single continuous stream of samples, or Multi-Path Delay Feedback (MDF) when it processes multiple parallel samples. Due to its optimal balance of high throughput and minimal hardware footprint, SDF remains the most widely adopted architecture for modern FFT cores.

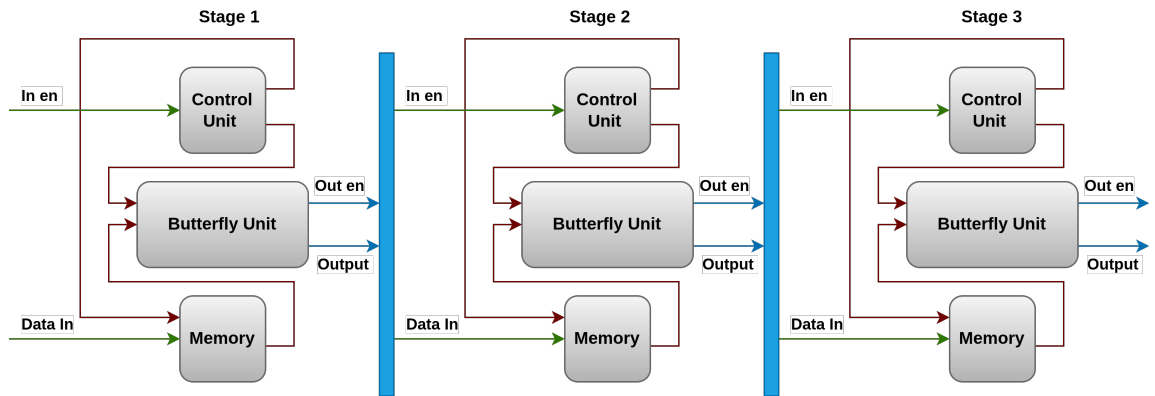


Figure 2.5: Simplified diagram of the SDF units discussed in this chapter.

2.5 Fixed-Point Arithmetic and Quantization Noise

In hardware implementations of the FFT, floating-point arithmetic is typically avoided due to its high resource consumption and latency. Instead, signals and twiddle factors are

represented using fixed-point formats, such as the Q format. For instance, a 32-bit data path might utilize a Q1.15 effective scaling, where a value of 1.0 corresponds to the integer 32767. The mathematical representation of this quantization scales a continuous signal $x(t)$ by a factor of $2^{W-1} - 1$, where W is the word length.

Because fixed-point variables have finite precision, rounding or truncation occurs after mathematical operations, particularly after complex multiplications with twiddle factors. This introduces a small amount of error known as quantization noise. This error accumulates with every successive stage of the FFT. The degradation is evaluated using the Signal-to-Quantization-Noise Ratio (SQNR), which compares the power of the original signal to the power of the quantization noise.

$$P_{x^\nu} = 10 \log_{10} \left(\frac{1}{N} \sum_{n=0}^{N-1} |x[n]|^2 \right)$$

$$\text{SQNR}|_{dB} = P_{x^\nu} + 6.02\nu + 4.77$$

Figure 2.6: Formulas for the signal power and Signal-to-Quantization-Noise Ratio (SQNR). P_{x^ν} represents the average power of the input signal. ν represents the number of bits (resolution).

To determine the absolute maximum SQNR for a given bit depth, we assume a full-scale square input wave, which provides a normalized linear average power of 1 ($10 \log_{10}(1) = 0$ dB). Consequently, for a 16-bit resolution, the maximum achievable SQNR is 101.09 dB. Table 2.1 demonstrates the theoretical maximum SQNR limits across common hardware bit depths.

Table 2.1: Maximum theoretical SQNR for various hardware bit depths (assuming full-scale square wave input).

Resolution (ν bits)	Maximum SQNR (dB)
8	52.93
12	77.01
16	101.09

2.6 Custom Multipliers

This thesis implements two distinct custom multiplier architectures: a generic carry-save array multiplier for general arithmetic operations and a specialized computation-sharing multiplier (CSHM) optimized for signal processing workloads.

2.6.1 Carry-Save Array Multiplier

A Carry-Save Array Multiplier (CSAM) accelerates arithmetic by eliminating the intermediate carry-propagation delays found in standard array multipliers. The operation consists of three main stages:

1. **Partial Product Generation:** A matrix of partial products is created using logical AND gates between the multiplicand and multiplier bits.
2. **Carry-Save Reduction:** Instead of rippling carries horizontally across rows, Carry-Save Adders (CSAs) are used. Each CSA passes independent sum and carry bits diagonally down to the next row, keeping the intermediate results as two unmerged vectors.
3. **Final Addition:** A traditional carry-propagate adder is used only at the very last stage to merge the final sum and carry vectors into the resolved product.

By breaking the horizontal carry chain during intermediate steps, the CSAM reduces the critical path delay while maintaining a highly regular hardware structure suitable for FPGAs.

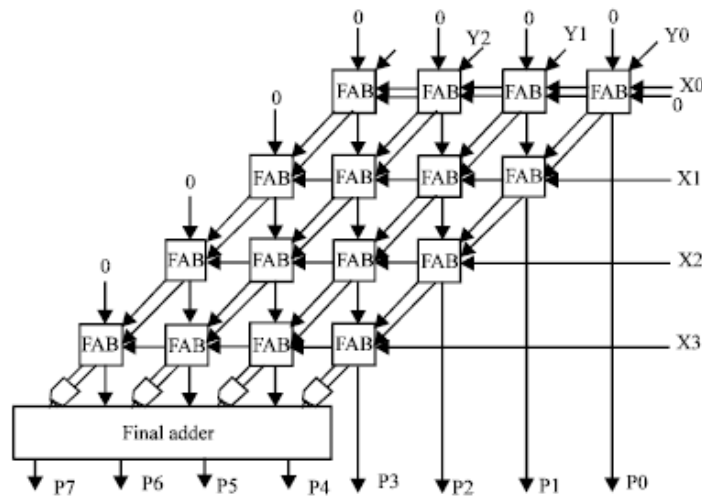


Figure 2.7: Top diagram of the Carry-Save Array Multiplier discussed in this chapter

2.6.2 Computation-Sharing Hardware Multiplier (CSHM)

The Computation-Sharing Hardware Multiplier (CSHM) minimizes hardware logic by identifying and reusing redundant partial products. It is highly efficient for signal processing tasks where a single input data stream is multiplied by various coefficients.

- **Alphabets and Sets:** CSHM simplifies multiplication by decomposing coefficients into a small set of recurring factor patterns, known as an *alphabet*. A common basis set consists of small odd integers, such as $\{1, 3, 5, 7\}$. The architecture precomputes the multiplicand against this limited set of values.
- **Addition and Assembly:** Generating the initial alphabet set requires minimal hardware (e.g., $3x$ is computed as $2x + x$, requiring just one shift and one adder). The final multiplication product is then reconstructed by selecting, shifting, and adding these pre-computed set values together. This computation-sharing approach drastically reduces the total number of resources and latency required compared to a classical multiplier.

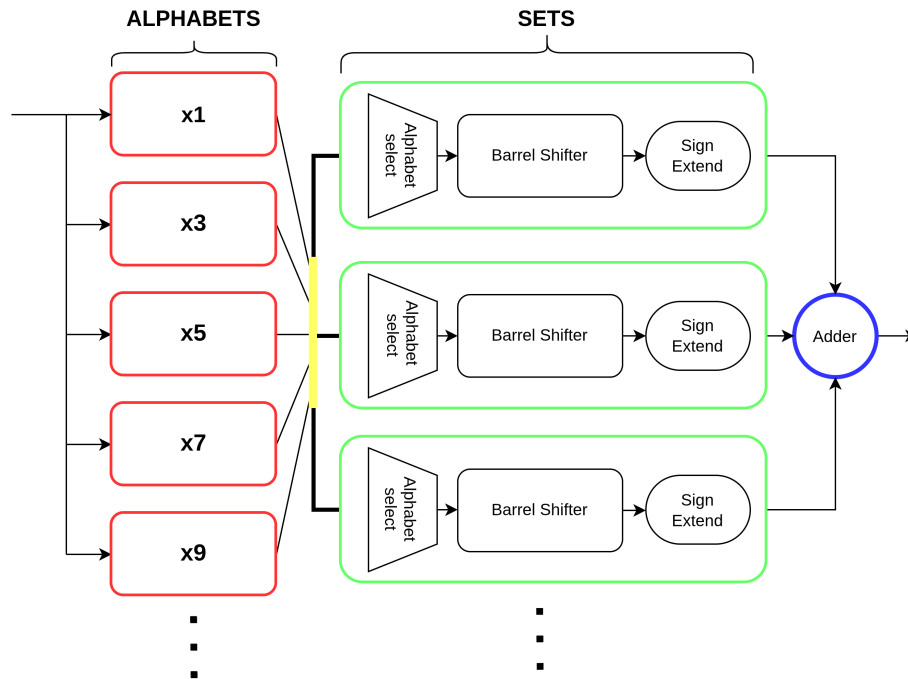


Figure 2.8: Top diagram of the CSHM Multiplier discussed in this chapter

- **Examples:**

Using a 2-Bit Set ($\{1, 3\}$): Consider multiplying an input x by the coefficient 27 (binary 011011_2). The coefficient is partitioned into 2-bit slices: 01_2 (1), 10_2 (2), and 11_2

(3). The multiplication is reconstructed as $(1x \cdot 2^4) + (1x \cdot 2^3) + (3x \cdot 2^0)$. The hardware routes and shifts the precomputed $1x$ and $3x$ values, avoiding full multipliers.

Using a 3-Bit Set ($\{1, 3, 5, 7\}$): Consider multiplying an input x by the coefficient 45 (binary 101101_2). The coefficient is partitioned into 3-bit slices: 101_2 (5) and 101_2 (5). The multiplication is reconstructed as $(5x \cdot 2^3) + (5x \cdot 2^0)$. The hardware only needs to compute the partial product $5x$ once, sharing that computation for both halves of the coefficient.

2.7 FPGA Hardware Resources

Modern Field-Programmable Gate Arrays (FPGAs) provide dedicated architectural primitives to accelerate digital signal processing algorithms and manage data streams efficiently. The hardware implementation of the proposed FFT and custom multipliers relies heavily on three core resources:

- **Digital Signal Processing (DSP) Blocks:** These are dedicated, hard-silicon arithmetic units embedded within the FPGA fabric (such as DSP48 slices in AMD/Xilinx architectures [3]). They are optimized to perform high-speed, wide-bit operations like multiply-accumulate (MAC) and pre-addition.
- **Block RAM (BRAM):** BRAMs are dedicated memory blocks [4] arranged in columns throughout the FPGA. They provide high-capacity, synchronous data storage and can be configured as single-port or dual-port memories.
- **Look-Up Table RAM (LUTRAM) / Distributed RAM:** LUTRAM is generated by the logic Look-Up Tables (LUTs) in the general logic fabric [4] to act as small memory elements. While it provides less capacity than a BRAM, LUTRAM is placed directly next to logic cells, making it efficient for implementing small memory structures.

Chapter 3

Hardware Implementation of the Proposed Algorithms

3.1 Introduction

This chapter presents the detailed hardware architectures developed for each FFT algorithm. It provides an in-depth analysis of critical design choices, including Block RAM (BRAM) allocation, DSP block utilization, delay register configuration, and pipelining strategies. Finally, it evaluates how these architectural decisions directly impact overall hardware resource utilization and computational latency.

3.2 Scalability

All implementations were designed with a strong emphasis on scalability. The core hardware module is fully parameterizable and capable of operating at any stage of the algorithms. Consequently, the overall design can easily scale its FFT size to any 2^m or 4^m points by simply instantiating and connecting the same hardware unit with the appropriate parameters.

3.3 Shared Architectural Elements

Before examining each algorithm individually, it is necessary to outline the design choices that are universally applied across all three implementations. Specifically, these shared architectural elements include the generation of twiddle factors, datapath optimization to minimize

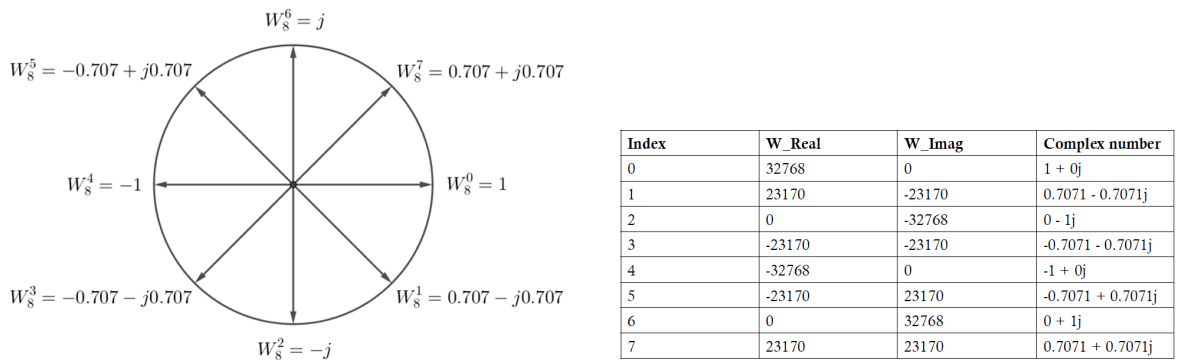
resource consumption, the application of different complex multiplier architectures and the datatypes used.

3.3.1 Twiddle Factors

Because the proposed hardware implementations are designed for FFTs of fixed sizes (although the underlying RTL is parameterizable, the synthesized modules target specific sizes), there is no need to compute twiddle factors dynamically at runtime. Calculating these rotational coefficients involves trigonometric operations that are computationally expensive and consume valuable FPGA logic resources. To optimize efficiency, all twiddle factors are precomputed at compile time and statically stored in on-chip memory.

Given that certain FFT architectures require simultaneous access to multiple twiddle factors, the choice of memory architecture is critical. The memory must provide sufficient read bandwidth to prevent stalling the datapath. To meet these high-throughput requirements, twiddle factors are implemented using read-only distributed memory (such as LUTRAM or dedicated flip-flops). This guarantees single-cycle, constant-time access, ensuring that the computational units can retrieve the required constants without delay.

For example, the Radix-4 algorithm requires three distinct twiddle factors simultaneously for each butterfly computation. Attempting to store these coefficients in BRAM would be highly inefficient and structurally limiting, as BRAM primitives are strictly restricted to dual-port (two-read) operations. In contrast, distributed read-only memory supports multi-port parallel reads and synthesizes very efficiently within the general FPGA logic fabric.



(a') Twiddle factors for $N=8$ represented on the unit circle.

(b') Twiddle values for $N=8$.

Figure 3.1: Twiddle factor visualization.

The twiddle factors are generated by sampling the complex exponential function, $W_N = e^{-j\frac{2\pi}{N}}$, at N discrete intervals, where N represents the size of the FFT. These ideal floating-point values are subsequently quantized to match the target fixed-point resolution used in the hardware datapath.

The intuitive approach to storing twiddle factors would be to utilize a single, centralized memory shared across all FFT stages. However, this is architecturally impractical for a highly pipelined design. First, standard BRAM primitives lack the multi-port capabilities required for simultaneous multi-stage access. Second, implementing a centralized memory using read-only LUTRAMs would require extensive logic replication to achieve the necessary read bandwidth, completely defeating the purpose of a unified memory structure. Consequently, the design employs a distributed memory architecture, allocating a dedicated memory module to each individual FFT stage. However, because different stages require different subsets of the twiddle factors, duplicating the entire N -point coefficient array at every stage would be highly inefficient. To optimize resource utilization, a custom generation script was developed that mimics the algorithmic dataflow of the FFT. This script precomputes and generates customized, stage-specific memory modules that contain only the exact coefficients required for that particular stage, drastically minimizing the overall memory footprint.

Table 3.1: Comparison of twiddle factor memory footprint between a naive duplicated storage approach and the proposed stage-specific optimized generation for an 8-stage Radix-2 FFT (DIT).

FFT Stage	Naive Storage (Duplicated)	Optimized Storage
Stage 1	128	1
Stage 2	128	2
Stage 3	128	4
Stage 4	128	8
Stage 5	128	16
Stage 6	128	32
Stage 7	128	64
Stage 8	128	128
Total Twiddle Factors	1024	255

This works similarly for the rest of the algorithms.

3.3.2 Datapaths

It is important to note that non-critical datapath registers are implemented without a hardware reset signal. This alleviates routing congestion across the FPGA fabric and theoretically enables the synthesis tool to achieve better timing and resource optimization.

3.3.3 Datatypes

Because the FFT processor is designed with parameterizable resolution, the datapath word length is dynamically determined by the selected twiddle factor size. This size establishes the $Q1.x$ fixed-point format for the system, where x represents the fractional bit width. The input data, assumed to be normalized within the range $(-1, 1)$, is initially quantized to this baseline resolution. However, as the data propagates through the core, the arithmetic operations within the butterfly units naturally expand the required bit width.

A common technique to manage this expansion is to scale down or truncate the output at each stage to maintain a strictly constant word length. However, this approach severely degrades the computational accuracy of the processor, particularly for high-order FFTs, resulting in a Signal-to-Quantization-Noise Ratio (SQNR) degradation of approximately 6 dB per additional stage [5] [6].

To preserve numerical precision while remaining scalable to large transform sizes, the proposed architecture employs a progressive bit-growth strategy. The datapath width is allowed to increase at each stage to accommodate the exact number of sequential additions within the butterfly unit, guaranteeing overflow protection. For complex multiplications, the intermediate products are truncated to retain only the most significant bits (MSBs) corresponding to the established data width, balancing high precision with efficient hardware utilization.

3.3.4 Complex Multiplication Techniques

To provide flexibility in resource management, the proposed architecture is parameterizable, allowing the design to choose between two distinct complex multiplication techniques: the traditional direct method and the Gauss 3-multiplier method.

Let the complex input data be $A = a_{re} + ja_{im}$ and the complex twiddle factor be $W = w_{re} + jw_{im}$. The resulting complex product is $P = p_{re} + jp_{im}$.

Traditional Complex Multiplication

The traditional approach computes the complex product through direct algebraic expansion. The real and imaginary components are calculated as:

$$p_{re} = a_{re}w_{re} - a_{im}w_{im} \quad p_{im} = a_{re}w_{im} + a_{im}w_{re} \quad (3.1)$$

This implementation requires exactly four separate multiplication operations and two additions/subtractions. In hardware, this directly translates to four DSP blocks per complex multiplier unit.

Gauss 3-Multiplier Technique

To reduce the dependency on DSP blocks, the design implements the Carl Friedrich Gauss complex multiplication algorithm. By algebraically factoring the equations, this technique trades one multiplication for three additional addition operations.

First, three intermediate addition and subtraction values are precomputed:

$$S_a = a_{re} + a_{im} \quad S_w = w_{re} + w_{im} \quad D_w = w_{im} - w_{re} \quad (3.2)$$

Next, three partial products (m_1, m_2, m_3) are generated using only three multipliers:

$$m_1 = w_{re} \cdot S_a \quad m_2 = a_{re} \cdot D_w \quad m_3 = a_{im} \cdot S_w \quad (3.3)$$

Finally, the real and imaginary components of the product are assembled using two final additions:

$$p_{re} = m_1 - m_3 \quad p_{im} = m_1 + m_2 \quad (3.4)$$

While this architecture increases the adder count to five, it successfully reduces the multiplier count to three. This translates in a 25% reduction in DSP utilization per multiplier.

3.4 Radix-2 Implementation

3.4.1 Review of Prior Implementation

The initial implementation of an FFT processor was developed as a collaborative project during a Microprocessor Design course. This architecture consisted of a baseline Radix-2 Single-Path Delay Feedback (SDF) core, designed without pipelining and operating with a throughput of one sample per stage and a data width of 32 bits with a Q1.15 quantization.

The synthesis results for this legacy design are presented in the following table. Rather than focusing on the physical area of the design, this comparison emphasizes the critical path and overall processing latency. Both of these performance metrics see significant improvements in the implementations shown later in this work, by pipelining and increasing throughput.

Table 3.2: Synthesis results and latency for the baseline 256-point FFT architecture.

Metric	FFT-256
Critical Path (ns)	13.976
LUTs	3161
Registers	1576
DSPs	56
Latency (clock cycles)	259

3.4.2 Overview of Proposed Implementation

In the new implementation, the throughput is increased from one to two samples per clock cycle. Additionally, the design incorporates extensive pipelining, with a particular focus on the internal datapath of the butterfly unit to reduce the critical path delay. The processor employs a Decimation-in-Time (DIT) architecture, which inherently requires the input data to be supplied in bit-reversed order while generating outputs in natural sequential order. Its counterpart, the Decimation-in-Frequency (DIF) architecture, operates conversely by accepting natural-order inputs and producing bit-reversed outputs. Because both algorithms share the same computational complexity and hardware requirements, the decision to implement a DIT rather than a DIF processor is primarily dictated by the specific data sequencing constraints and integration requirements of the broader system.

3.4.3 Butterfly Radix-2

Algorithm 1 shows the operational flow of the RTL implementation for the pipelined butterfly unit. The process begins with the complex multiplication of input B and the twiddle factor W , utilizing one of the two selectable multiplier architectures. To ensure correct alignment for the final butterfly computation, adjustable delay registers synchronize input A with the latency of the multiplication stage. In the final phase, the product is scaled to

the target word length before executing the cross-over addition and subtraction operations. Concurrently, a valid signal is propagated through a matching delay pipeline to flag output readiness. In the first stage of any Radix-2 FFT, the twiddle factor is strictly $W = 1 + j0$. The architecture uses this mathematical property by replacing the complex multiplier with a trivial bit-shift.

Analyzing the arithmetic operations within the butterfly module, it is evident that the required bit growth between stages is 1 bit due to the single addition performed at the end of the butterfly.

Algorithm 1 Pipelined Radix-2 Butterfly

Require: Complex inputs A, B ; twiddle factor W ; *start* signal

Ensure: Complex outputs Out_1, Out_2 ; *done* signal

Step 1: Complex multiplication of B by W

- 1: **if** $stage = 1$ **then**
- 2: $B_{mult} \leftarrow B \ll (Tw_Width - 1)$ $\triangleright W = 1$: shift replaces the multiplier
- 3: **else if** $SimpleMult = 1$ **then**
- 4: $B_{mult} \leftarrow \text{TraditionalComplexMult}(B, W)$ $\triangleright 4$ mult, 2 add
- 5: **else**
- 6: $B_{mult} \leftarrow \text{GaussComplexMult}(B, W)$ $\triangleright 3$ mult, 5 add
- 7: **end if**

Step 2: Pipeline balancing

- 8: $A_{delayed} \leftarrow \text{Delay}(A, L_{mult})$ $\triangleright$ match multiplier latency
- 9: $done \leftarrow \text{Delay}(start, L_{total})$ $\triangleright$ match total pipeline latency

Step 3: Scaling and butterfly

- 10: $B_{scaled} \leftarrow \text{Truncate}(B_{mult}, Tw_Width - 1)$ $\triangleright$ remove twiddle scaling
 - 11: $Out_1 \leftarrow A_{delayed} + B_{scaled}$
 - 12: $Out_2 \leftarrow A_{delayed} - B_{scaled}$
-

3.4.4 First Stage

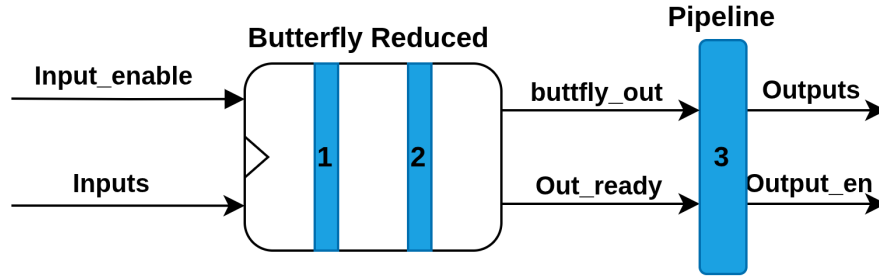


Figure 3.2: Radix-2 first stage top module.

The first stage is the simplest part of the processor design. Not only is the butterfly unit itself simplified, but the overall stage requires less hardware because no delay registers are needed to align the input data. This can be seen in the far-left stage of the Radix-2 dataflow in Figure 2.2. As a result, the hardware for this first stage consists entirely of just the simplified butterfly unit.

3.4.5 Control Unit

The control unit is responsible for coordinating where inputs are stored, when butterfly operations start, and how the delay buffers function. This is achieved using two counters of variable size, which depends on the stage number.

The first is the *stride segment counter*, which times the writing of new inputs to memory and tracks when the second counter should start. This counter increments every time four new samples arrive (shown as green lines in the graphs).

The second is the *butterfly operation counter*, which starts when the samples required for the first butterfly operation arrive. These are the blue samples in the graph, they represent the last samples of a butterfly cluster needed to complete the set of two inputs. Since only one physical butterfly unit exists, we cannot process all two operations at once even if we have the data. Therefore, only one operation happens at a time, and the other data must wait.

To make the design simpler and more portable, these counters do not iterate through all operations in a stage. Instead, each counter counts up to the number of butterflies in the respective cluster (in the graph, this number is 2). The length of these counters is calculated dynamically based on parameters from the SDF unit module that indicate the current stage.

Finally, there are other signals, such as the butterfly counter enable, which validates that the inputs and outputs of the butterfly are valid. There is also a flush counter that tracks the number of remaining operations when the input enable signal goes low. This is needed because the unit requires extra time to finish processing after the last input is received due to the initial delay while waiting for values to arrive.

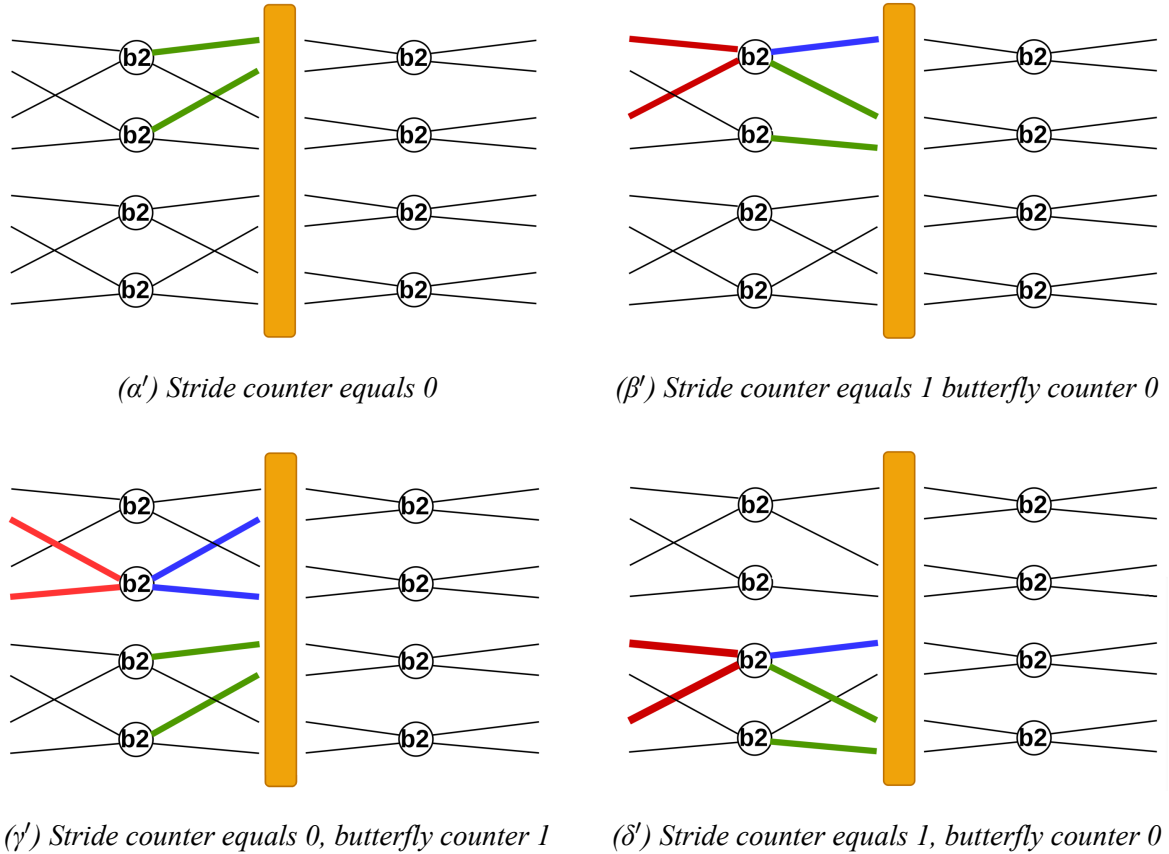


Figure 3.3: Operation flow of the counters managing the pairs of data points.

3.4.6 Delay Registers and Final Design

To properly align the incoming data, it must be stored temporarily. We achieve this by using simple delay buffers with an adjustable (parameterizable) depth. This allows the same buffer design to be reused across all stages, even though each stage requires a different latency.

For this design, only two delay registers are needed per stage. Their depths are calculated as:

- $\text{Depth_A} = 2^{\text{STAGE_NUM}-1}$

- $\text{Depth_B} = \text{Depth_A} \times 2$

Because the buffer depth doubles with each additional stage, the overall hardware area increases exponentially for larger FFT sizes. With the addition of these delay registers, the Stage module is complete. In the diagram, the output dataflow pipelines are highlighted in blue, and the input dataflow pipelines are highlighted in green.

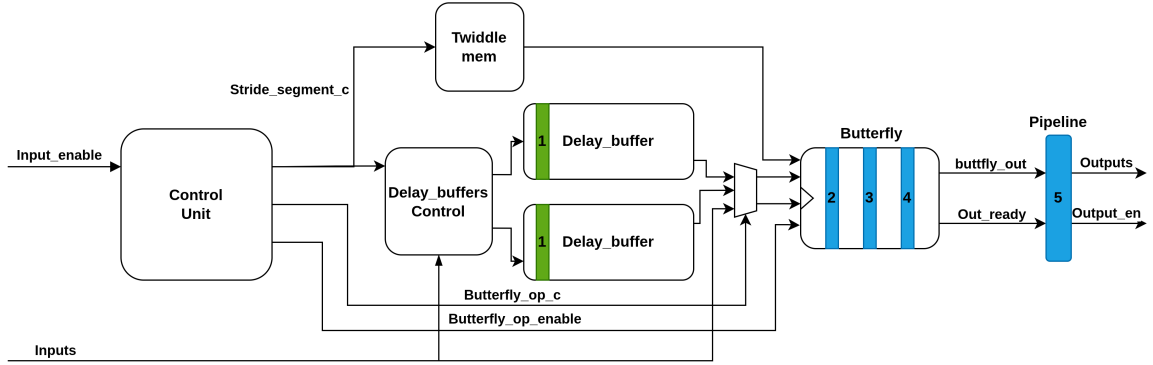


Figure 3.4: Radix-2 general stage unit top module.

In the top-level module, which connects all stage units to form a complete FFT, each stage's outputs are chained directly to the next stage's inputs. The first stage receives the input samples, and the final stage produces the results.

To accommodate bit growth, each stage instantiation can have a different data width. The top module calculates the maximum required bit width based on the input parameters, and the RTL dynamically scales the bit width for the specific stages that require it. For example, if the starting parameters specify a data width of 14 and a twiddle width of 10, the top module will only begin increasing the bit size after the fourth stage.

Because unoptimized widths can waste resources, the general rule for this design is that the starting data width should be equal to the twiddle width plus one (for Radix-2). This constraint is not enforced automatically in the RTL solely to provide greater flexibility for testing and simulating the design.

3.4.7 Implementation Results using an ISO Clock

These implementations utilize a generous clock frequency of 100 MHz (a 10 ns clock period), an 8-bit and 16-bit twiddle factor width to highlight the area differences between the implemented algorithm based on multiplier design, FFT size and twiddle width on an Artix-7

AC701 Evaluation Platform (xc7a200tfg676-2). Furthermore, the design can work for any twiddle widths but only the extremes will be shown (8 and 16) for presentation reasons, the intermediate twiddle sizes scale linearly between those values. Lastly, only powers-of-four FFT sizes are presented to ensure direct comparability with subsequent radix-4 designs.

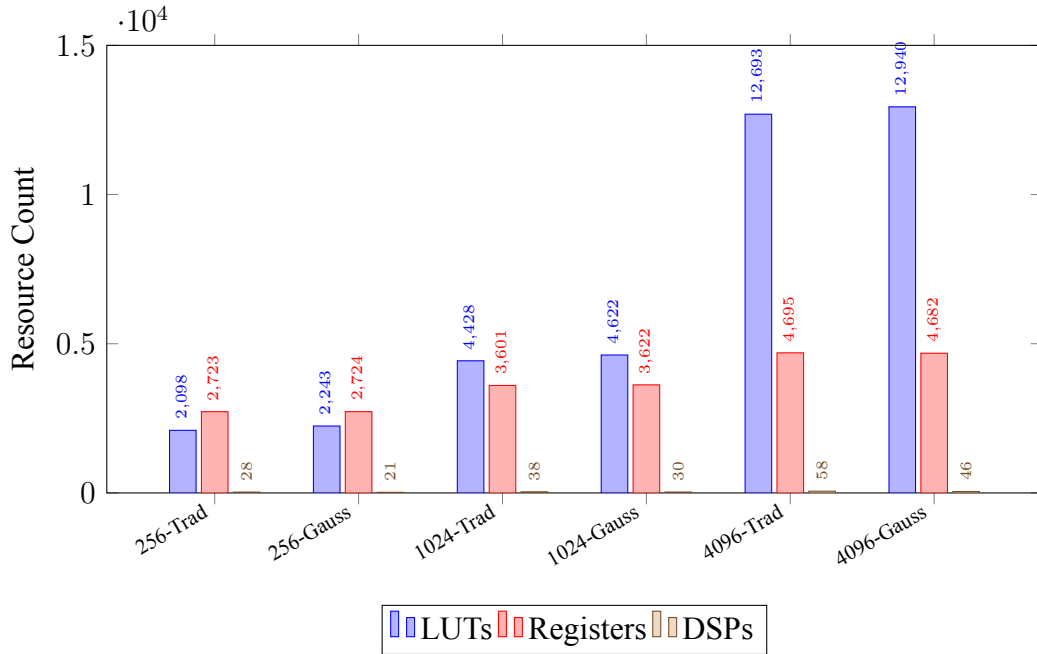


Figure 3.5: Utilization comparison for Radix-2 FFT implementations across sizes 256, 1024, and 4096 under ISO clock constraints (16-bit twiddle width).

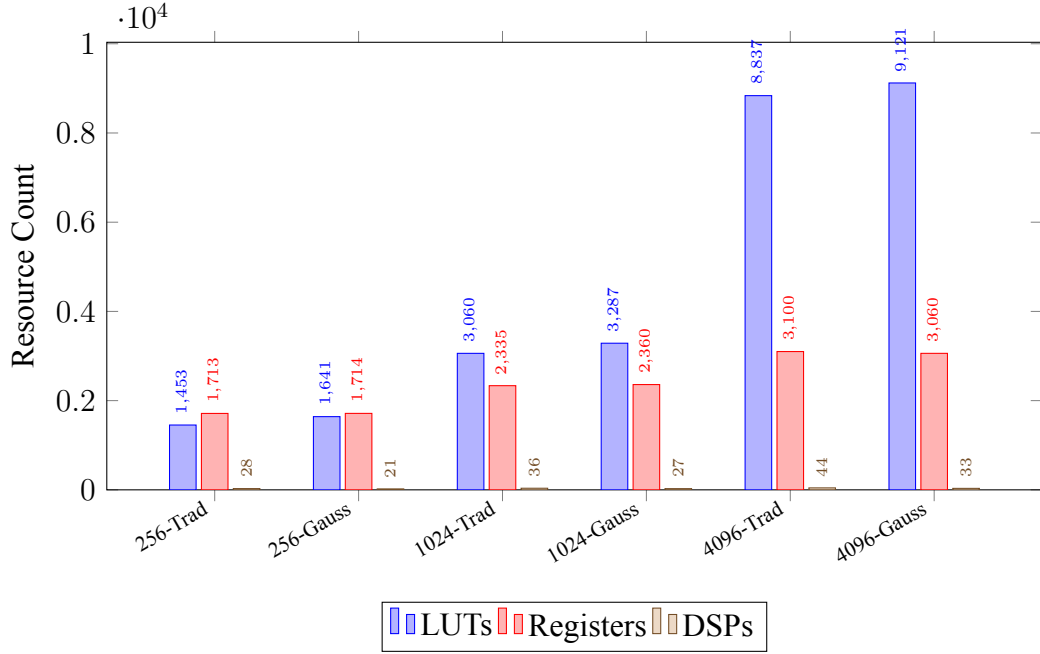


Figure 3.6: Utilization comparison for Radix-2 FFT implementations across sizes 256, 1024, and 4096 under ISO clock constraints (8-bit twiddle width).

Here we can see how reducing the bit width lowers the overall area of the design, and how the Gauss multiplier uses 3/4 of the DSPs as expected.

Table 3.3: FIFO Radix-2 Latency.

Metric	New-256	New-1024	New-4096	Old-256
Latency Trad (cycles)	165	559	2105	259
Latency Gauss (cycles)	172	568	2116	—

Due to the registers used for data alignment, the register growth for 4096 samples is substantial, and a closer inspection reveals that it doubles with each successive increase in FFT size. Furthermore, a comparison between the legacy 256-point and new 256-point FFT implementations shows a reduction in overall latency. Although the throughput is doubled, the latency is not reduced by half, this is caused by the extra pipelining introduced in the new stage unit, which adds a constant cycle penalty but ultimately yields significantly better timing results.

3.5 Radix-4 Implementation

3.5.1 Overview of Proposed Implementation

For the implementation of the Radix-4 algorithm, a methodology similar to that of the Radix-2 design is adopted. The architecture is configured to achieve the maximum throughput supported by the Radix-4 butterfly (four samples per clock cycle), while exploring alternative buffering strategies to efficiently store and align the data. As with the design, this implementation follows a Decimation-in-Time (DIT) structure, requiring the input sequence to be supplied in digit-reversed order (base-4 reversal).

3.5.2 Butterfly Radix-4

Algorithm 2 operates similarly to the Radix-2 butterfly, but is adapted for Radix-4 arithmetic. It reuses the same multiplication techniques and requires delay registers to align the A inputs with the outputs of the multipliers.

Analyzing the arithmetic operations within this butterfly module, it is evident that the required bit growth between stages is 2 bits, due to the two consecutive additions performed at the end of the butterfly.

3.5.3 First Stage

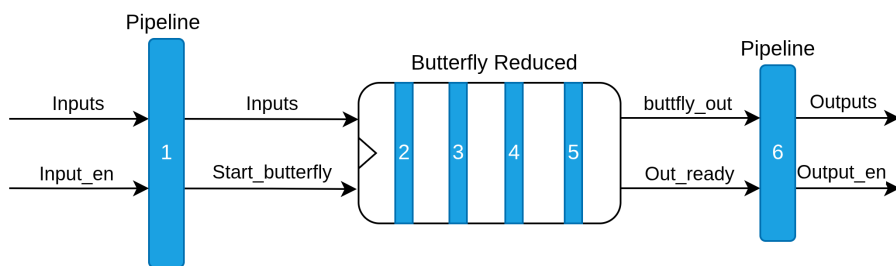


Figure 3.7: Radix-4 first stage top module.

The first stage is again the simplest part of the processor design. The butterfly unit itself is simplified, like in Radix-2, and the overall stage requires less hardware because no delay registers are needed to align the input data. This can be seen in the far-left stage of the Radix-4 dataflow in Figure 2.2. As a result, the hardware for this first stage consists entirely of just the simplified butterfly unit.

Algorithm 2 Pipelined Radix-4 Butterfly**Require:** Complex inputs A, B, C, D ; twiddle factors W_0, W_1, W_2 ; *start* signal**Ensure:** Complex outputs $Out_1, Out_2, Out_3, Out_4$; *done* signal**Step 1: Complex multiplication of B, C, D by W_0, W_1, W_2**

```

1: if  $stage\_num = 1$  then
2:    $B_{mult} \leftarrow B \ll (Tw\_Width - 1)$  ▷ Shift replaces multiplier
3:    $C_{mult} \leftarrow C \ll (Tw\_Width - 1)$   $D_{mult} \leftarrow D \ll (Tw\_Width - 1)$ 
4: else if  $SimpleMult = 1$  then
5:    $B_{mult} \leftarrow SimpleMult(B, W_0)$  ▷ Traditional complex multiplier
6:    $C_{mult} \leftarrow SimpleMult(C, W_1)$   $D_{mult} \leftarrow SimpleMult(D, W_2)$ 
7: else
8:    $B_{mult} \leftarrow CheapMult(B, W_0)$  ▷ Gauss complex multiplier
9:    $C_{mult} \leftarrow CheapMult(C, W_1)$   $D_{mult} \leftarrow CheapMult(D, W_2)$ 
10: end if

```

Step 2: Pipeline balancing

```

11:  $A_{delayed} \leftarrow Delay(A, delay\_mult + 1)$  ▷ Match multiplier latency + scale reg
12:  $done \leftarrow Delay(start, delay\_mult + 3)$  ▷ Match total pipeline latency

```

Step 3: Scaling

```

13:  $B_{scaled} \leftarrow Truncate(B_{mult}, Tw\_Width - 1)$  ▷ Remove twiddle scaling
14:  $C_{scaled} \leftarrow Truncate(C_{mult}, Tw\_Width - 1)$ 
15:  $D_{scaled} \leftarrow Truncate(D_{mult}, Tw\_Width - 1)$ 

```

Step 4: Butterfly Stage 1 (Intermediate variables)

```

16:  $t_0 \leftarrow A_{delayed} + C_{scaled}$ 
17:  $t_1 \leftarrow A_{delayed} - C_{scaled}$ 
18:  $t_2 \leftarrow B_{scaled} + D_{scaled}$ 
19:  $t_3 \leftarrow B_{scaled} - D_{scaled}$ 

```

Step 5: Butterfly Stage 2 (Outputs)

```

20:  $Out_1 \leftarrow t_0 + t_2$ 
21:  $Out_2 \leftarrow t_1 - j \cdot t_3$  ▷ Implemented as  $t_{1r} + t_{3i}$  and  $t_{1i} - t_{3r}$ 
22:  $Out_3 \leftarrow t_0 - t_2$ 
23:  $Out_4 \leftarrow t_1 + j \cdot t_3$  ▷ Implemented as  $t_{1r} - t_{3i}$  and  $t_{1i} + t_{3r}$ 

```

3.5.4 Control Unit

The control unit operates similarly to the Radix-2 implementation, but the internal counters are scaled to match the Radix-4 stage requirements, and butterfly execution is triggered upon the arrival of the fourth input sample rather than the second.

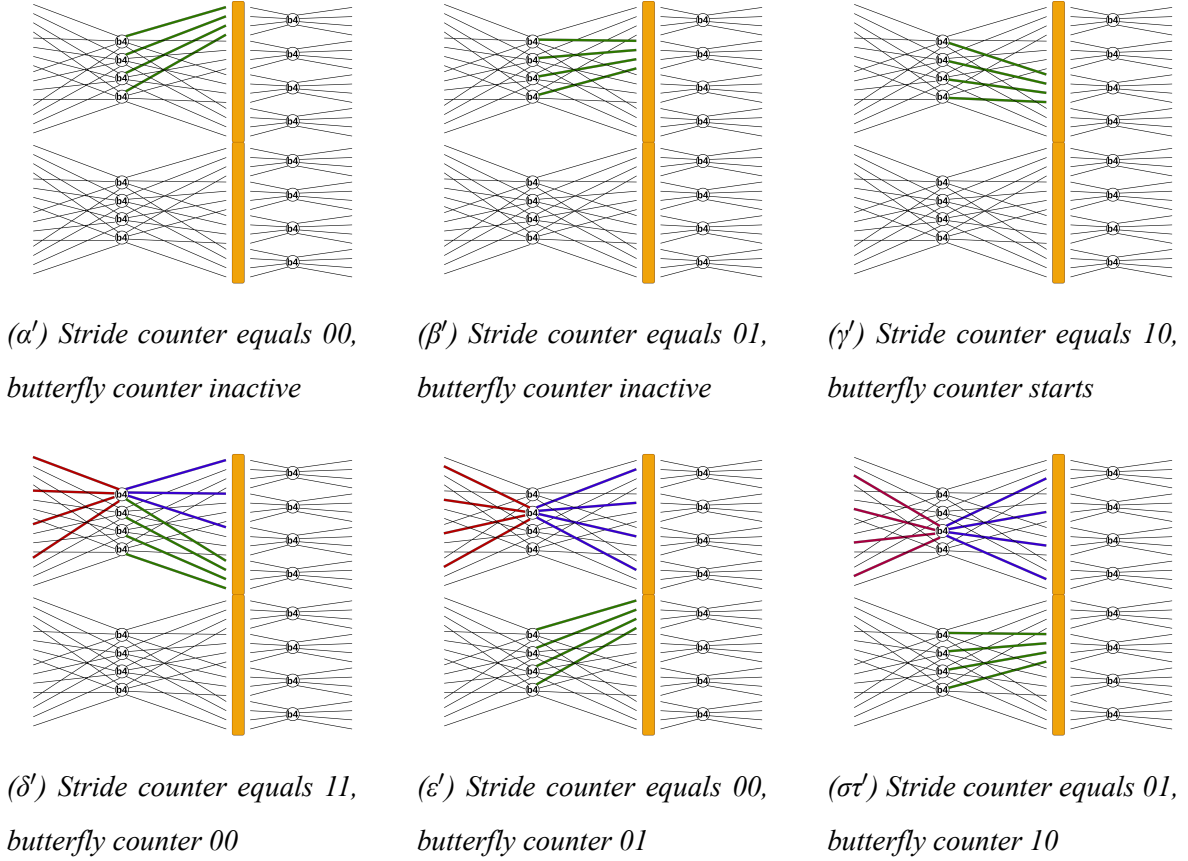


Figure 3.8: Operation flow of the counters managing the sets of four (quads) data points.

3.5.5 Initial Delay Register Architecture

The delay buffers are the most critical component of the design, as they facilitate the storing, rearranging, and timing of the data. They are essentially shift registers that delay the data by the correct number of cycles. Each buffer has four input ports and one output port. The inputs are always written to specific positions in the delay buffer (0, $\text{buffer_size}/4$, $2 \cdot \text{buffer_size}/4$, and $3 \cdot \text{buffer_size}/4$); this is necessary to queue the data correctly.

A `write_enable` signal is raised based on the *stride segment counter*, instructing the buffer to store the inputs. Similarly, `read_enable_1` and `read_enable_2` signals are raised based on the *butterfly enable counter* to retrieve data. In essence, these signals dictate the operating

mode of the shift registers for any given clock cycle.

Distinct modes are required to time the data correctly. Since the patterns of incoming and outgoing data differ, a simple shifting buffer would lead to data overwrites, as the unit sometimes needs to write to locations where valid data has not yet been computed. To solve this, I implemented a secondary shift register in each module. This second register temporarily stores incoming data that would otherwise overwrite useful entries. When the timing is right, the design rotates or combines the two shift registers according to the *stride segment counter*.

Although this strategy theoretically minimizes the number of registers required for data alignment and delay, the synthesis results were problematic. This outcome is likely due to the structural complexity of the delay registers, which prevented the synthesis tool from efficiently inferring standard shift-register primitives (such as SRL16/SRL32). From now on a new approach will be used.

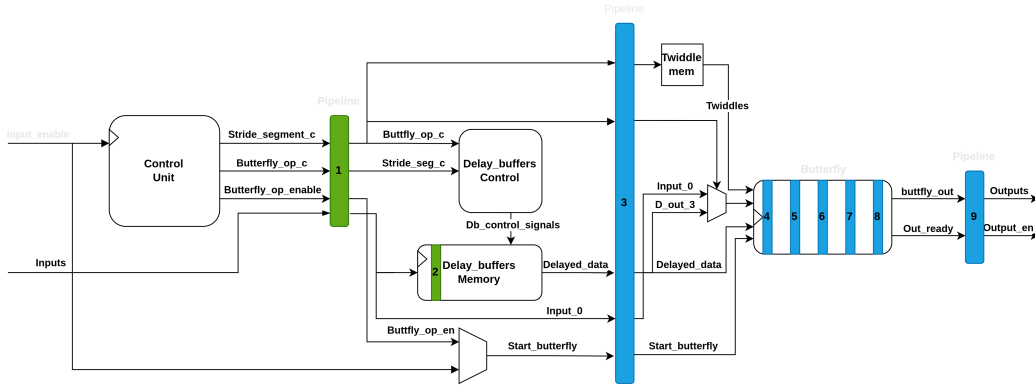


Figure 3.9: Top diagram Radix-4 stage unit delay registers.

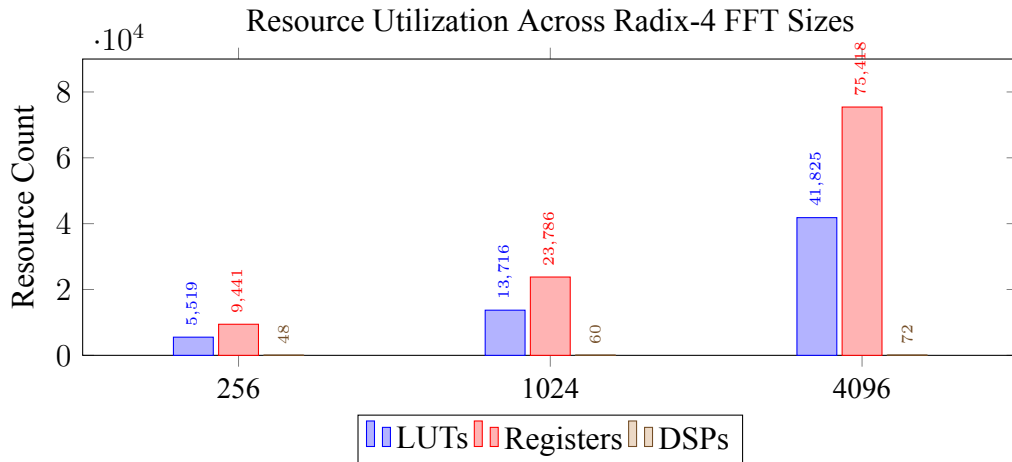


Figure 3.10: Utilization comparison for Radix-4 FFT implementations across sizes under ISO clock constraints.

3.5.6 Proposed BRAM-Compatible Memory Structure

The initial design of the memory and delay buffers was inefficient, as it shifted all data from register to register on every clock cycle. Although shift registers are typically simple and fast, the complexity of these specific buffers likely hindered the synthesis tool's optimization. To resolve this, I implemented a more traditional memory architecture where the data remains stationary, and read/write pointers manage the data organization. These pointers are controlled by the two counters discussed previously in the control unit.

Since this simpler memory design was more robust, I aimed to make it compatible with Block RAM (BRAM) modules. To achieve BRAM inference, I had to ensure that each memory block utilized no more than one write port and one read port (Simple Dual-Port) per physical BRAM resource. Given that the design requires four reads and four writes per cycle, I had to carefully structure the memory banks to ensure that no combination of simultaneous accesses exceeded the BRAM hardware capabilities. Consequently, instead of storing incoming samples in a linear sequence (e.g., $[0 \rightarrow 3]$, $[4 \rightarrow 7]$), I implemented a custom addressing algorithm.

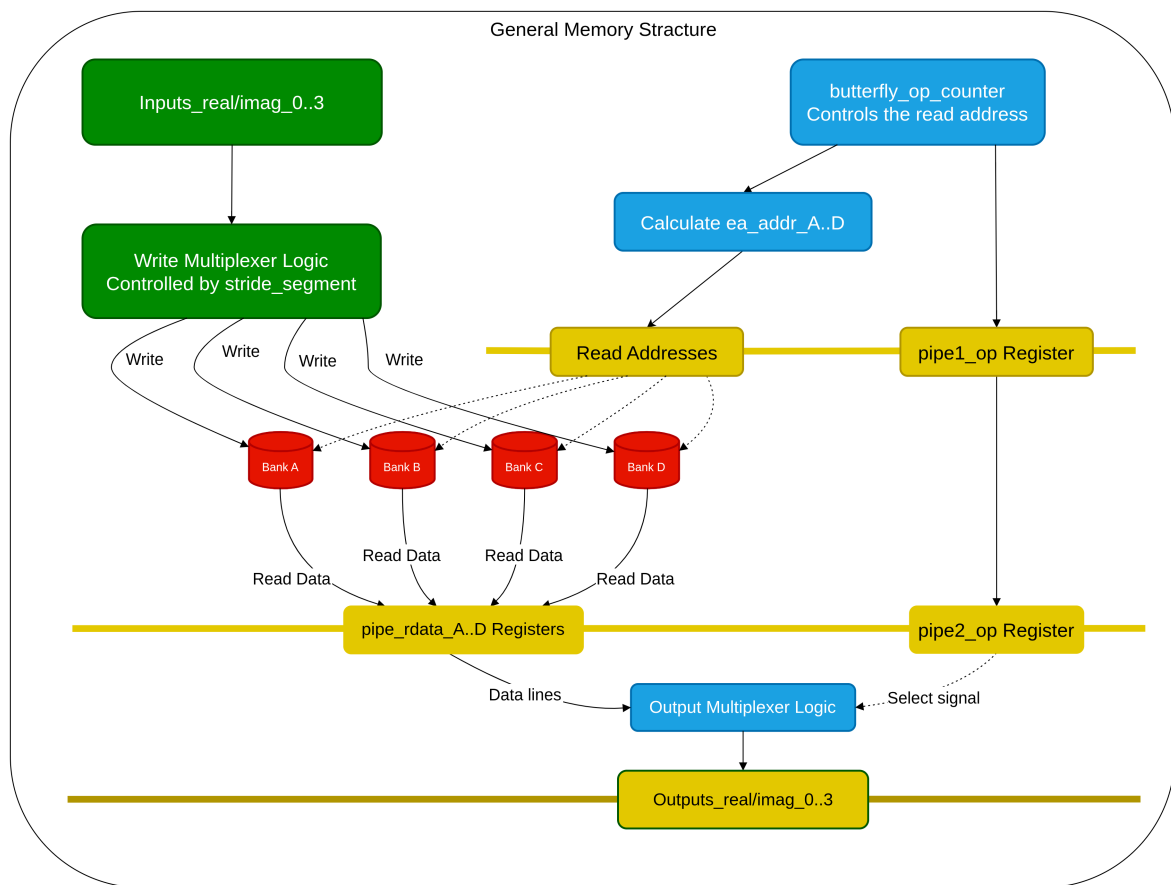


Figure 3.11: Memory module general Organization.

The following figure 3.12 describes the memory architecture and its operation. The numbers represent the arrival order of the samples, which are fed into the unit in batches of four (e.g., $[1 \rightarrow 4]$, $[5 \rightarrow 8]$, etc.). The memory is partitioned into four banks to enable simultaneous parallel reads and writes.

The green lines illustrate the write patterns, writing begins at the green pointer's head and concludes at the pointer's tail. The remaining lines represent the read patterns, which are designed to span across all four banks. By rotating or shifting the starting point of the write operation for every other line, I avoided a vertical alignment of the read patterns. Without this rotation, the system would attempt to read four values from a single bank simultaneously, which is impossible with standard dual-port BRAM. While increasing the number of banks could solve this, I chose this shifting approach to keep resource and register utilization as low as possible.

Lastly, because the memory input and output access patterns differ, incoming samples would overwrite unread data during concurrent read and write operations. To prevent this data hazard, a ping-pong buffering scheme was adopted, routing consecutive data windows to alternate memory banks. Consequently, this double-buffering approach requires twice the storage capacity for each bank.

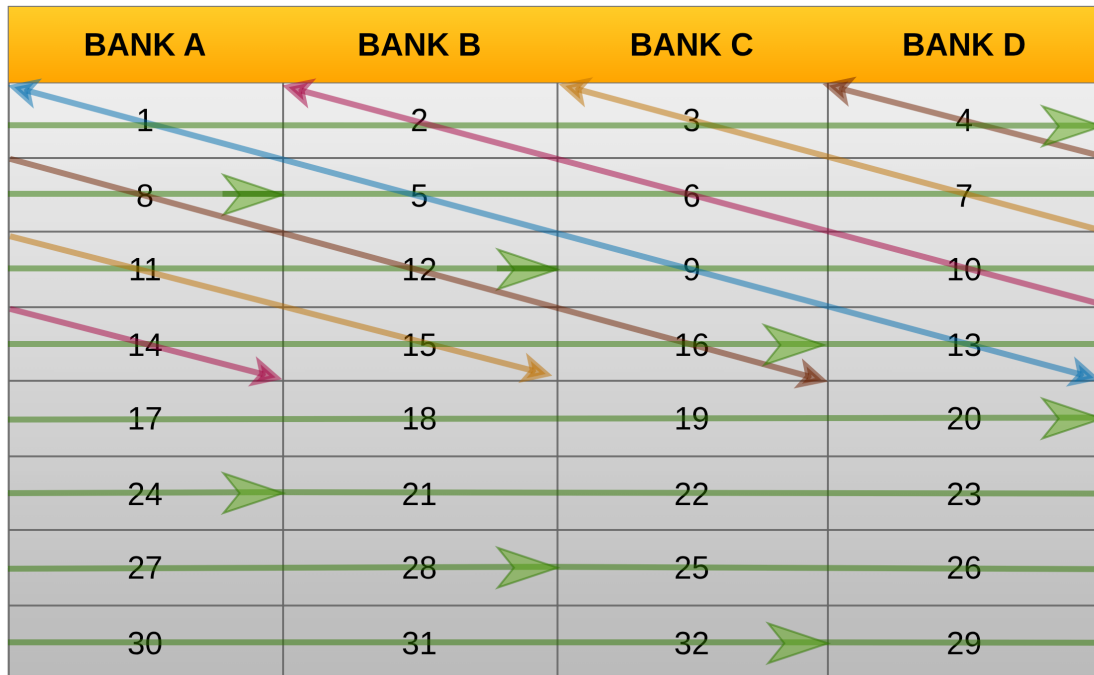


Figure 3.12: Memory module organization and access pattern.

3.5.7 Final Design and Results using an ISO Clock

The top-level module of the Radix-4 implementation follows the same structural organization as the Radix-2 design, the primary distinction lies in the stage-to-stage connections, which must accommodate a 2-bit growth per stage as previously established.

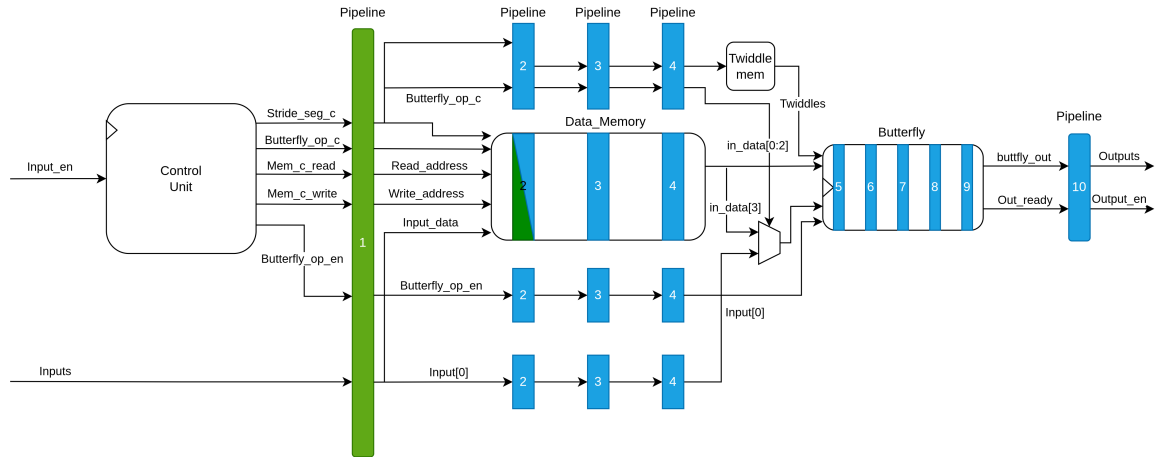


Figure 3.13: Top diagram Radix-4 stage unit with new memory.

These implementations use the same parameters as in the previous section of the radix-2 implementation.

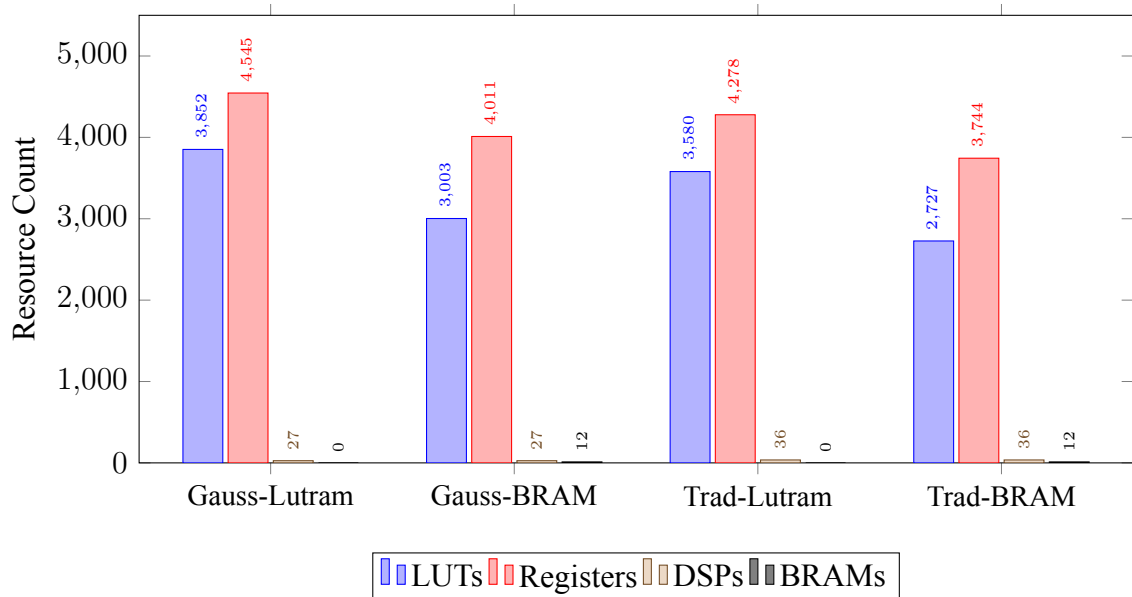


Figure 3.14: Resource utilization for Radix-4 FFT with new memory at $N = 256$ (16-bit).

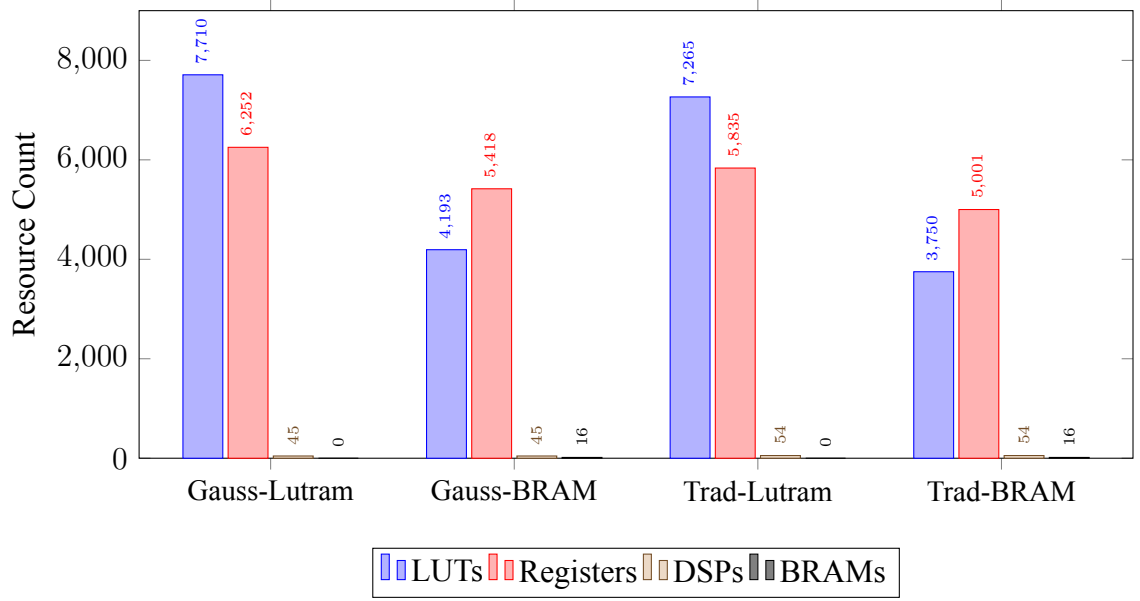


Figure 3.15: Resource utilization for Radix-4 FFT with new memory at $N = 1024$ (16-bit).

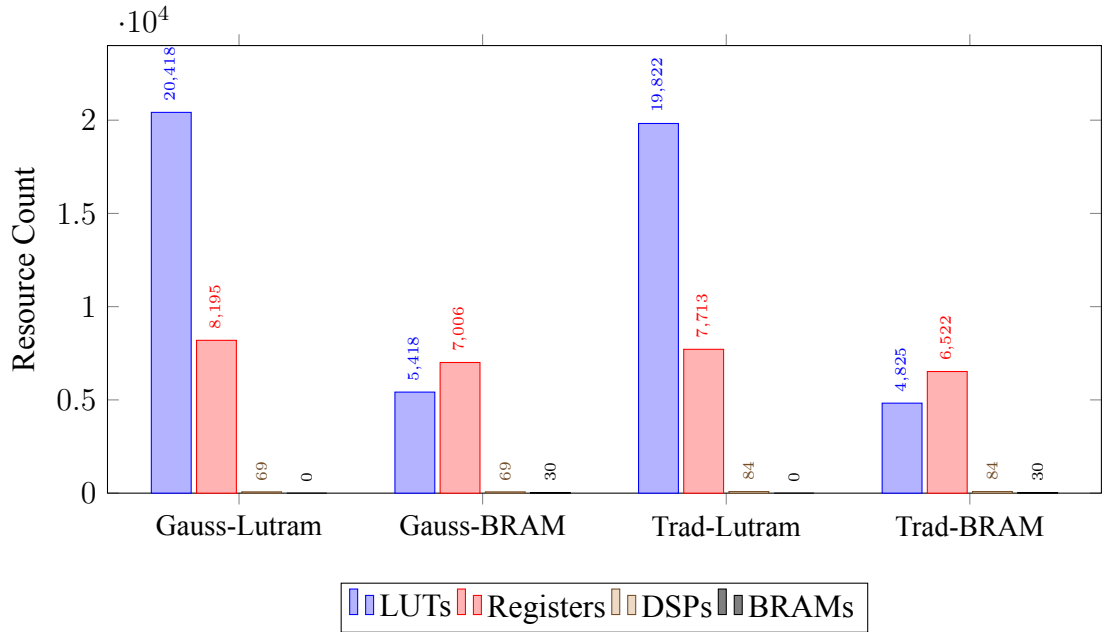


Figure 3.16: Resource utilization for Radix-4 FFT with new memory at $N = 4096$ (16-bit).

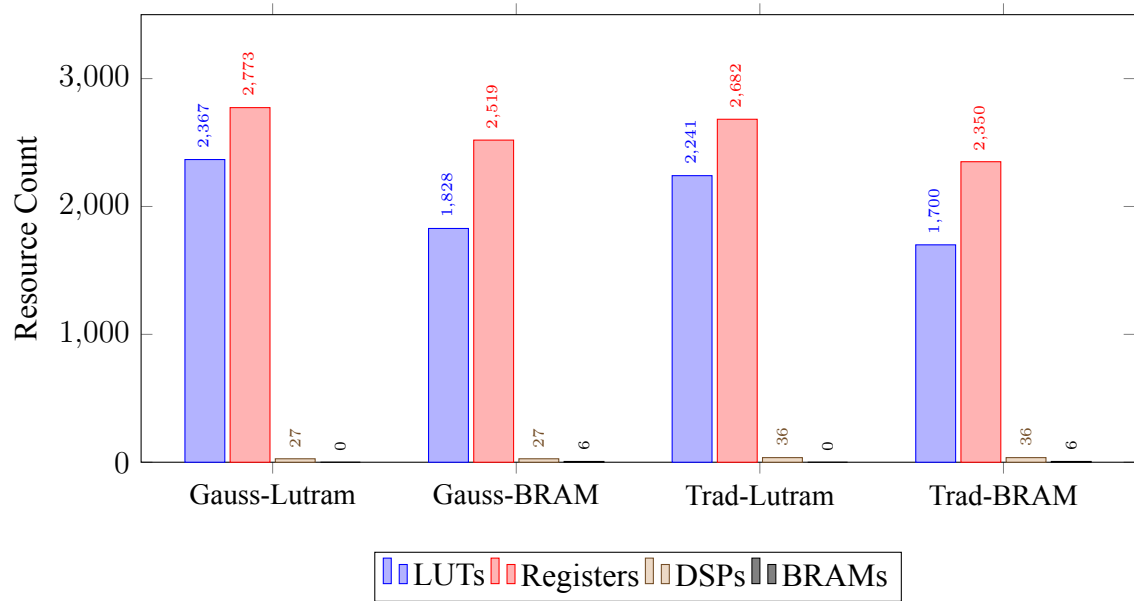


Figure 3.17: Resource utilization for Radix-4 FFT with new memory at $N = 256$ (8-bit).

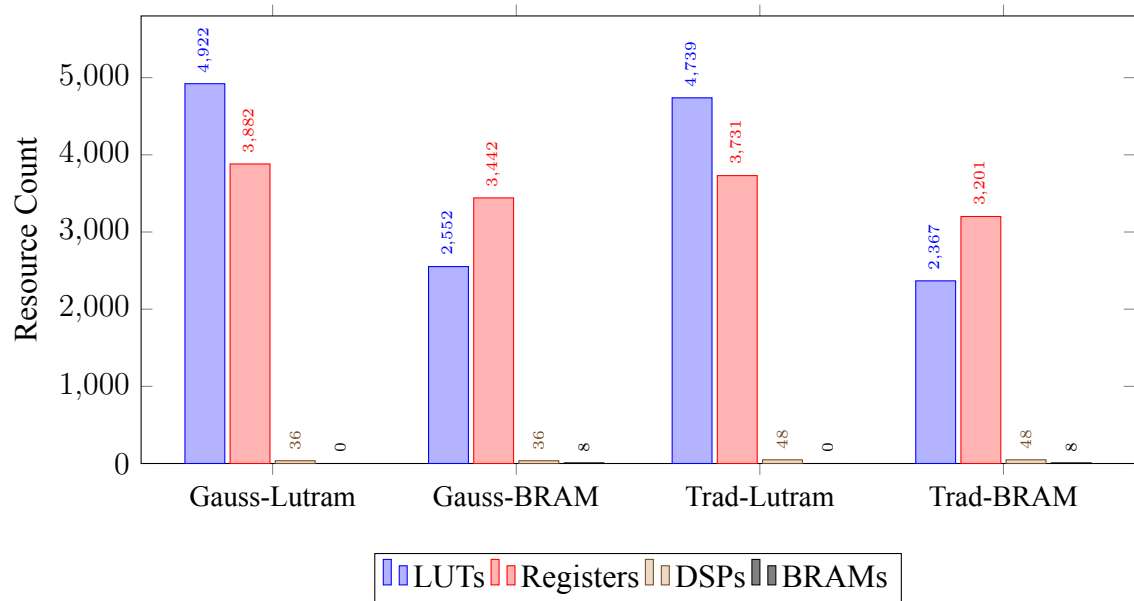


Figure 3.18: Resource utilization for Radix-4 FFT with new memory at $N = 1024$ (8-bit).

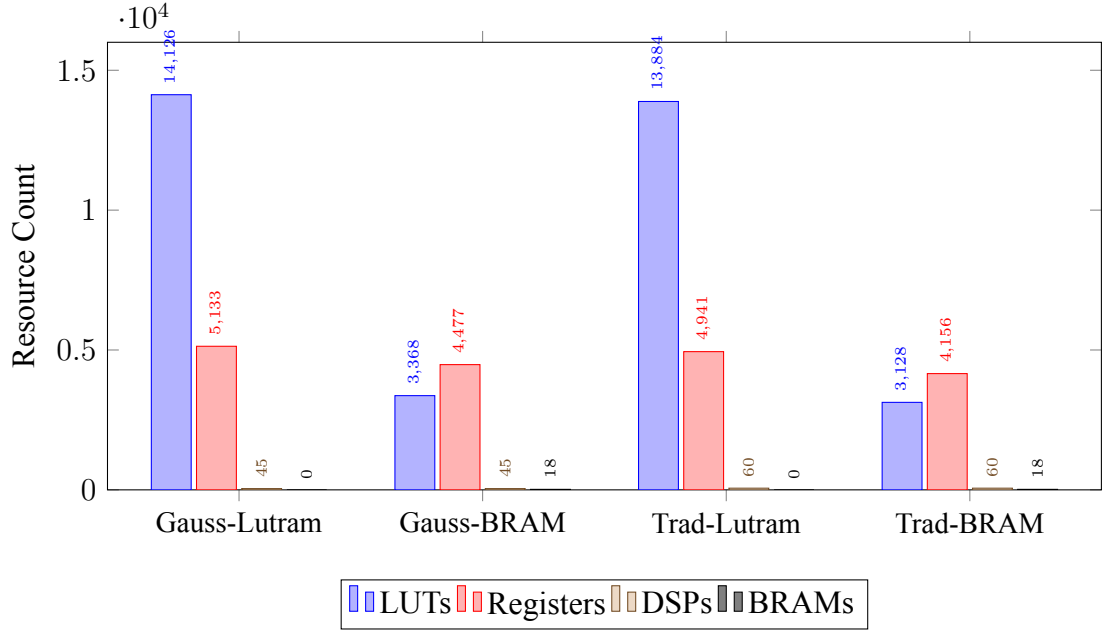


Figure 3.19: Resource utilization for Radix-4 FFT with new memory at $N = 4096$ (8-bit).

These results demonstrate that the redesigned memory architecture substantially outperforms the shift-register delay approach, even when using distributed RAM (LUTRAM). Utilizing dedicated Block RAMs (BRAMs) brings further reductions in logic resource consumption, with the efficiency difference between LUTRAM and BRAM based memory becoming more evident at larger FFT sizes.

Table 3.4: FIFO Radix-4 Latency.

Metric	256	1024	4096
Latency Trad (cycles)	98	300	1078
Latency Gauss (cycles)	101	304	1083

The results indicate that the latency of the Radix-4 implementation is approximately halved compared to the Radix-2 design 3.3, while simultaneously delivering double the throughput. The choice of Bram or Lutram does not change the latency of the design.

3.6 Split-Radix Implementation

3.6.1 Overview of Proposed Implementation

The implementation methodology for the split-radix algorithm largely follows the approach from the previous architectures. However, rather than sticking to the conventional flow which relies on irregular L-shaped butterfly units and is inherently throughput-constrained to at most two samples per clock cycle, a different approach was adopted from this paper [5]. This approach processes up to four samples per cycle while supporting power-of-two (and consequently power-of-four) transform sizes.

This regularized version was selected primarily because its uniform datapath maps better onto hardware, achieving latency and throughput metrics that remain competitive with the pure Radix-4 implementation. Furthermore, a Decimation-in-Frequency (DIF) decomposition was utilized, offering a more hardware-friendly control and routing structure. The numerical output and precision remain identical to the Decimation-in-Time (DIT) equivalent, with the only operational difference being that the input sequence is in natural order while the output is produced in bit-reversed order.

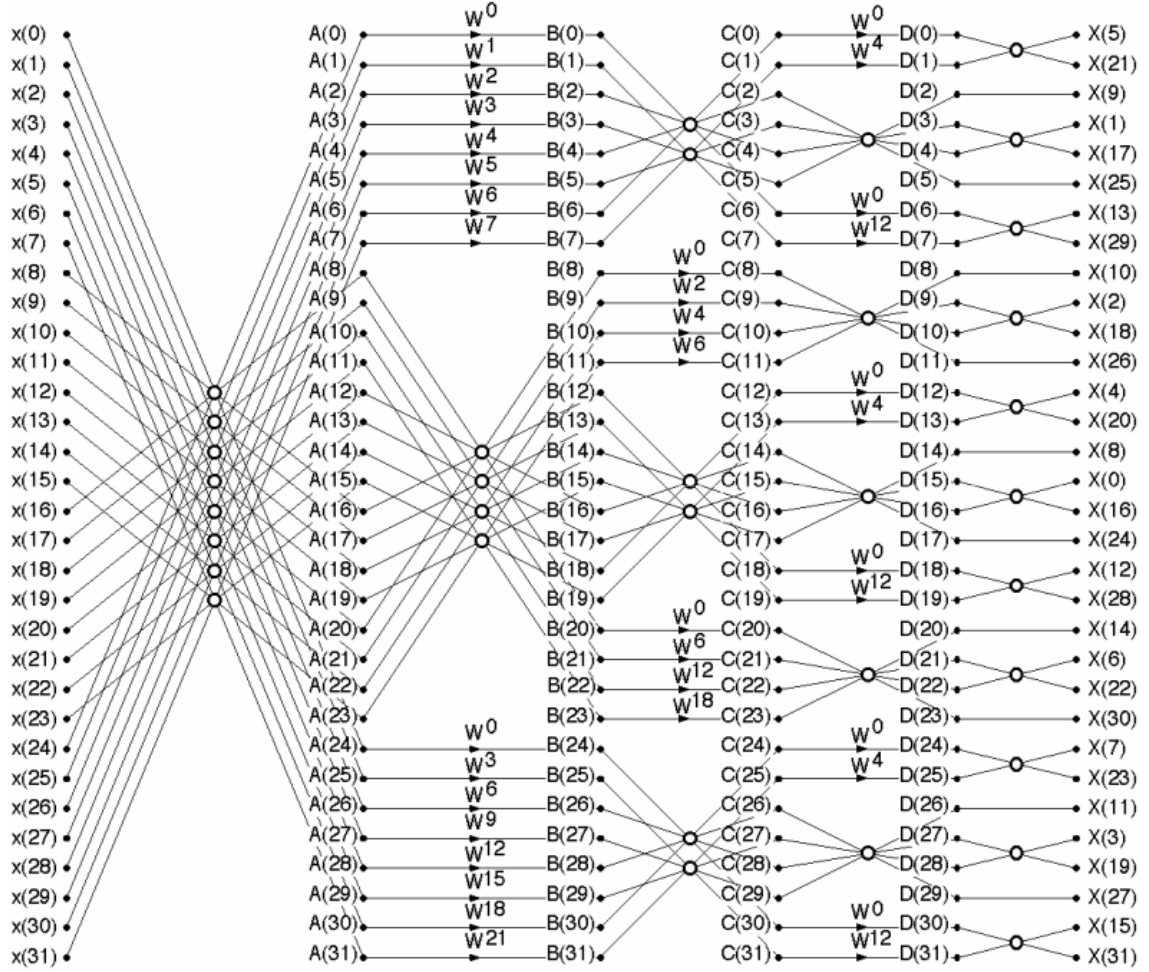


Figure 3.20: New Split-Radix Dataflow structure, example for an FFT size of 32.

To manage the architectural complexity of the split-radix algorithm, even within its new formulation, the datapath is structured into five distinct stage modules, each tailored to specific operational requirements. The initial stage, which only computes butterfly operations, closely resembles the Radix-4 stage architecture. The second stage integrates both butterfly units and complex multipliers. Following these are the penultimate and final stages, which complete the output decomposition. The intermediate third stage is the scalable one, while the others are designed to handle exceptions in the pattern. For smaller transform sizes such as an $N = 32$ point FFT, this intermediate stage is not needed and is therefore not shown in the representative block diagram.

Finally, this implementation exclusively employs the redesigned memory structure established in the Radix-4 architecture for the earlier sections. Dedicated shift-register delays are retained solely in the final two stages, where the delay depths are shallow enough that register usage is not catastrophic and allocating dedicated Block RAM primitives would be

wasteful.

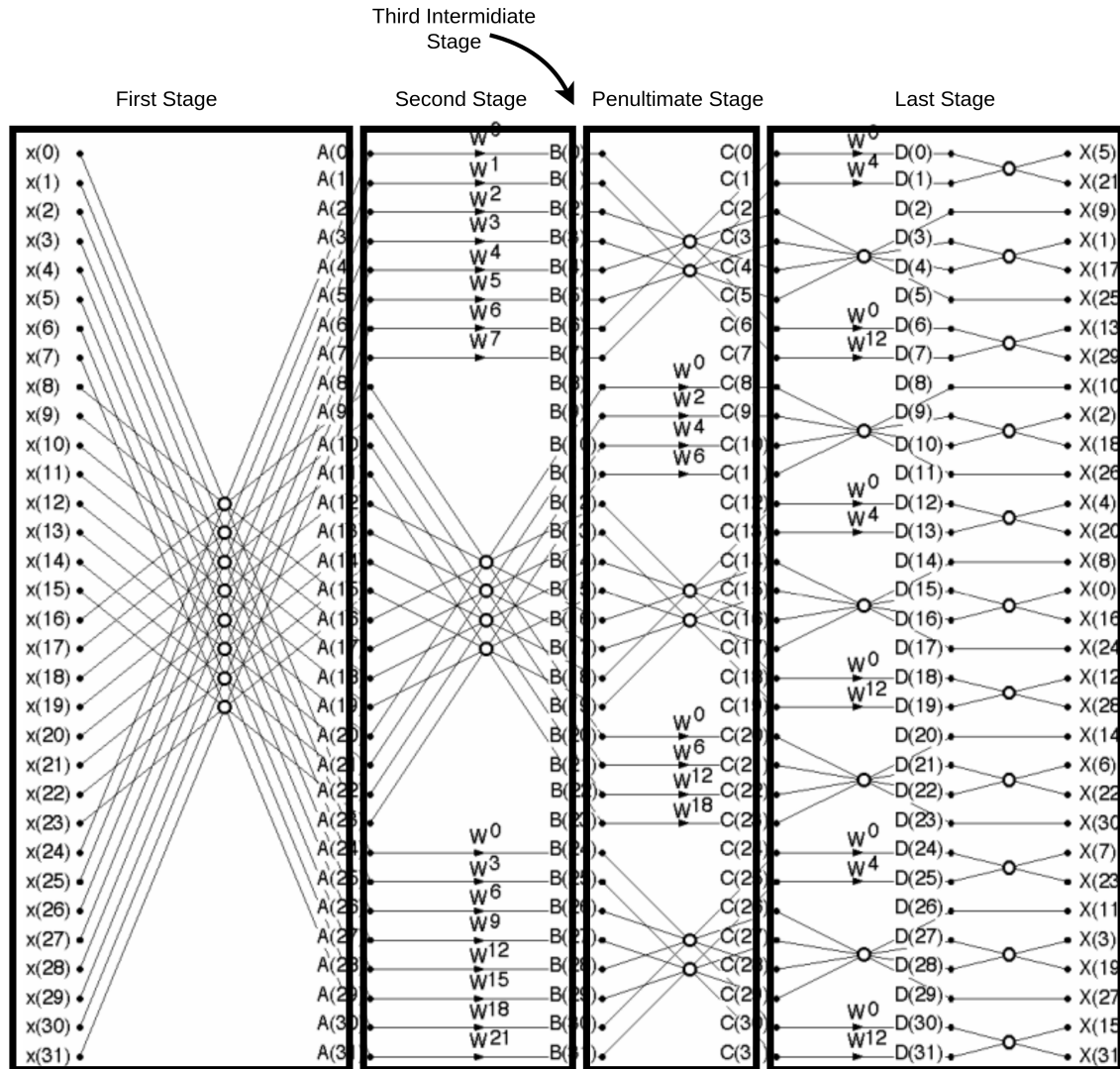


Figure 3.21: Split-Radix stages, example for an FFT size of 32.

3.6.2 Butterfly Split-Radix and Complex Multipliers

Unlike the architectures discussed previously, the split-radix butterfly module does not integrate the complex multiplication step internally, as the multiplication is done in the subsequent stage in this algorithmic 3.20 flow. This separation significantly simplifies the internal logic and datapath of the butterfly unit.

For the required complex multiplications, the design reuses the multiplier modules and multiplication techniques established in the prior architectures.

An analysis of the arithmetic datapath reveals a wordlength growth of 2 bits between consecutive stages, driven by the two sequential addition operations performed at the input

of the butterfly. However, this bit growth does not continue through the entire pipeline, it stops halfway through the total number of stages. Specifically, the maximum bit width is reached at the stage corresponding to the ceiling division of the total stages by two (e.g., in a 9-stage, 512-point FFT, the bit growth terminates at the fifth stage).

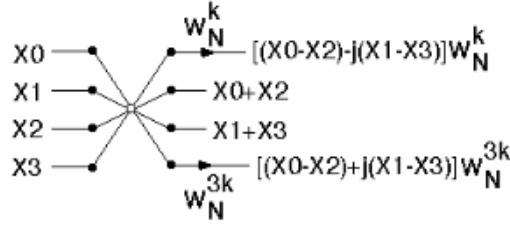


Figure 3.22: Split Radix butterfly.

Algorithm 3 Pipelined Split-Radix Butterfly Core

Require: Complex inputs A, B, C, D ; *start* signal

Ensure: Complex outputs $Out_1, Out_2, Out_3, Out_4$; *done* signal

Step 1: Initial Additions and Subtractions (Pipeline Stage 1)

- | | |
|----------------------------------|---|
| 1: $diff_{02} \leftarrow A - C$ | ▷ Calculates $(A_r - C_r) + j(A_i - C_i)$ |
| 2: $diff_{13} \leftarrow B - D$ | ▷ Calculates $(B_r - D_r) + j(B_i - D_i)$ |
| 3: $y_2 \leftarrow A + C$ | ▷ Calculates $(A_r + C_r) + j(A_i + C_i)$ |
| 4: $y_3 \leftarrow B + D$ | ▷ Calculates $(B_r + D_r) + j(B_i + D_i)$ |
| 5: $start_reg \leftarrow start$ | ▷ Delay start signal to match datapath |

Step 2: Cross-Branch Computations and Final Outputs (Pipeline Stage 2)

- | | |
|---|--|
| 6: $Out_1 \leftarrow diff_{02} - j \cdot diff_{13}$ | ▷ $Re : diff_{02_re} + diff_{13_im}, Im : diff_{02_im} - diff_{13_re}$ |
| 7: $Out_2 \leftarrow y_2$ | ▷ Passed directly from Stage 1 register |
| 8: $Out_3 \leftarrow y_3$ | ▷ Passed directly from Stage 1 register |
| 9: $Out_4 \leftarrow diff_{02} + j \cdot diff_{13}$ | ▷ $Re : diff_{02_re} - diff_{13_im}, Im : diff_{02_im} + diff_{13_re}$ |
| 10: $done \leftarrow start_reg$ | |
-

3.6.3 First Stage

The first stage of the split-radix FFT closely resembles the general stage unit of the Radix-4 architecture. It employs the same control logic and the new split-radix butterfly, alongside a modified memory structure designed to accommodate the distinct input and output access patterns required by this stage.

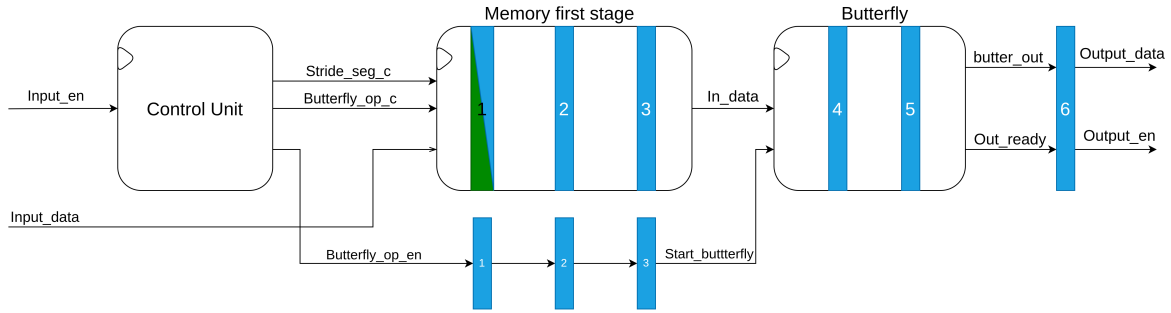


Figure 3.23: First Stage Split-Radix.

In the memory schematic, the numerical values denote the sequential arrival order of the input samples, which are ingested in parallel batches of four (e.g., [1, 2, 3, 4] followed by [5, 6, 7, 8]). The green arrow and red line indicate the respective start and end addresses for the write operations executed during a single clock cycle, with this spatial writing pattern shifting every $N/4$ cycles.

Conversely, the reading patterns, distinguished by color, illustrate a diagonal data retrieval. This physical arrangement ensures that memory reads optimally utilize two ports per bank, enabling the concurrent fetching of strided sequences such as (1, 9, 17, 25) followed by (2, 10, 18, 26). Finally, a ping-pong buffering scheme is used as previously.

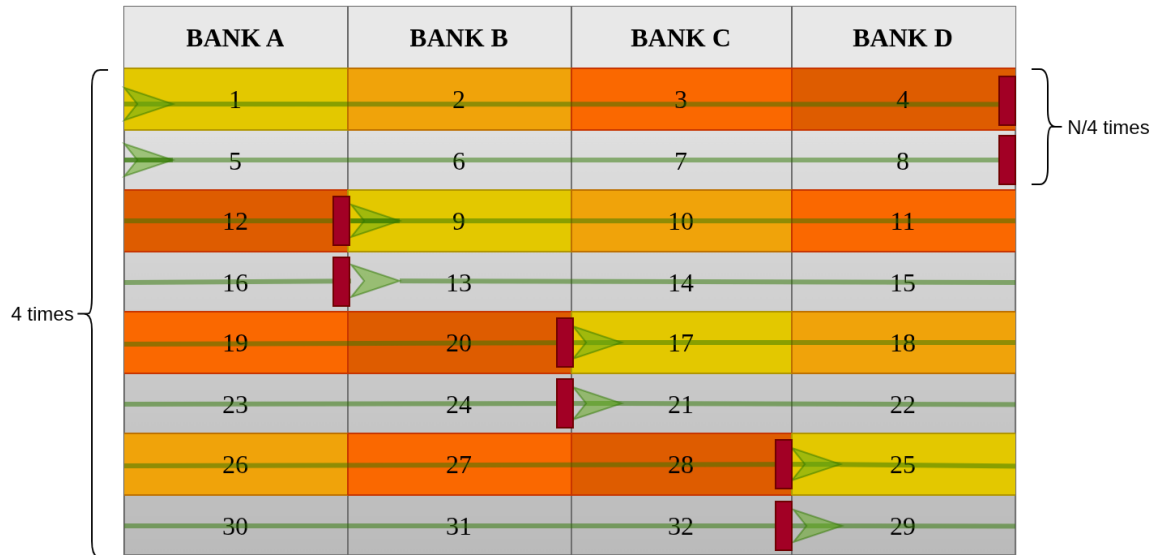


Figure 3.24: Memory pattern First Stage Split-Radix.

3.6.4 Second Stage

While this stage of the split-radix FFT utilizes the same control logic as the first stage, its internal hardware structure is fundamentally different. In previous stages, the dataflow processed the butterfly operations sequentially from top to bottom. However, applying this top-to-bottom computation pattern in this stage, and all the upcoming, resulted in an irregularly shuffled output sequence that did not conform to any standard digit-reversal pattern 3.5. This non-standard permutation severely complicated data unscrambling, creating a significant bottleneck for a high-performance FFT core.

To resolve this routing issue, the computation flow was deliberately restructured. The modified datapath now calculates and outputs the central butterflies first, followed by the upper multiplication outputs, and finally the lower outputs. This reordered scheduling successfully yields a standard bit-reversed output sequence, drastically simplifying the final decoding process.

$$k(p, N) = \begin{cases} p, & \text{if } N \leq 2 \\ 4 \cdot k\left(p, \frac{N}{4}\right) + 1, & \text{if } 0 \leq p < \frac{N}{4} \\ 2 \cdot k\left(p - \frac{N}{4}, \frac{N}{2}\right), & \text{if } \frac{N}{4} \leq p < \frac{3N}{4} \\ 4 \cdot k\left(p - \frac{3N}{4}, \frac{N}{4}\right) + 3, & \text{if } \frac{3N}{4} \leq p < N \end{cases} \quad (3.5)$$

Figure 3.25: Inverse mapping formula for digit reshuffling, p is the output index, N is the size of the FFT and k the real index

Figure 3.26 illustrates the internal dataflow of this stage unit using an $N = 32$ point FFT as an example. The red paths trace the initial outputs from the first unit, which route two operands to the complex multipliers and two operands directly to the Stage 2 butterfly. After a four-cycle delay, on the fifth clock cycle, the black paths show the point at which the stage unit begins computing the first butterfly operations. This sequential processing continues until all butterflies are computed, with their corresponding output pattern highlighted in green. Following the butterfly operations, the system must output the multiplier results, which are calculated concurrently as the data arrives. Finally, the blue lines indicate this output routing pattern for the Stage 2 complex multipliers.

Crucially, the output sequence of the butterflies exactly mirrors the input pattern of Stage

2, while the multiplier outputs align with the initial data arrival pattern of Stage 1. This structural symmetry will be exploited to implement the next stage.

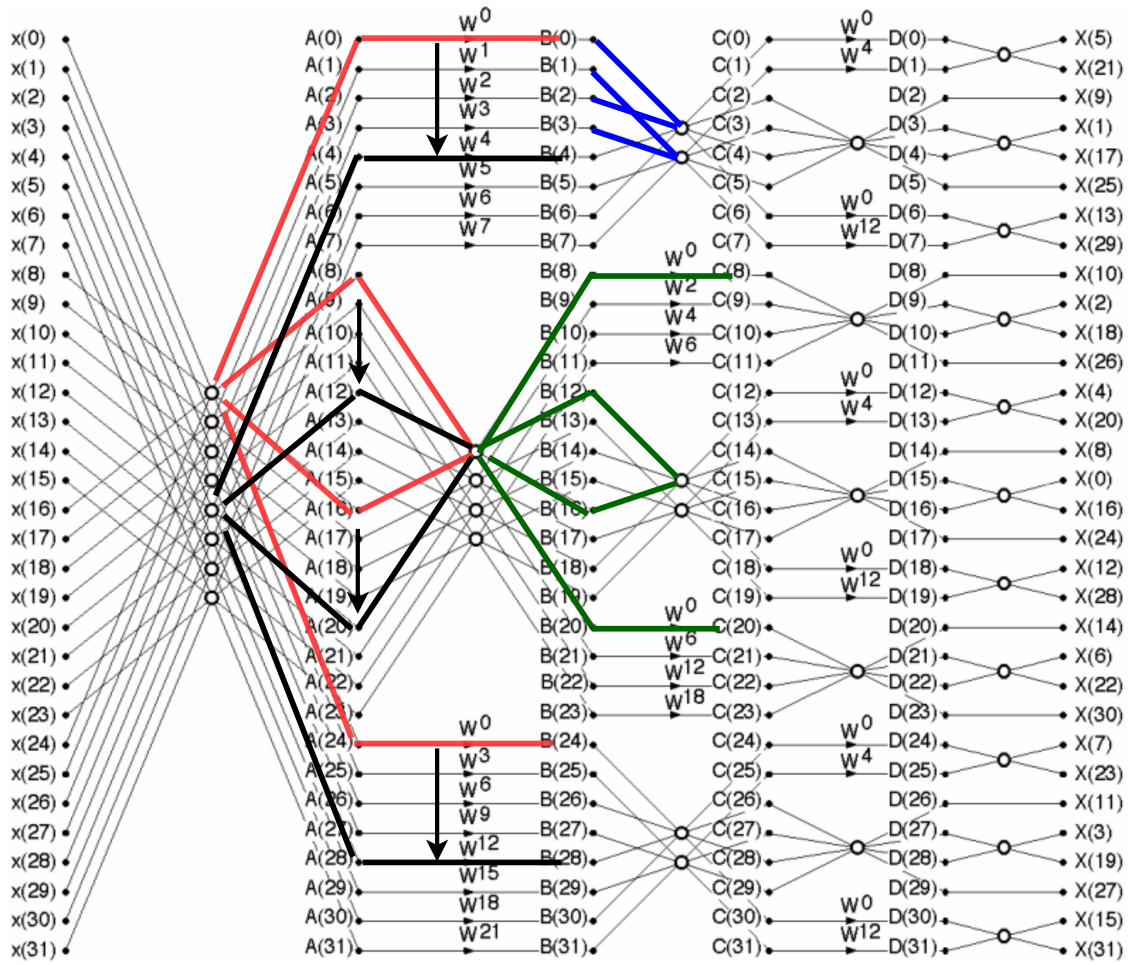


Figure 3.26: Input-Output flow of Split-Radix second stage.

To keep the data organized in memory, we use the same technique as the Radix-4 design. However, we had to drastically change the input and output patterns to match the new flow of the split-radix algorithm.

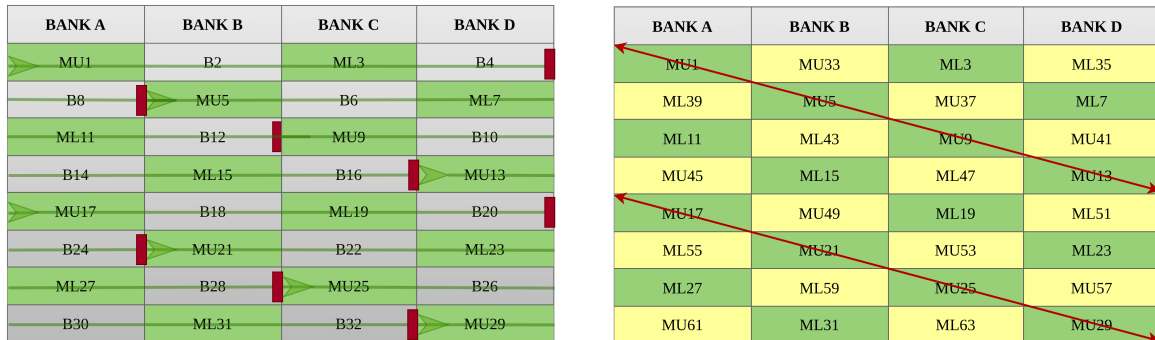


Figure 3.27: Memory pattern second stage split radix.

tiplication mode and multiplier type. Using an adjustable delay module ensures that the data always stays correctly aligned regardless of the configuration.

3.6.5 Third Stage

For this stage, the control logic and input data delay management become significantly more complex, as the unit must be fully parameterizable to function as a reproducible intermediate stage. To address this, a dedicated Finite State Machine (FSM) was designed to act as the control unit. However, before detailing the FSM architecture, the dataflow must first be established.

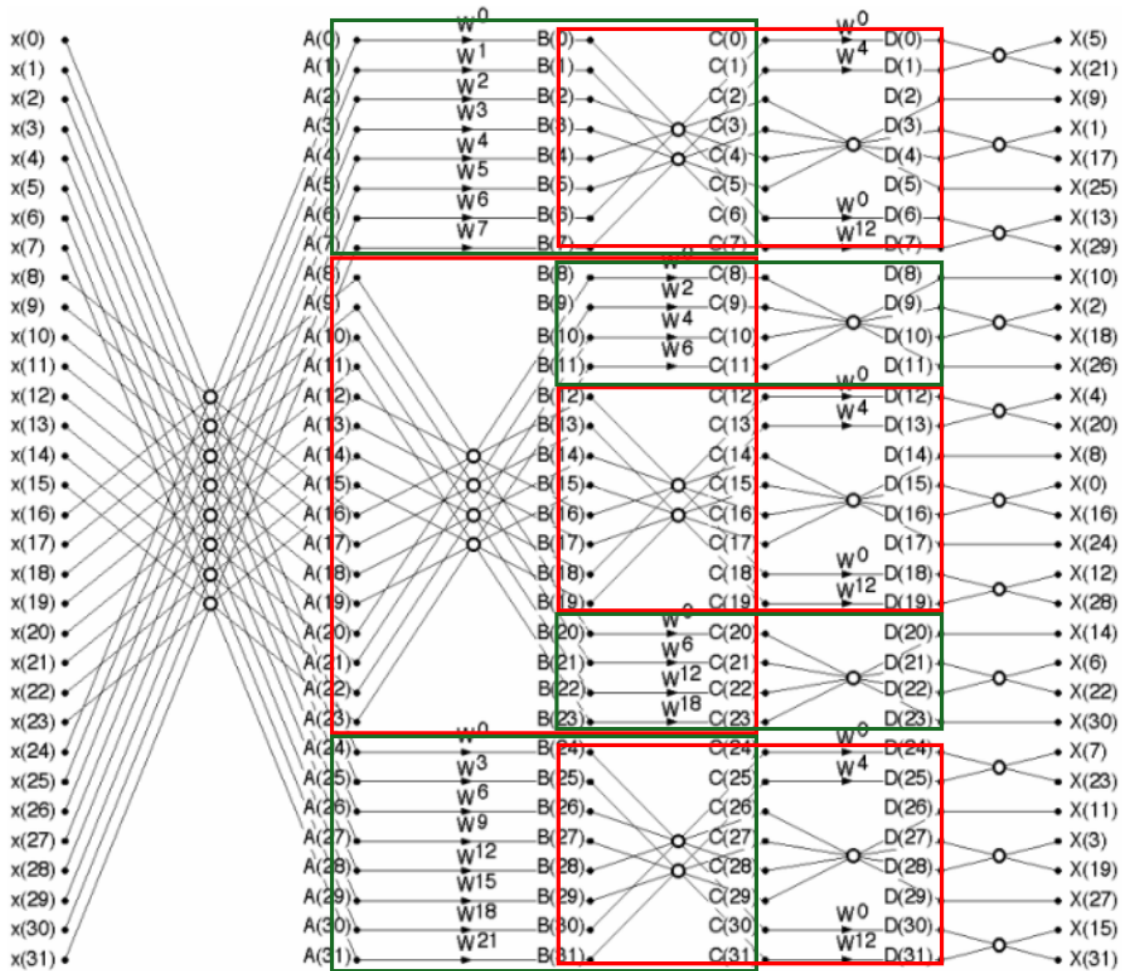


Figure 3.29: Third stage Dataflow

The algorithm is partitioned into two distinct phases: Stride 0 and Stride 1. Stride 0 processes data originating from the previous stage's butterflies, while Stride 1 handles data from the previous stage's multipliers. In the accompanying figure 3.29, Stride 0 is highlighted in

red and Stride 1 in green.

As noted earlier, the butterfly output pattern of Stage 2 mirrors its own input pattern, whereas the multiplier output pattern mirrors the input pattern of Stage 1. This structural symmetry is the primary motivation for splitting the algorithm in this manner. To achieve the desired bit-reversed output sequence, these strides are again computed from the center outwards. Consequently, Stride 0 receives inputs from the butterflies and operates as a scaled-down Stage 2, while Stride 1 receives inputs from the multipliers and operates as a scaled-down Stage 1, effectively mirroring a recursive functional structure in hardware.

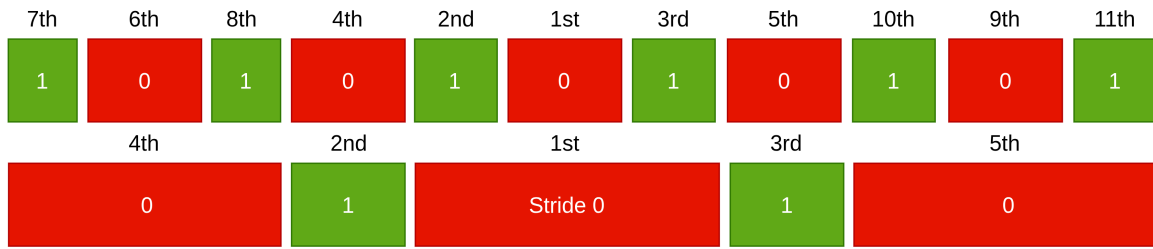


Figure 3.30: Third stage Dataflow sequence

Figure 3.30 shows the stride sequence for a specific arrangement that occurs in the forth and fifth stages of an FFT larger than 256 points, which falls within the intermediate stages handled by this module. The red squares represent recursive instances of the Stage 2 like structure, while the green squares represent instances of the Stage 1.

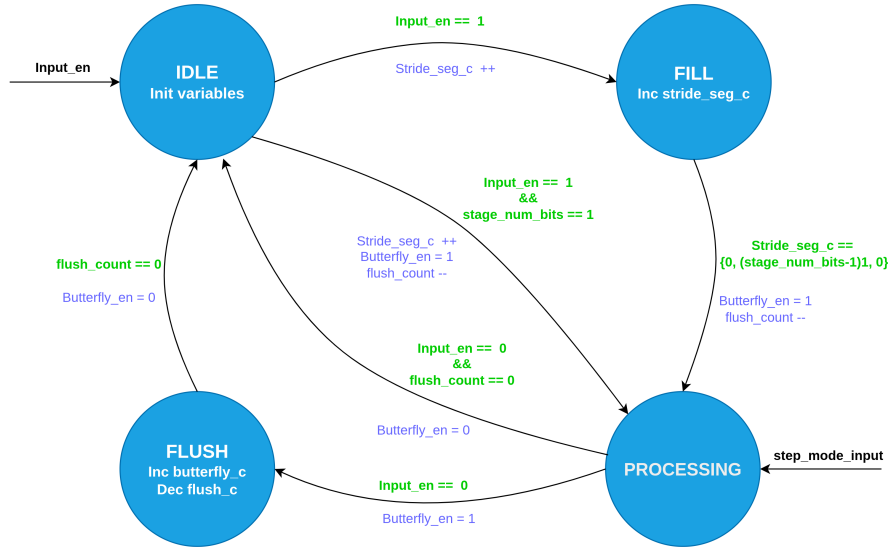


Figure 3.31: Third stage Control Unit

The control unit is made up by a four-state Finite State Machine (FSM). The FSM ini-

tializes in the **IDLE** state, awaiting valid incoming data. Upon detecting an active `Input_en` signal, it transitions to the **FILL** state to save the initial data in memory required to start computation. Once all the necessary data has arrived, the FSM enters the **PROCESSING** state. In this active phase, the butterfly enable signal is asserted and continuous computation occurs. The stride mode is computed through the internal counter and the step mode input coming from the previous stage. When the input stream concludes (`Input_en == 0`), the machine shifts to the **FLUSH** state. This phase maintains active computation to empty the internal datapath pipeline. Once the residual data is fully processed and the flush counter reaches zero, the FSM returns to the **IDLE** state.

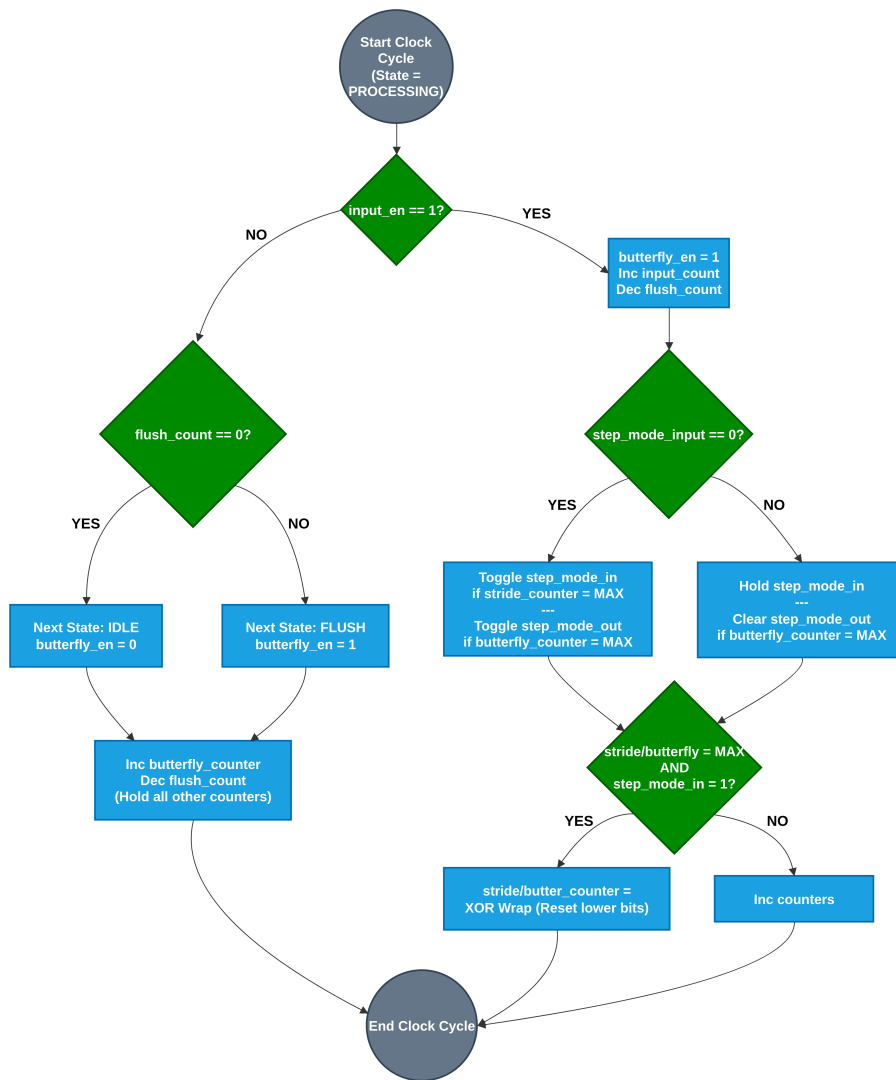


Figure 3.32: More detailed diagram for the processing state

Lastly, the memory pattern algorithm is a mix of the first and second stages' memory, where the input and output multiplexing is swapped by the current stride the unit is processing.

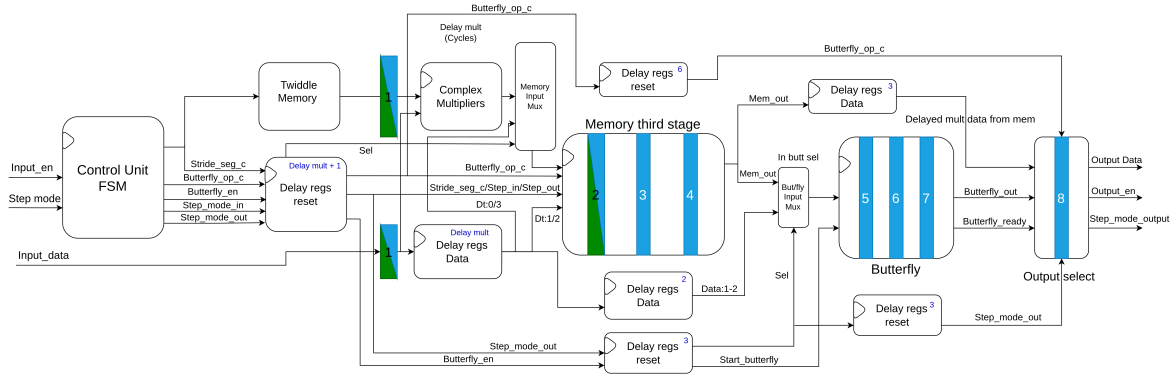


Figure 3.33: Third Stage Unit Top Diagram.

3.6.6 Penultimate Stage and Last Stage

These stages did not fit the memory pattern of the generalized third stage, and instead of complicating the memory further, they were designed separately. Since these stages do not need to delay the input data for more than 2 cycles, the general memory structure was skipped, and delay registers were used for data alignment instead. This did not require any sophisticated design rather, the correct delays for each data path were determined through behavioral testing and prior implementation experience.

Additionally, because a memory module is not used, the butterfly and complex multipliers can run in parallel, shortening the pipeline depth. Also, the penultimate stage uses the same control module as the third stage, and the last stage uses a reduced version of it and both follow the same dataflow using strides.

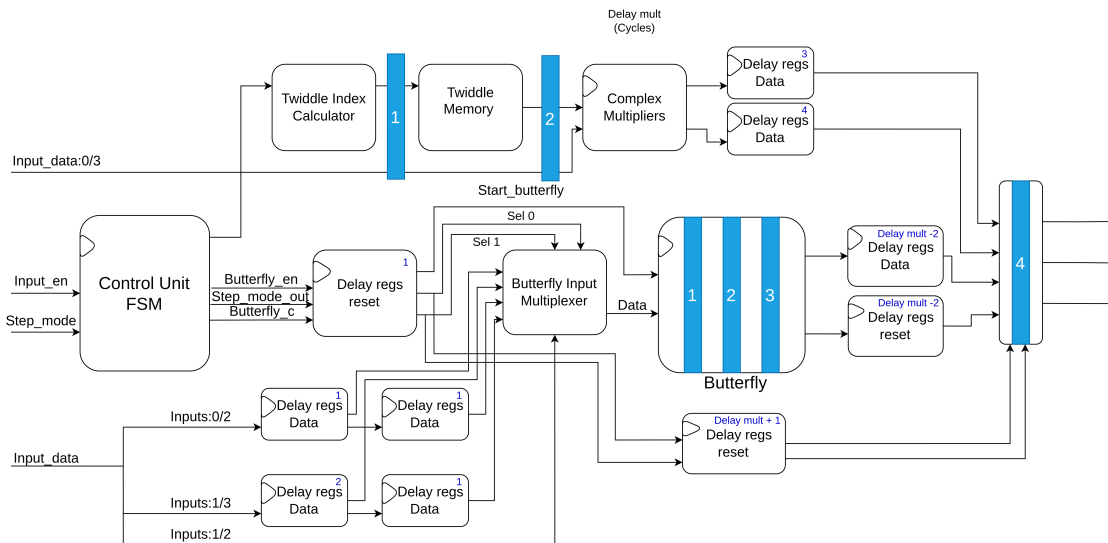


Figure 3.34: Penultimate Stage Unit Top Diagram.

For the last stage, the final two stages of a split-radix FFT were essentially combined. This is because the last stage does not use multipliers or a butterfly, and has zero delay for the inputs. It performs a simple addition and subtraction, which can fit between pipelines without causing any timing problems.

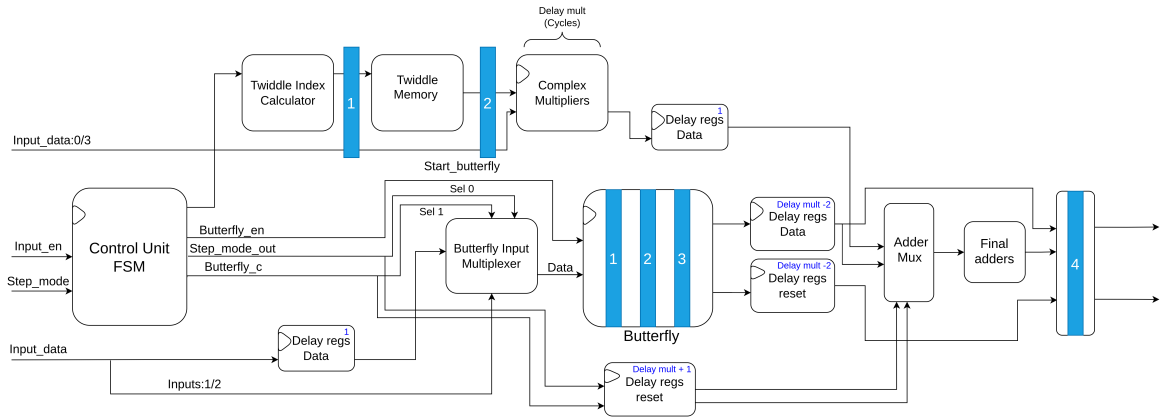


Figure 3.35: Last Stage Unit Top Diagram.

3.6.7 Implementation Results using an ISO Clock

These implementations use the same parameters as in the previous sections of the Radix-2 and Radix-4 implementation.

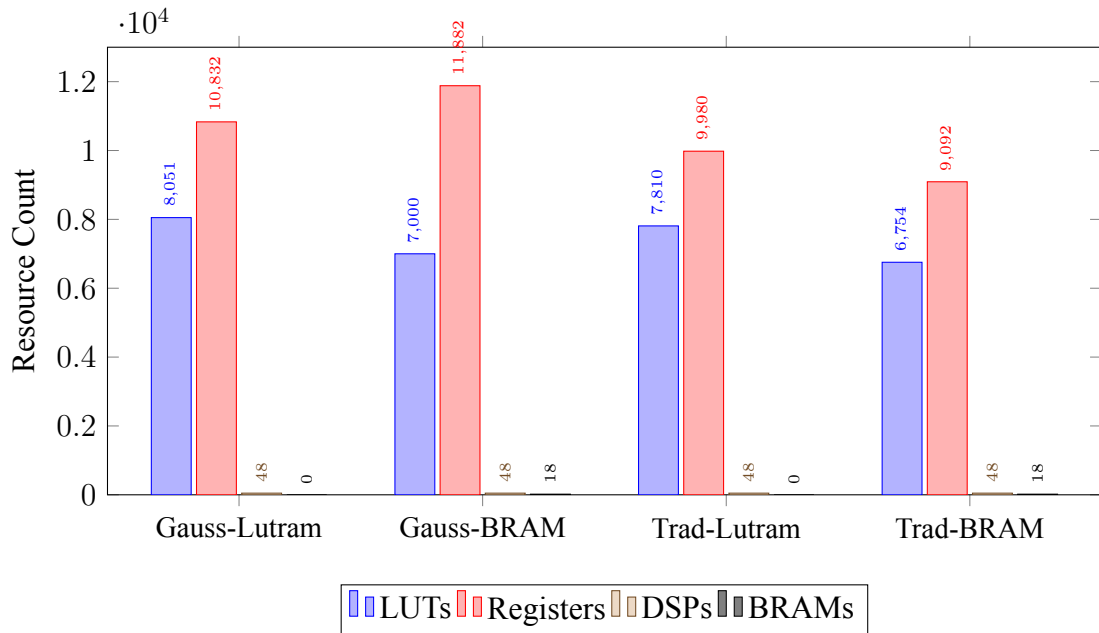


Figure 3.36: Resource utilization for Split-Radix FFT with new memory at $N = 256$ (16-bit).

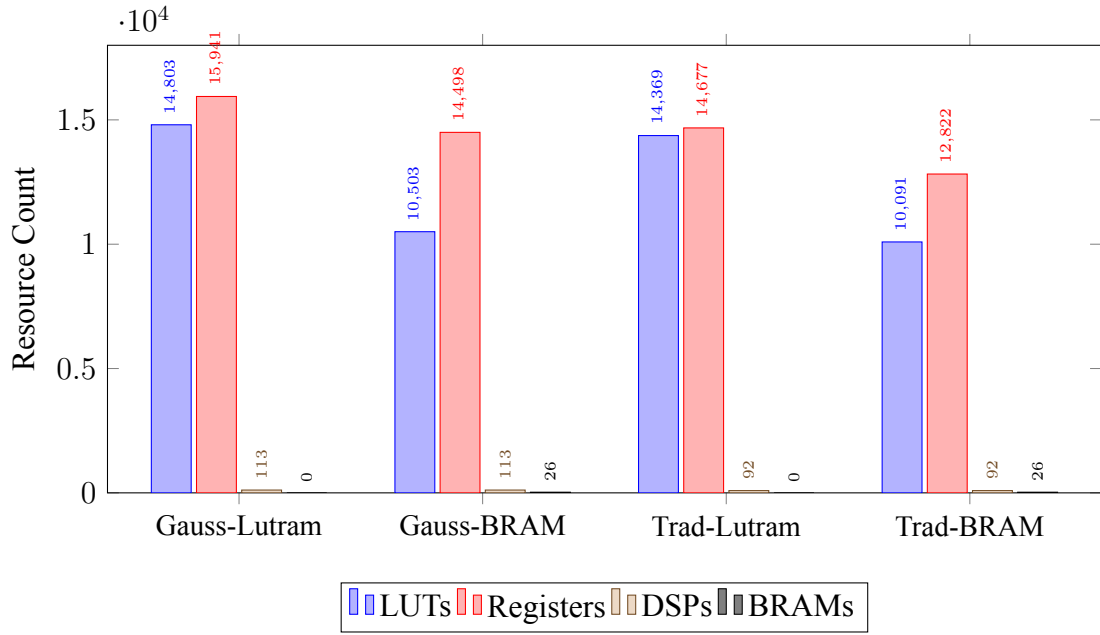


Figure 3.37: Resource utilization for Split-Radix FFT with new memory at $N = 1024$ (16-bit).

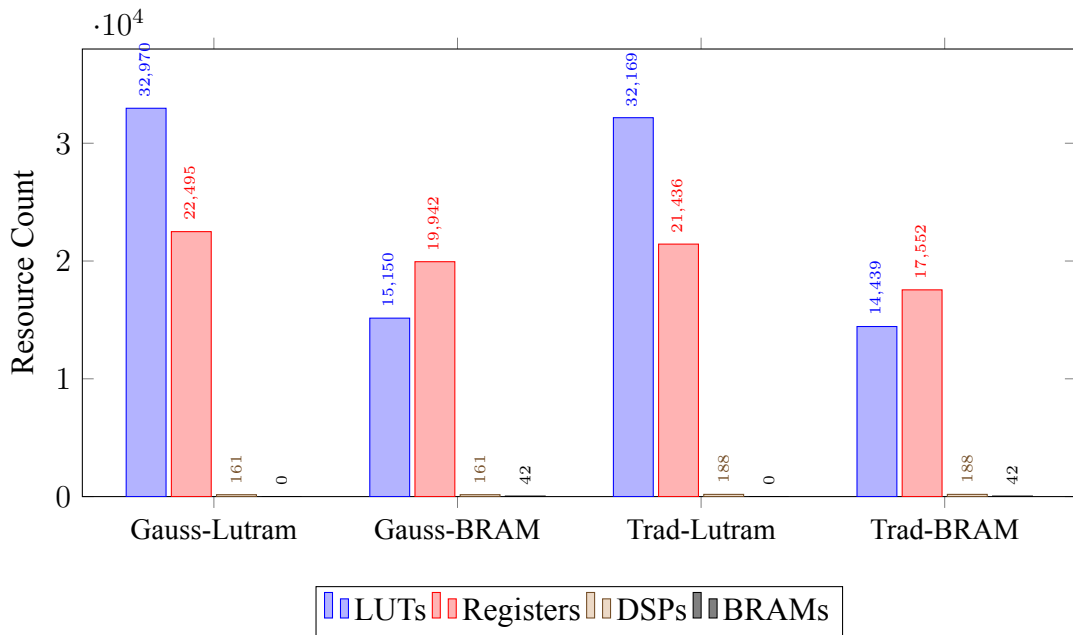


Figure 3.38: Resource utilization for Split-Radix FFT with new memory at $N = 4096$ (16-bit).

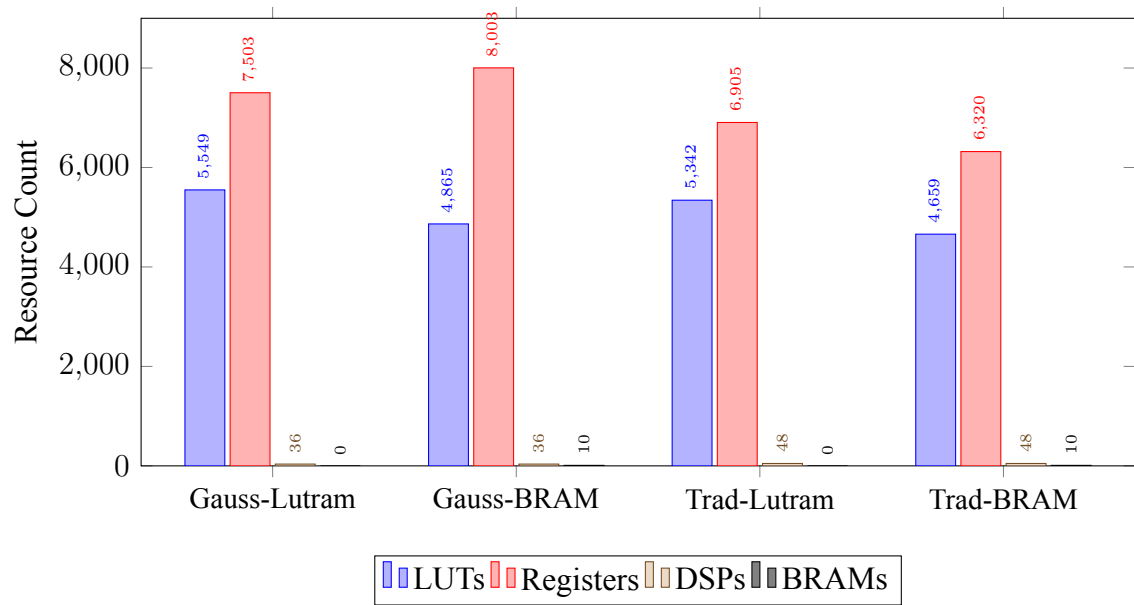


Figure 3.39: Resource utilization for Split-Radix FFT with new memory at $N = 256$ (8-bit).

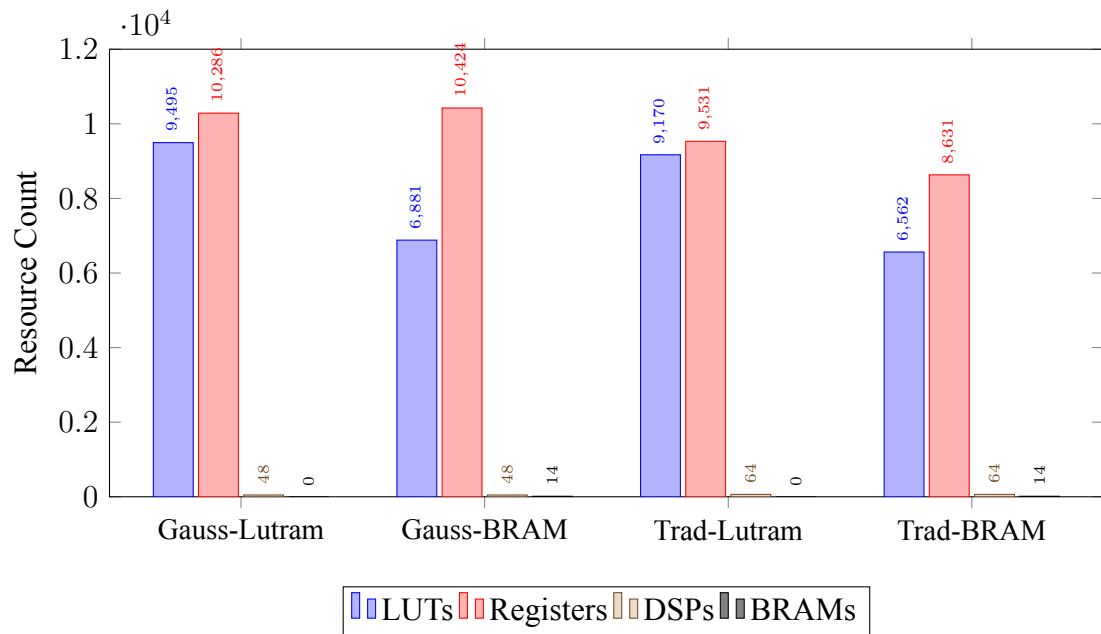


Figure 3.40: Resource utilization for Split-Radix FFT with new memory at $N = 1024$ (8-bit).

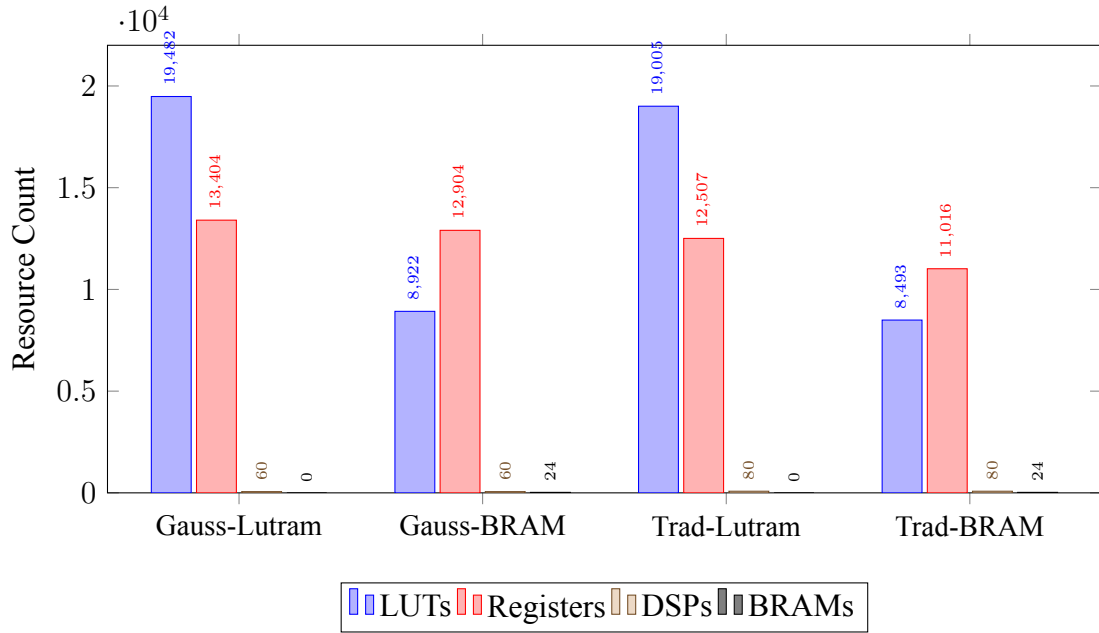


Figure 3.41: Resource utilization for Split-Radix FFT with new memory at $N = 4096$ (8-bit).

Again, as the FFT size increases, we can observe sizable area improvements from using BRAM instead of LUTRAM. However, an irregularity appears where the DSP count does not follow the predicted $3/4$ rule. This discrepancy likely occurs because, even though the RTL specifies which arithmetic operations should map to DSPs, the synthesis tool overrides these attributes due to the size and routing of the design.

Table 3.5: FIFO Split-Radix Latency.

Metric	256	1024	4096
Latency Trad (cycles)	161	515	1877
Latency Gauss (cycles)	167	523	1887

The latency results are worse than Radix-4 3.4, primarily because split-radix uses twice as many stages, though they are fractionally better than Radix-2 3.3.

3.7 Timing and Critical Path Optimization

This section presents several optimizations regarding the timing of the algorithmic designs. Although the pipelining in the current architecture is deep, the dedicated silicon primitives of the FPGA, specifically the DSPs and BRAMs, require precise handling to operate

at their maximum frequency.

3.7.1 DSP Optimizations

In the current design, a primary issue is that the synthesis tool collapses the pipeline registers between the multiplications and the final additions within the complex multiplier. This occurs because DSP blocks inherently support multiply-accumulate (MAC) arithmetic, prompting the synthesis tool to merge these operations and inadvertently bypass the intended pipeline architecture.

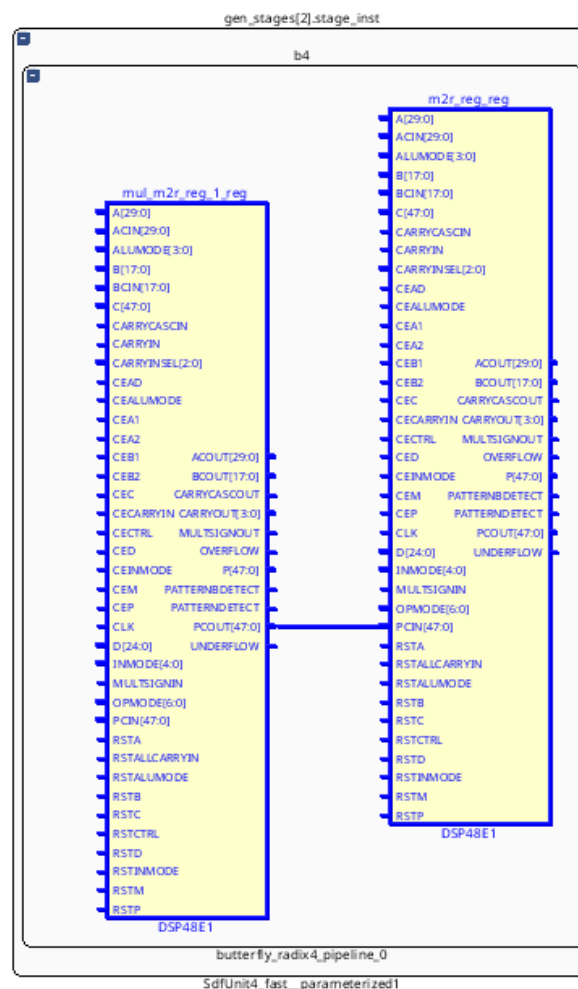


Figure 3.42: Critical path of unoptimized DSPs.

Figure 3.42 shows the critical path reported by Vivado. It essentially consists of two cascaded DSP blocks: the left one performs a multiplication and feeds its output directly to the right one, which performs the other multiplication and adds the results. The "lost" register between the multiplication and the addition is likely embedded inside one of the DSPs, as

they have the ability to absorb registers from the FPGA fabric to operate faster under certain conditions. To solve this, the cascade must be broken by adding one more pipeline stage.

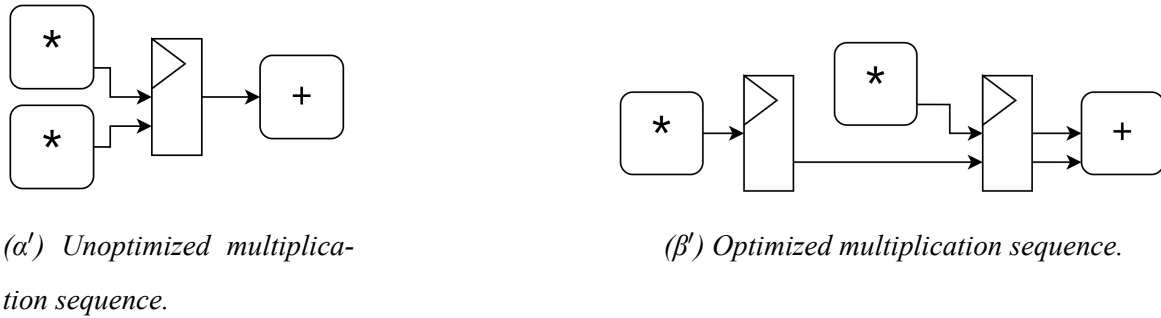


Figure 3.43: Comparison of DSP multiplication sequences.

As shown in Figure 3.43, the left figure represents the original design, while the right illustrates the new optimized approach. Changing the multiplication sequence in this way—by adding another pipeline register to one of the multiplication paths—allows the DSP unit to absorb more registers into its internal fabric, thereby improving the critical path.

We can verify this internal register inference by executing the following command in the Vivado TCL console after the implementation has finished:

```
foreach dsp [get_cells -hierarchical -filter {REF_NAME == DSP48E1}]
{
  puts "DSP: $dsp"
  puts "  AREG = [get_property AREG $dsp]"
  puts "  BREG = [get_property BREG $dsp]"
  puts "  CREG = [get_property CREG $dsp]"
  puts "  MREG = [get_property MREG $dsp]"
  puts "  PREG = [get_property PREG $dsp]"
}
```

Listing 3.1: TCL script to verify DSP internal register utilization.

Ideally, if the DSP block is fully pipelined, the query command will show that its internal registers are actively utilized. Within the DSP48E1 slice, the input data registers (AREG and BREG) support an internal pipeline depth of up to 2. The remaining registers (CREG, MREG, and PREG) have a maximum depth of 1.

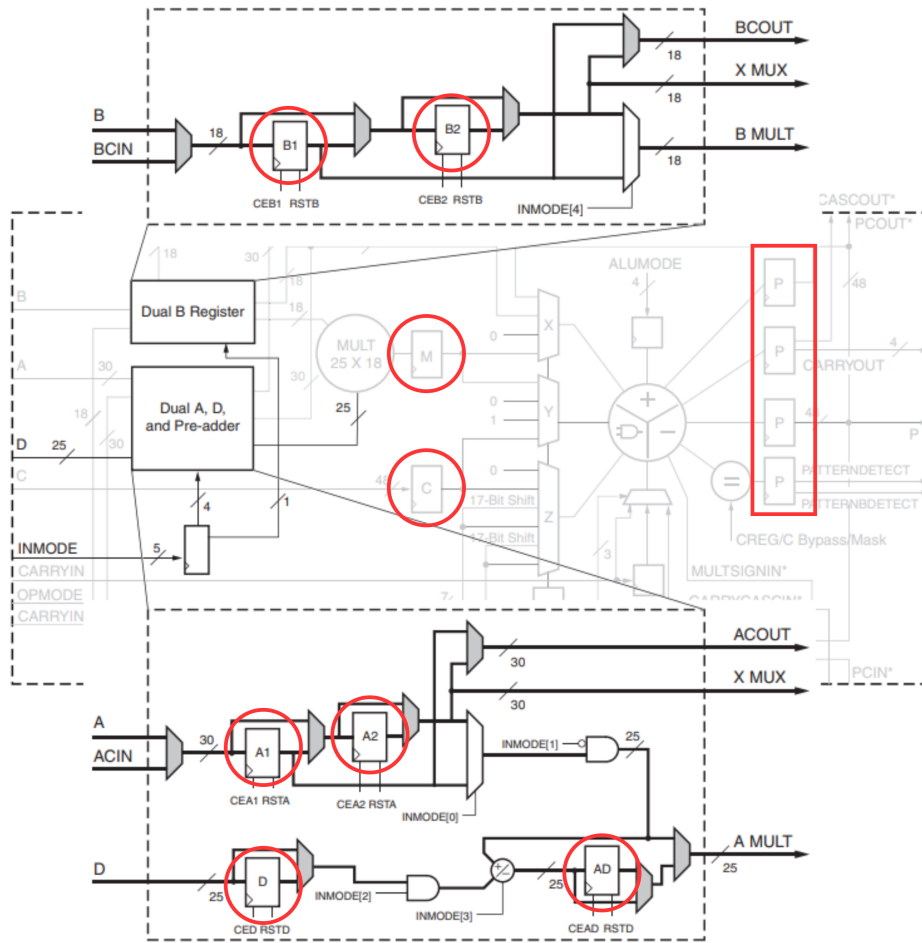


Figure 3.44: Register positions inside the DSP48E1 schematic.

Initially, executing this script returned mostly 0s and a few 1s for the implemented DSPs. After the optimization, the command output showed values of 2 for the inputs and 1 for the rest. This clearly proves that the extra pipeline registers were pulled straight into the DSP blocks, resolving the critical path bottleneck. This change was applied to both the traditional and Gauss complex multiplications. Unfortunately, this optimization adds one extra pipeline stage to the stage unit module.

Another limitation of the on-chip DSP48E1 slices is that wider twiddle factors and consequently larger datapath widths exceed the capacity of a DSP primitive, which supports operands up to 25×18 bits. When either operand exceeds these limits, the synthesis tool is forced to cascade two DSP slices to compute a single multiplication, significantly increasing resource consumption and making timing worse. To prevent this cascade penalty, the twiddle factor width has to be reduced, ensuring that intermediate data widths remain within the native 25×18 -bit limits of a DSP slice throughout the pipeline.

3.7.2 Memory and Bram Optimization

When the memory used BRAMs, a similar problem occurred at the memory output. Although the read path was already pipelined, the synthesis tool mapped this register to the primary synchronous output register of the BRAM. This register is mandatory simply to operate the BRAM primitive, which requires a minimum read latency of one clock cycle by default. To solve this, an additional pipeline stage was added to the BRAM output path.

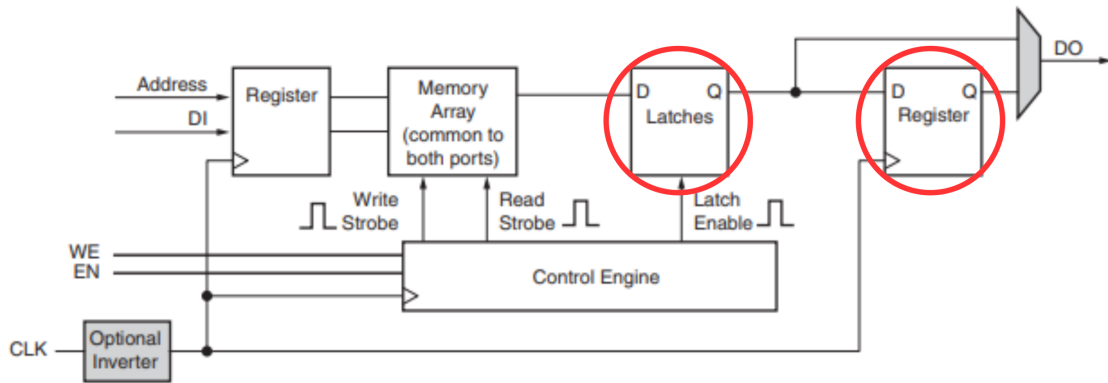


Figure 3.45: Register positions inside the RAMB36E1/RAMB18E1 schematic.

Finally, specifically for the split-radix memory module, the input and output multiplexer paths had to be broken up using two pipeline stages. These multiplexing paths were particularly deep due to the increased addressing and routing complexity of the split-radix memory algorithm compared to the Radix-2 and Radix-4 architectures.

Chapter 4

Functional Simulation and Algorithmic Verification

4.1 Overview

To test the hardware designs, it is necessary to ensure that they work correctly for every combination of parameters and across various data inputs. To achieve this, an automated Python script was developed. Using the specified design parameters, this script automatically generates all the needed test inputs, simulates the behavior of the RTL design, compares the hardware output against a golden model, and provides the final results.

The verification script has several configurable parameters, such as the FFT size, data width, twiddle factor width, multiplier technique, bit-growth logic, and hardware optimizations for DSP and memory, and type of FFT algorithm. An example of a function call executing a specific parameter combination is shown below:

```
main(N=4096, Data_width=i+12, Tw_width=i, Num_of_windows=100,  
     type_fft=2, iterations=10, signal_mode="high", bit_growth=0,  
     SimpleMult=0, Fast_DSP=0, carry_save=0, Bram=0, Cshm=0,  
     output_pipeline_bram=0, input_pipeline_bram=0, waves=False)
```

Additionally, the `Num_of_windows` parameter defines how many continuous input streams of size N are processed during a single simulation, while `iterations` determines how many independent times the testbench is executed. The `signal_mode` parameter will be explained in a subsequent section. Finally, the `Cshm` and `carry_save` options are used to select dedicated multiplier architectures, which are yet to be discussed in detail.

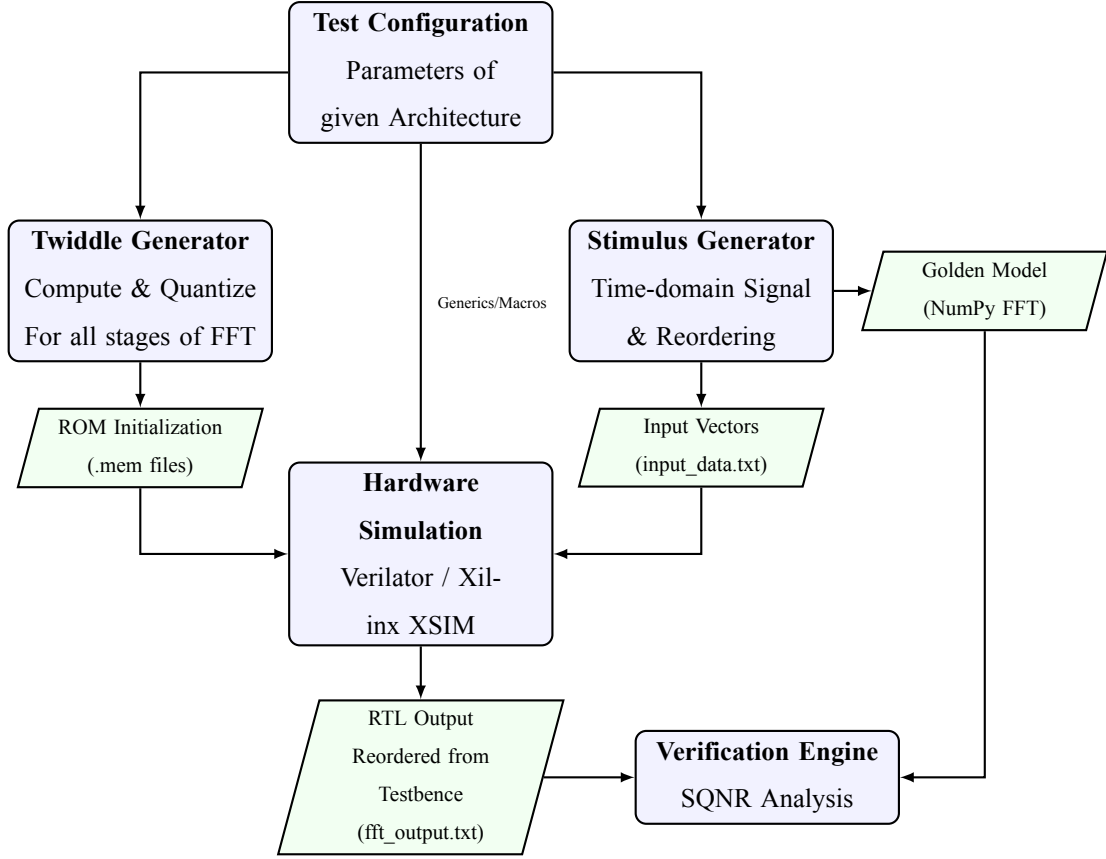


Figure 4.1: Automated verification flow implemented in Python, showing the data path from data generation to hardware simulation and final numerical analysis.

4.2 Stimulus Generation

Twiddle generation was discussed at the start of the implementation chapter, here, it only needs to be noted that each set of twiddles for every stage is written to a unique file in order for the RTL to have access.

For the stimulus or input data generation, there are three modes: low, high, and random frequencies, in order to test if there are any edge-case weaknesses in the design. These are generated using random frequencies, phases, and amplitudes to create a window of size N , representing a sampled real-world signal. After that, since fixed-point arithmetic is used, the input is scaled to $[-1.0, 1.0]$ because there is no dynamic scaling based on inputs within the processor. Based on the type of algorithm, the data is reordered to match its required input. Lastly, Numpy's recommended random number generator is used to ensure the randomness of the frequencies, phases, and amplitudes.

4.3 Simulation

For the simulation, a testbench is created which takes the input data, feeds it into the design at x samples per clock cycle, waits for the output, and reorders it if needed. Finally, it writes all the output data to a file. This testbench is fully capable of handling continuous streaming windows of data. Originally, the simulator used was Icarus Verilog, but it did not work well with some of the required SystemVerilog functionalities. Verilator was then chosen for its unmatched performance, making it easier to simulate thousands of data windows per run. Unfortunately, implementing the bit growth feature required complex RTL, using generate blocks to instantiate modules with different parameters per iteration. This caused bugs with the Verilator compiler, thus, Vivado XSIM was also used specifically for when the bit growth parameter was active.

4.4 Comparison with Golden Model

For the golden model, NumPy's FFT function was used with the exact same input data as the hardware design. To compare the quality of the results, the SQNR metric is used. Here are the results for low, high and random signal types:

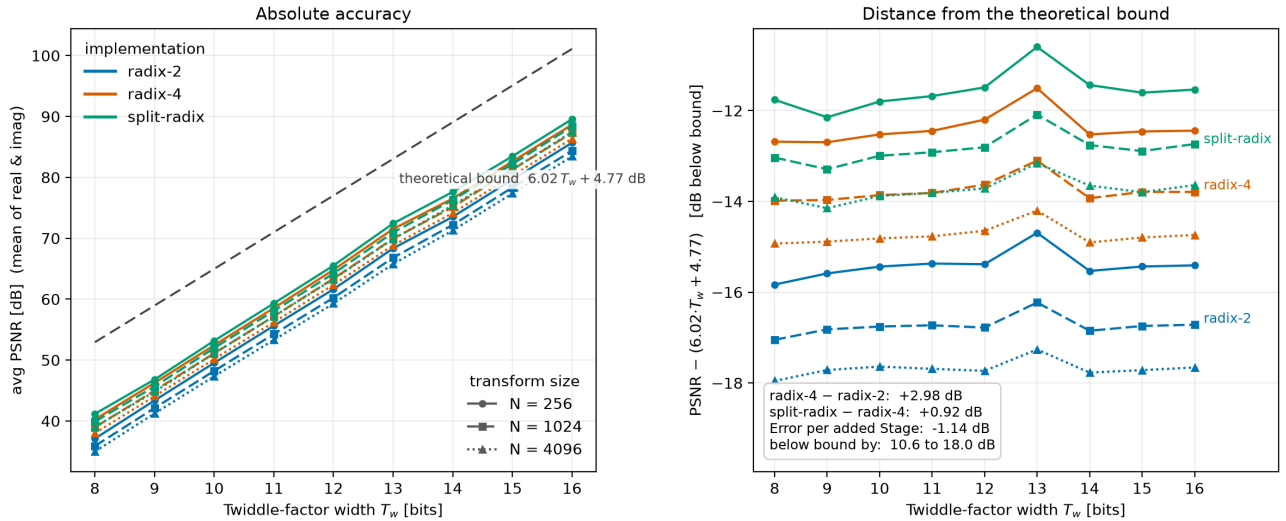


Figure 4.2: FFT output accuracy - low frequency input (averaged over SimpleMult / CheapMult)

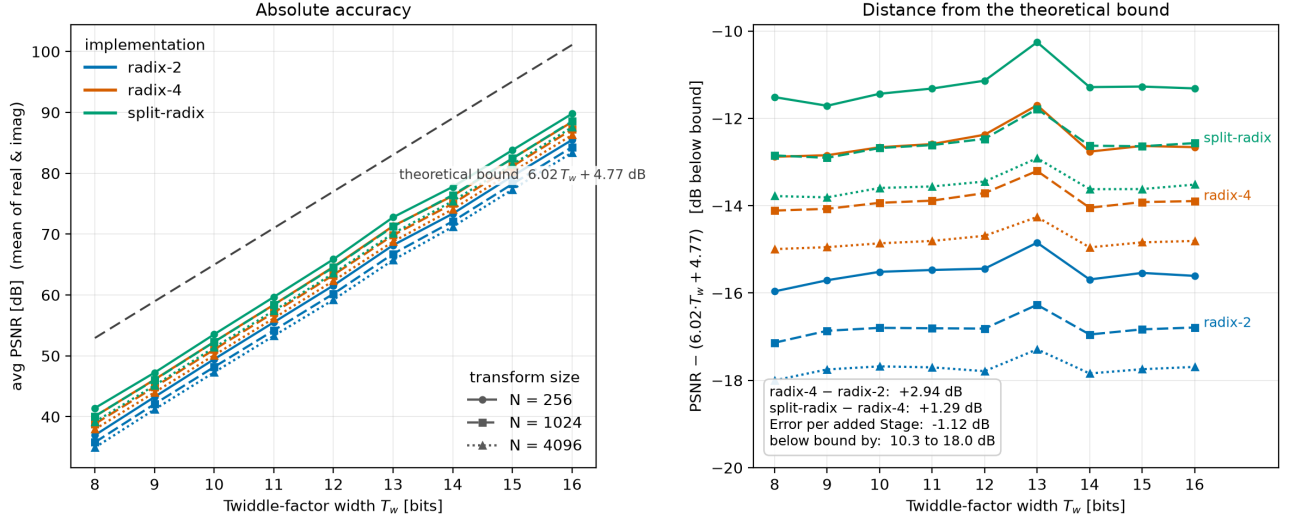


Figure 4.3: FFT output accuracy - high frequency input (averaged over SimpleMult / CheapMult)

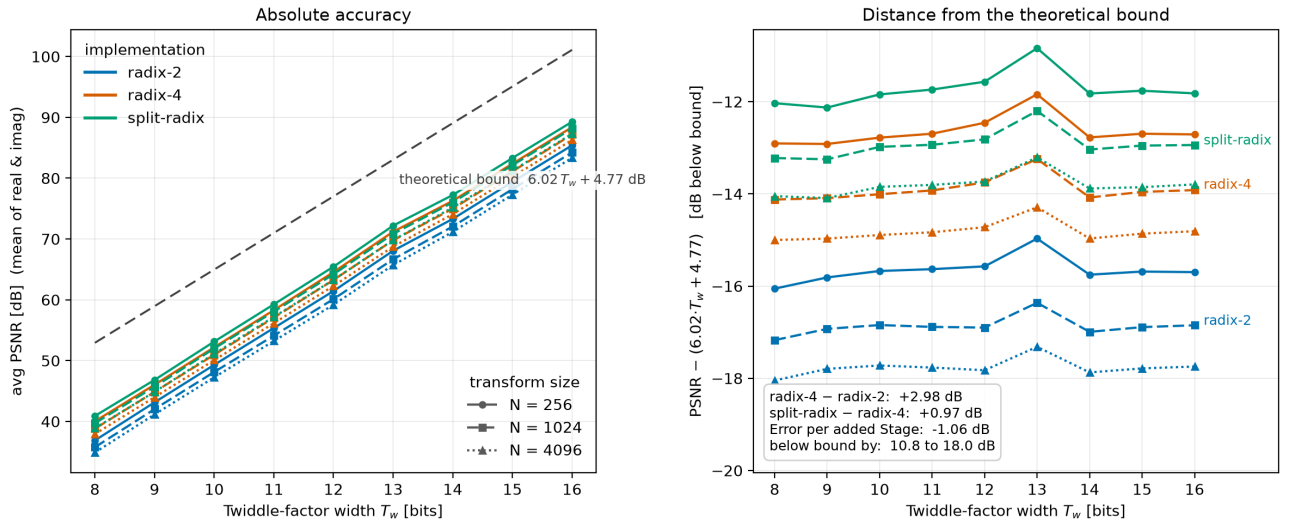


Figure 4.4: FFT output accuracy - random frequency input (averaged over SimpleMult / CheapMult)

Firstly, it can be seen that there are no meaningful differences in the SQNR results across the different input signal profiles. These results were generated across 10,000 random input windows. Furthermore, the "Absolute accuracy plot" clearly demonstrates that the output quality improves consistently as the twiddle bit-width increases, scaling from approximately 35 dB to 90 dB. While this correlation is an expected mathematical outcome rather than an achievement, it highlights the design's scalability. This configurable feature allows for the appropriate hardware resolution to be selected based on specific application requirements.

The "Distance from the theoretical bound" graph illustrates the distance of each point from the theoretical upper bound. This clearly demonstrates the performance of each design based on the algorithm and transform size. The Split-Radix architecture achieves the highest accuracy, closely followed by Radix-4, while Radix-2 has the lowest performance. This aligns with the theoretical number of arithmetic operations required by each algorithm where having more operations means lower accuracy. Finally, it can be observed that for every quadrupling of the FFT size N (which adds two stages for the Split-Radix and Radix-2 architectures, and one stage for Radix-4), the output quality degrades by only roughly 1 dB. This is significantly better than the results discussed earlier. Lastly, an unexpected spike in performance can be seen at a twiddle width of 13 bits.

Chapter 5

Custom Multipliers

In this chapter, two types of custom multipliers are implemented to evaluate the effects of not using dedicated FPGA silicon resources (such as DSP blocks), as well as to see if timing performance can be further improved. The first architecture is a carry-save array multiplier, which serves as a general-purpose multiplier. The second is a Computation Sharing Multiplier (CSHM), a signal processing-specific, multiplierless unit capable of multiplying an input by a finite set of predetermined multiplicands based on its hardware configuration.

5.1 Carry-Save Array Multiplier Architecture

The architecture is built around a fully pipelined carry-save multiplier, which is designed to be fast by breaking the math down into smaller, clocked steps. The operation works through the following stages:

- **Input and Initial Setup:** When the multiplication starts, the input numbers are saved into registers, and the first partial product is generated.
- **Carry-Save Reduction Pipeline:** For the middle bits of the multiplier, the design uses a chain of carry-save adders. Instead of waiting for a "carry" bit to travel all the way through a long addition, this method saves the sum and carry bits separately and passes them to the next stage, which greatly speeds up the clock frequency.
- **Signed Number Handling:** The hardware correctly processes signed (negative) numbers by treating the very last bit (the most significant bit) differently. If needed, it subtracts this final partial product using two's complement logic.

- **Chunked Final Addition:** After the carry-save stages, a final carry-propagate adder is used to combine the sum and carry bits into the final answer. To prevent this final addition from slowing down the system, it is split into smaller pieces defined by a `CHUNK` parameter (e.g., adding only 8 bits per clock cycle).
- **Output Registration:** Finally, the fully calculated product is placed into an output register, and a “done” signal is raised to let the rest of the system know the result is ready.

Lastly, it is important to note that the latency of this multiplier is dependent on the bit-width of its inputs. Consequently, increasing the twiddle factor and data widths leads to higher latency and resource utilization. To ensure correct synchronization, this latency is dynamically calculated within the stage module to properly align the data paths. Furthermore, the overall computational accuracy of the FFT remains unaffected by this multiplier, as it operates deterministically.

5.2 CSHM Architecture

The general idea behind this multiplier was discussed in the second chapter of this thesis. The primary difference between a classic CSHM unit and this implementation is that the shift amounts (controlled by the barrel shifters within the arithmetic sets) are variable rather than fixed. This design choice was made to ensure the module can support different FFT sizes, twiddle factor bit-widths, algorithms, and specific operational stages. Each of these configurations requires a unique subset of twiddle factors throughout the entire FFT computation.

5.2.1 Architecture Choices

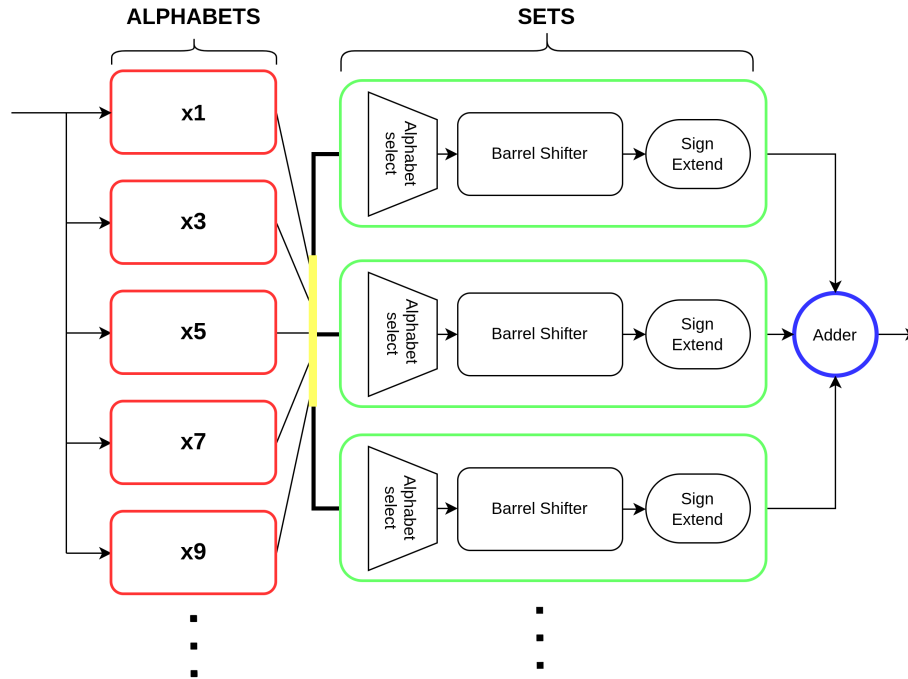


Figure 5.1: Top diagram of the general CSHM Multiplier

In a traditional complex multiplication, each data sample is multiplied twice: once by the real component and once by the imaginary component of the twiddle factor. Rather than utilizing a separate CSHM module for each individual multiplication, a single combined module can be used to process both multiplications for the same input sample. This architecture allows the precomputed alphabets to be shared, while the subsequent shift and add operations remain unique to each multiplicand. This approach can potentially save significant hardware resources. For each multiplicand, a unique control key is constructed that dictates the specific alphabets, shift amounts, and signs required to calculate the result. This key is stored in the twiddle factor memory in place of the actual twiddle factor value, and it is generated by a custom script that will be discussed later.

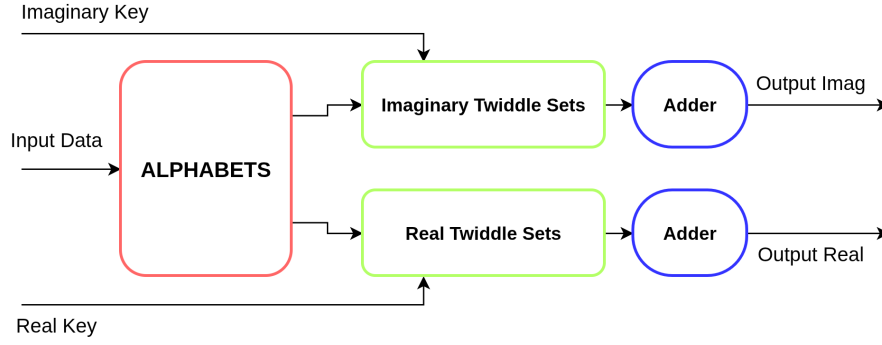


Figure 5.2: Top diagram of the CSHM Architecture used

5.2.2 Hardware Resource Comparison

Before proceeding with the implementation, a structural choice must be made regarding the size of the sets and alphabets. To make this decision, the hardware resource utilization of 2-set and 3-set CSHM modules (with 4 alphabets) is compared against the carry-save multiplier discussed previously. For the CSHM architecture, only half the number of modules is required compared to the standard carry-save implementation, as each CSHM module processes two real multiplications simultaneously. However, if the Gauss method is applied to the carry-save complex multipliers, which reduces the operation from four to three real multiplications, the CSHM implementation requires two-thirds of the modules. The results will be scaled accordingly in order to approximate which version is worth implementing

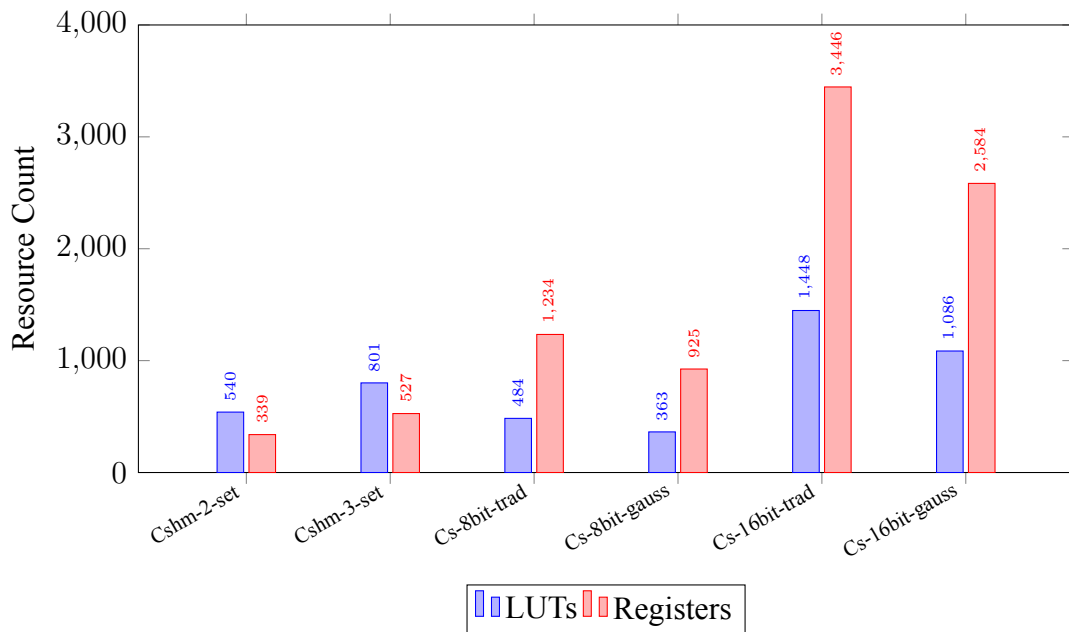


Figure 5.3: Utilization comparison for different custom multipliers.

The results indicate that for small twiddle bit-widths, the resource utilization of the two multiplier configurations is very similar, suggesting that the 2-set architecture is the better choice due to its simplicity. In contrast, for larger twiddle widths, the difference in resource consumption becomes significant. In these cases, utilizing two, three, or even more sets provides a substantial hardware advantage. Furthermore, across all configurations, the CSHM architecture utilizes significantly fewer registers due to its fixed pipeline depth. This pipeline is substantially shorter than that of the 8-bit carry-save multiplier (5 cycles versus 11 cycles), and this gap in register utilization widens even further as the data bit-width increases. Since the structure of this thesis heavily implies the use of small twiddle factors, the 2-slot version will be implemented.

5.2.3 Exploration of Alphabets and Set Combinations

To determine what number of alphabets required for the CSHM implementation, a custom script was developed to calculate the minimum number of slots and alphabets for a given twiddle factor bit-width, FFT size, algorithm type, and stage. By iterating through all these parameters, the script provides an overview of the design that can be implemented.

The script begins by generating the coefficients that the hardware needs. The twiddle factors $W_N^k = e^{-j2\pi k/N}$ are calculated, quantised to T_w bits, and then reduced to the subset that a particular stage of radix-2, radix-4 and split-radix needs. The real and imaginary parts of the coefficients form the set of target values $\mathcal{T}$.

The hardware is then described as a set of values it can reach. A single slot with certain alphabets $\mathcal{A}$ can produce

$$\mathcal{V}(\mathcal{A}, T_w) = \{ \sigma \alpha 2^s : \alpha \in \mathcal{A}, \sigma \in \{+1, -1\}, 0 \leq s < T_w \}, \quad (5.1)$$

and a multiplier with S such slots can produce every sum of S elements of $\mathcal{V}$.

$$\mathcal{R}(\mathcal{A}, T_w, S) = \left\{ \sum_{i=1}^S v_i : v_i \in \mathcal{V}(\mathcal{A}, T_w) \right\}. \quad (5.2)$$

A configuration is acceptable when every target is in this reachable set, or close enough to it:

$$\forall t \in \mathcal{T} \quad \exists r \in \mathcal{R} \quad |r - t| \leq \varepsilon, \quad \varepsilon = p \cdot \max_{t \in \mathcal{T}} |t|, \quad (5.3)$$

The tolerance fraction p is a parameter of the search. With $p = 0$ the multiplier must reproduce every coefficient exactly, a non-zero p shows an approximation, trading coefficient accuracy for hardware that can be smaller, and the resulting error is bounded by ε .

The search itself is exhaustive but ordered so that the cheapest hardware is found first. For a given number of terms, the alphabet is grown one odd integer at a time, starting from $\{1\}$, and the coverage condition of (5.3) is tested after each increase. Increasing the alphabet can only add members to $\mathcal{R}$, so the first alphabet that satisfies the condition is the smallest one that does, and the search for that number of slots/sets can stop there. The script is repeated for increasing numbers of slots. If the alphabet reaches the permitted maximum without covering the targets, that number of slots is not enough for the chosen coefficient width.

The outcome is the minimum hardware configuration for each set of parameters. The number of slots determines how many shifters and adders are used, while the alphabet size determines the number of pre-computed multiples. For the generation of the encoded keys, a second script uses the chosen configuration to find, for every required coefficient, the particular combination of alphabet member, sign and shift that produces it. The one with the smallest total shift is chosen. These combinations are stored as the keys that drive the multiplier at run time, so that the memory contents of the final design are produced.

The following results present the minimum number of alphabets required for 2-set and 3-set architectures when allowing for approximations.

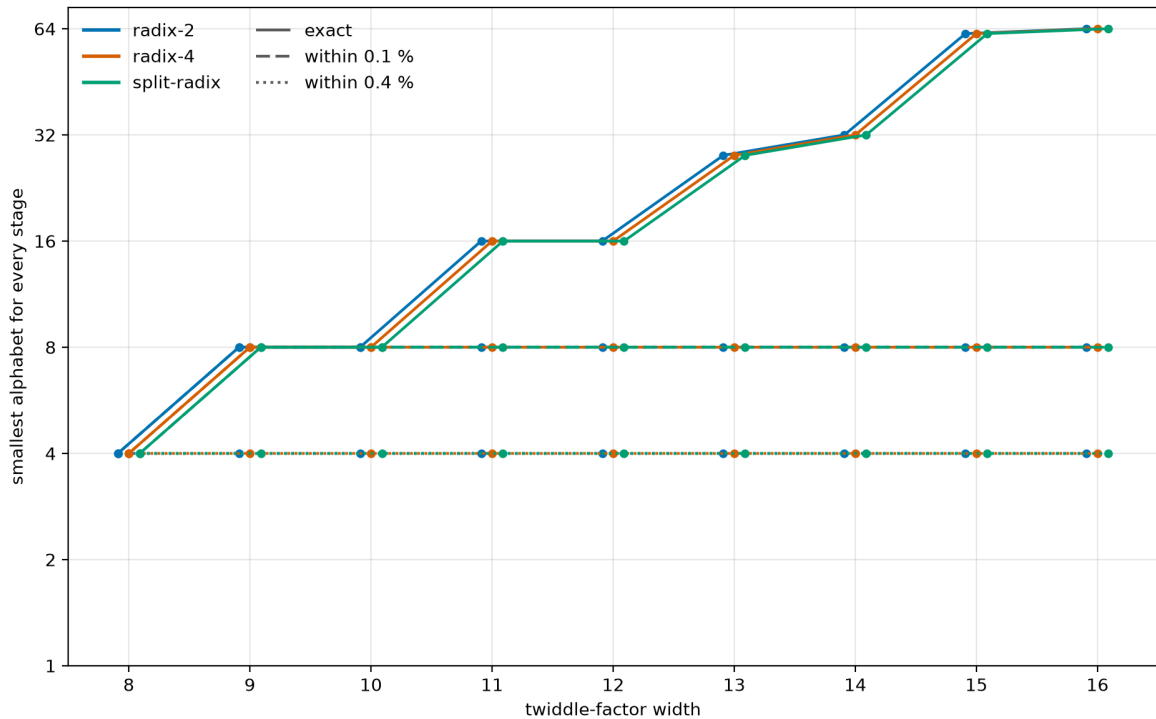


Figure 5.5: Alphabet needed vs twiddle width ($N = 256$, 2 slots)

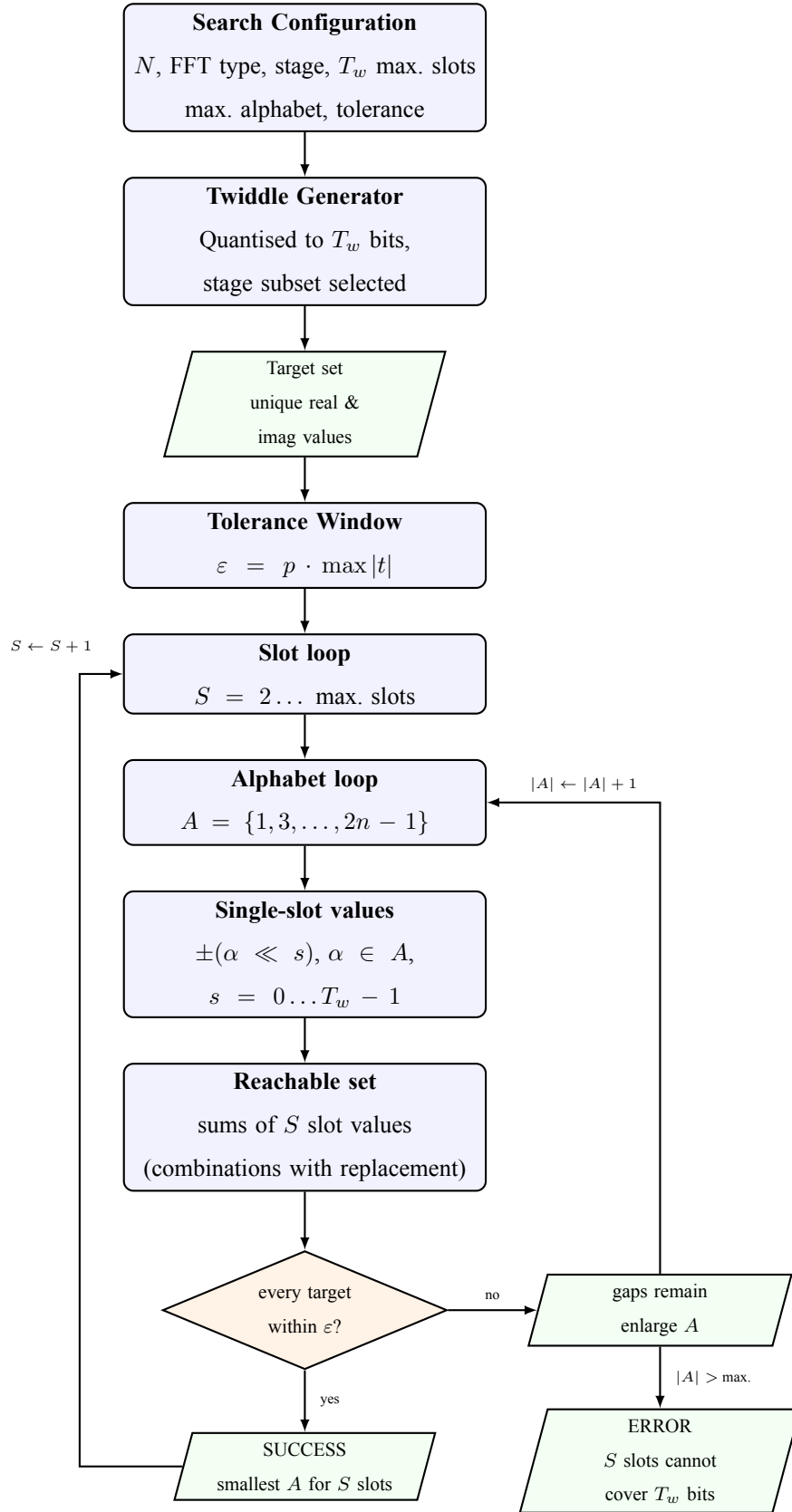


Figure 5.4: Slot and alphabet search implemented. For every slot count the alphabet is enlarged until the reachable set covers all quantised twiddle factors of the stage within the tolerance window.

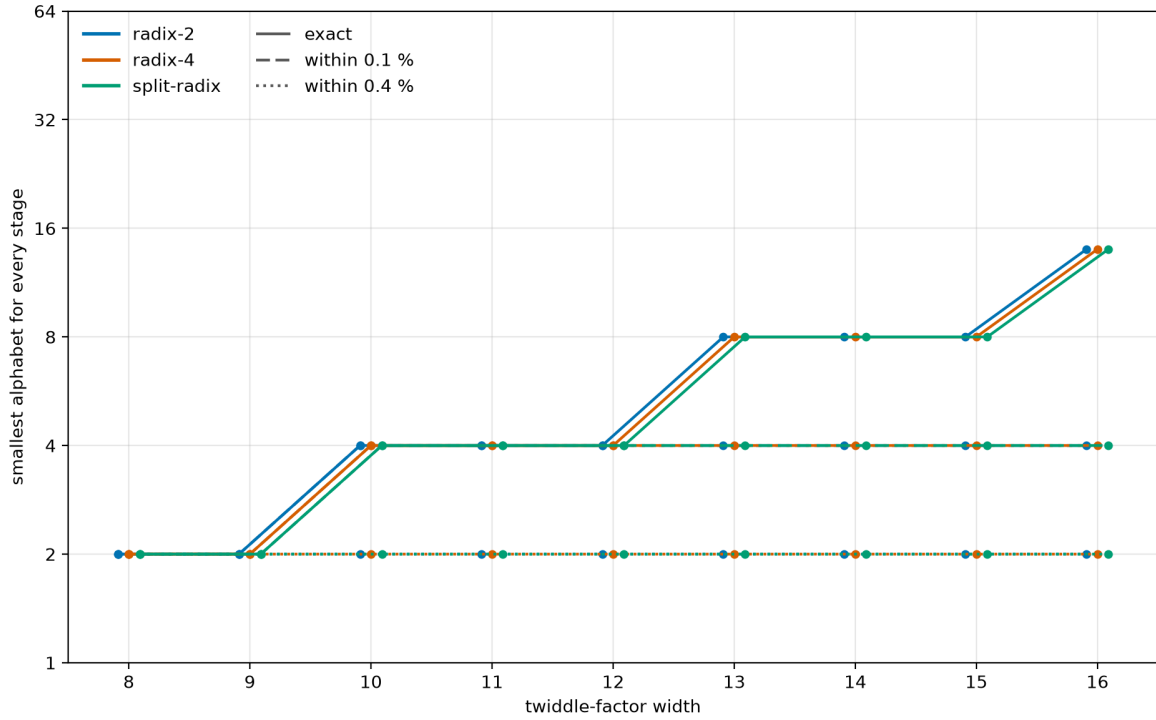


Figure 5.6: Alphabet needed vs twiddle width ($N = 256$, 3 slots)

The results demonstrate a significant reduction in hardware resource utilization achieved with minimal approximation. The acceptable error tolerance was determined empirically through trial and error.

The following results present the minimum number of alphabets per stage of the FFT for 2-slots.

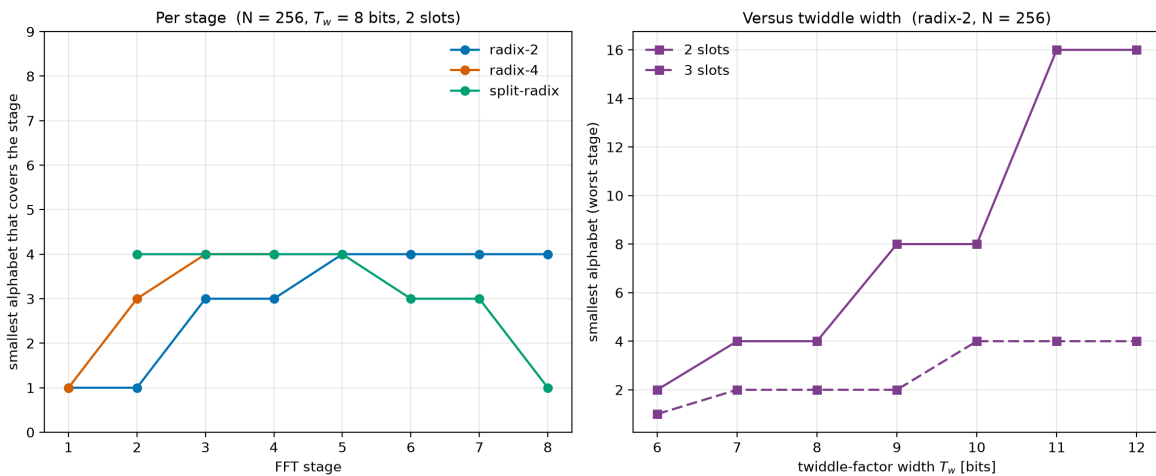


Figure 5.7: CSHM alphabet exploration with no tolerance.

As shown in the results, most stages across all FFT types need the maximum number

of alphabets. To keep the design simple and easy to scale, the same maximum-sized CSHM unit is used for every stage. While this uses slightly more hardware than necessary for a few stages, the extra cost is very small.

Lastly, the next figure presents a detailed plot of the CSHM keys for stage 8 of a 256-point Radix-2 FFT, configured with a 13-bit twiddle width, 2 slots, and 8 alphabets.

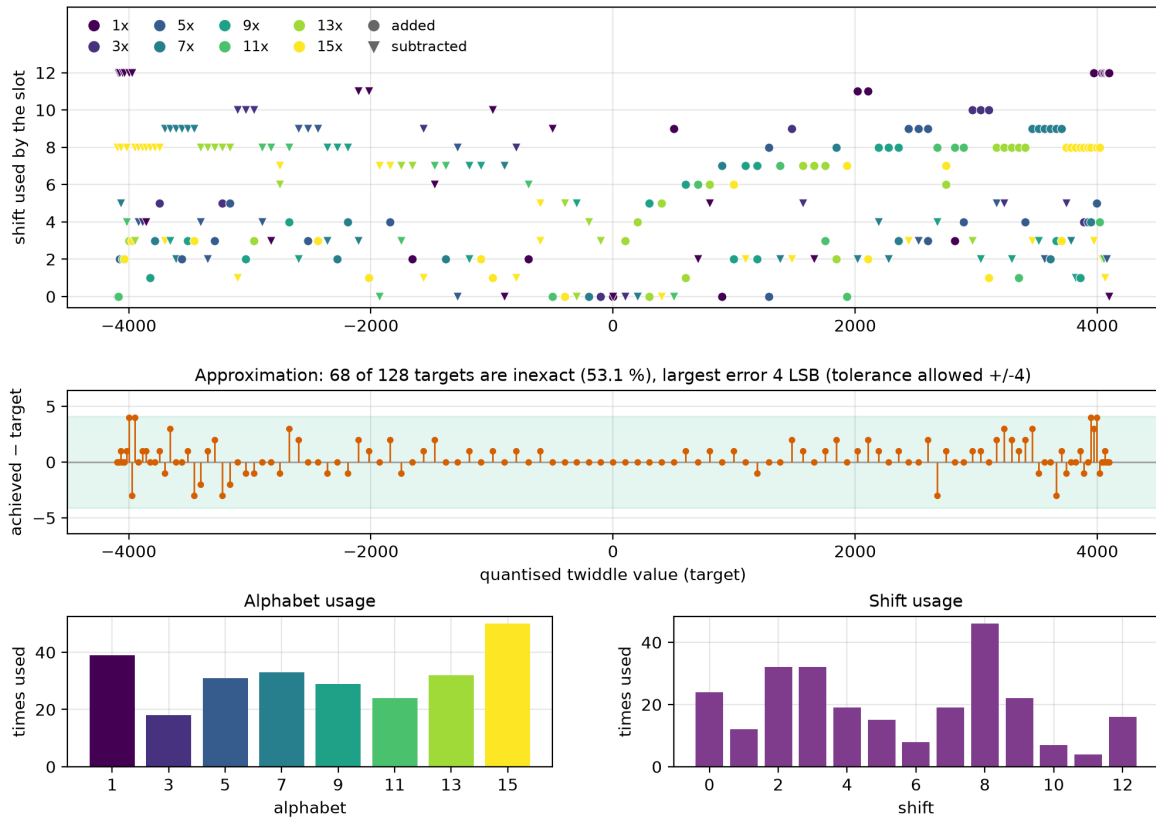


Figure 5.8: CSHM key ROM.

5.2.4 Simulation Accuracy Results

For the simulation and later hardware implementation, two CSHM modules were implemented and evaluated: a 2-slot, 4-alphabet design with a 0.4% approximation tolerance, and a 2-slot, 8-alphabet design with a 0.1% tolerance. Approximate units were chosen because, for smaller twiddle bit-widths, introducing a slight approximation is essential to achieve enough resource savings to make the CSHM architecture advantageous over the carry-save multiplier. Additionally, testing these two configurations allows for analyzing how minor tolerance variations influence the final computational accuracy of the FFT.

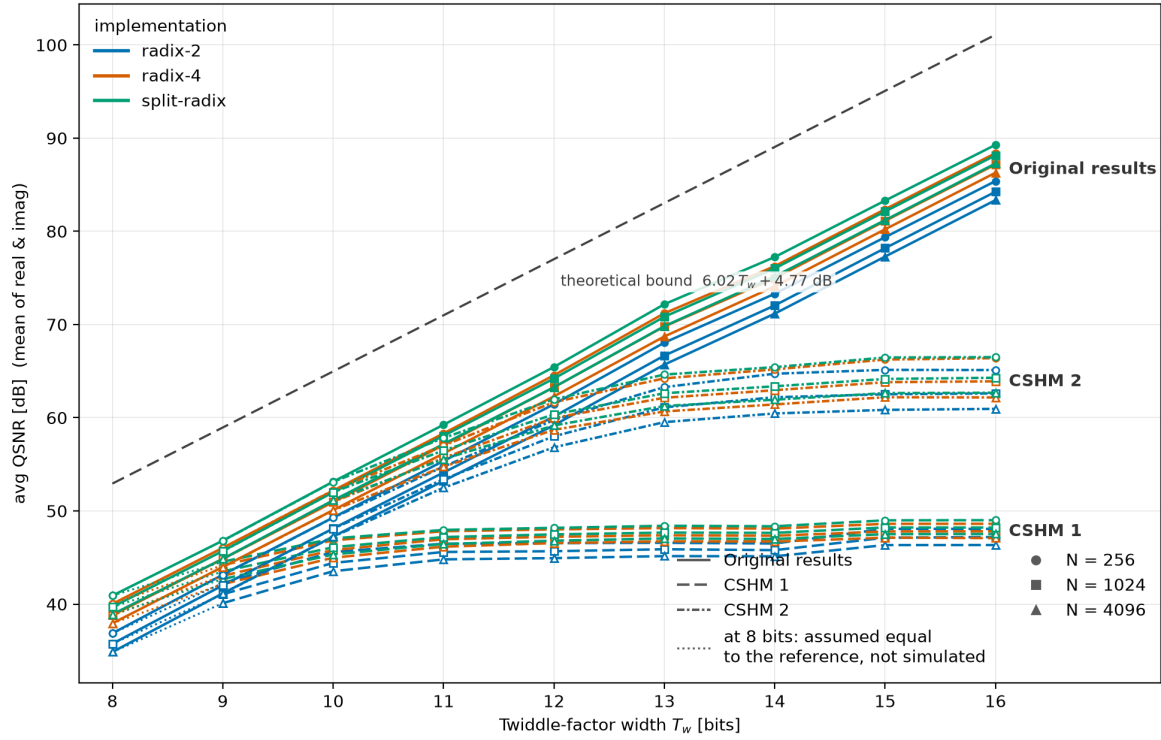


Figure 5.9: Different CSHM modules and their accuracy.

The results show that the 0.4% tolerance configuration has acceptable accuracy up to a 10-bit twiddle factor width with minimal loss, whereas the 0.1% tolerance design has good performance up to 13 bits. These small tolerances appear to bottleneck the overall accuracy beyond a certain bit-width, causing the SQNR curve to flatten rather than continuing to improve. Across all tested configurations, the error patterns remain the same across different FFT sizes and architecture types.

Chapter 6

Implementation Results

This chapter presents a comprehensive, though not exhaustive, selection of hardware implementation results. The evaluated architectures are contrasted based on their area utilization, critical path delay, and power consumption. Specifically, the analysis explores the performance trade-offs associated with different FFT types, twiddle factor bit-widths, Block RAM (BRAM) and Digital Signal Processor (DSP) utilization, as well as the integration of custom multipliers.

As established in previous chapters, all implementations utilizing DSP blocks also incorporate BRAMs, as this configuration has better overall performance. Conversely, designs featuring custom multipliers strictly avoid BRAMs, aligning with their primary design objective to minimize reliance on dedicated FPGA silicon resources.

All architectures were synthesized and implemented using AMD Xilinx Vivado. To maximize timing performance, the synthesis stage was configured with the retiming option enabled, and the implementation stage utilized the extra timing optimization directive (Extra-TimingOpt). All other tool parameters were kept at their default values.

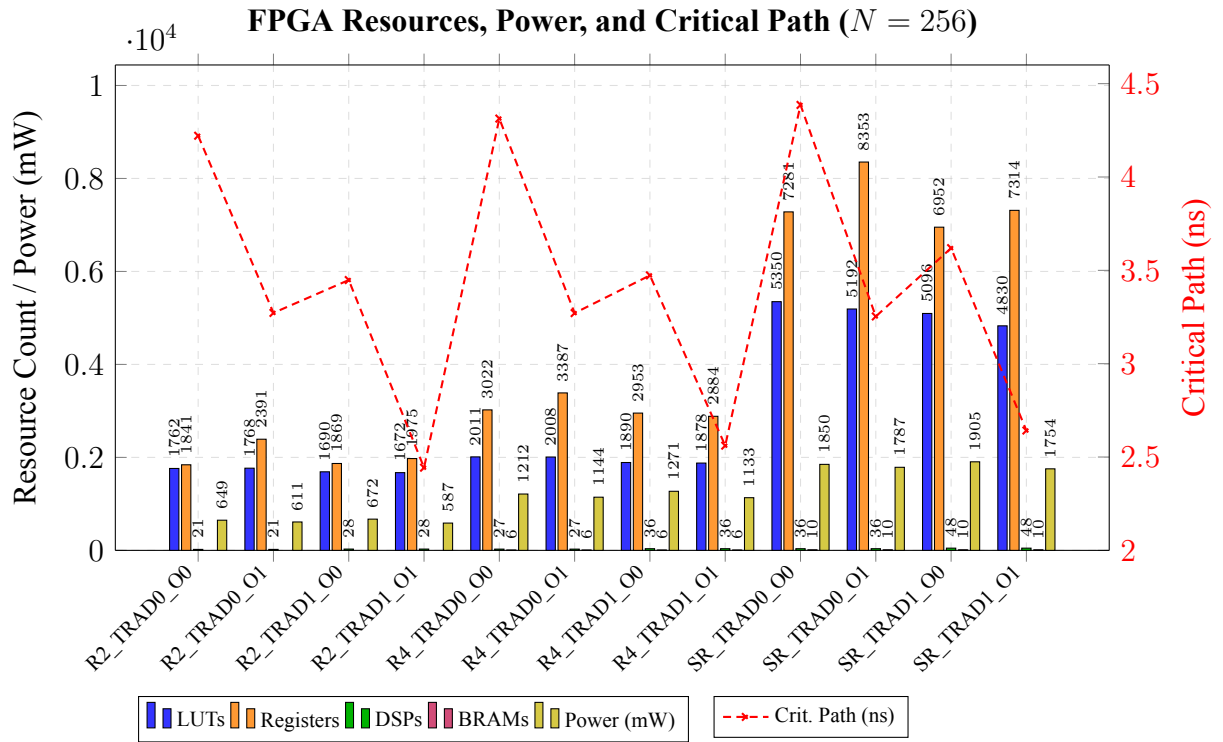
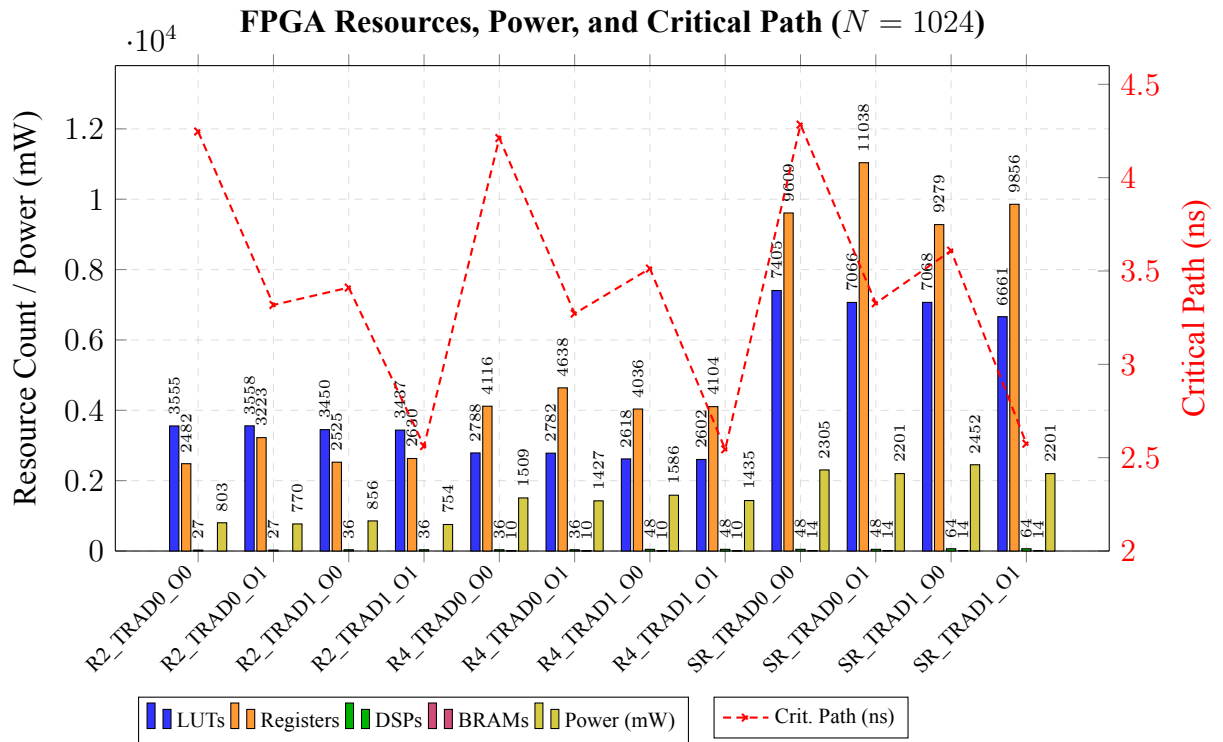
The bit widths chosen for implementation are 9, 13, and 16, representing the extremes and an optimal midpoint. Specifically, 9 bits is the minimum width supported by the CSHM module, 13 bits yields the best normalized results in accuracy testing, and 16 bits is the maximum width evaluated in this thesis.

Table 6.1: Hardware specifications of the AMD Xilinx Artix-7 XC7A200T-2FBG676 FPGA used for all implementations.

Feature	Specification
Device Family	Artix-7
Logic Cells	215,360
Look-Up Tables (LUTs)	134,600
Flip-Flops (Registers)	269,200
DSP Slices	740
Block RAM (36 Kb blocks)	365 (Total: 13,140 Kb)

6.1 9-Bit Unoptimized vs. Optimized Design

This section evaluates the impact of the DSP and Block RAM (BRAM) optimizations detailed in the last section of the third chapter. Furthermore, it analyzes the overall area utilization of the presented implementations, highlighting hardware trade-offs. R2 stands for Radix-2, R4 stands for Radix-4, SR stands for Split-Radix, TRAD stands for traditional or gauss multiplication and O stands for optimized or not.

Figure 6.1: FPGA results 9-bit Unopt vs Opt ($N = 256$).Figure 6.2: FPGA results 9-bit Unopt vs Opt ($N = 1024$).

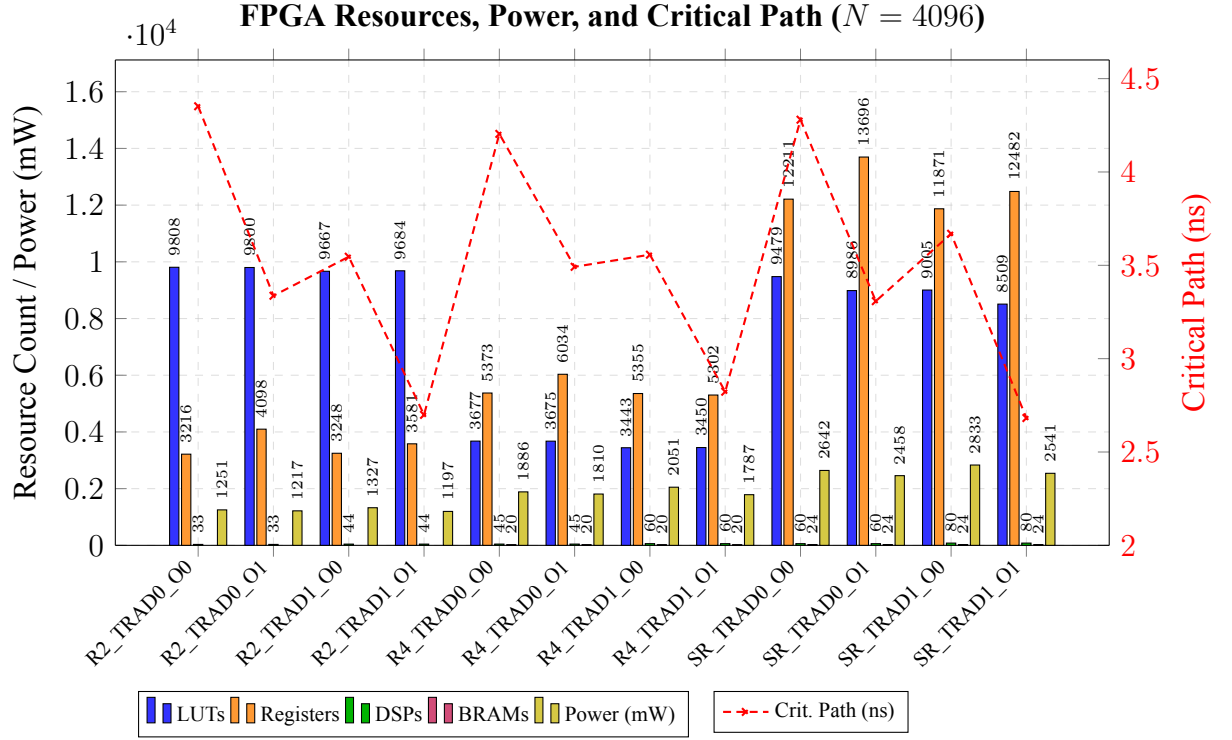


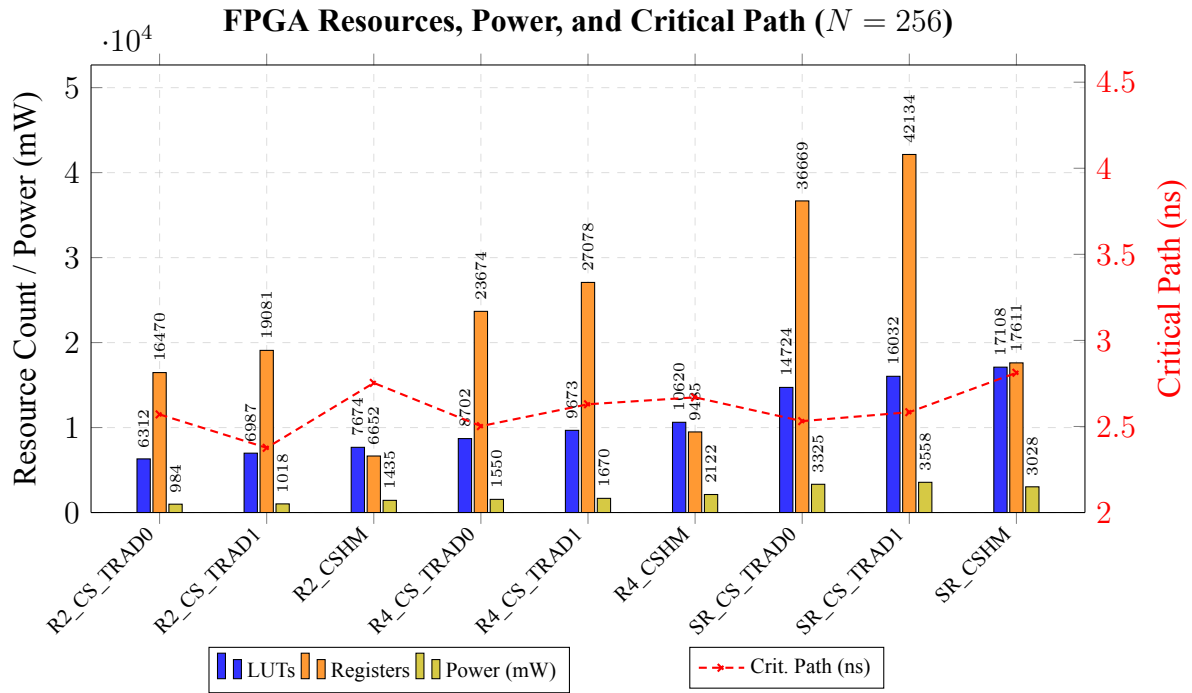
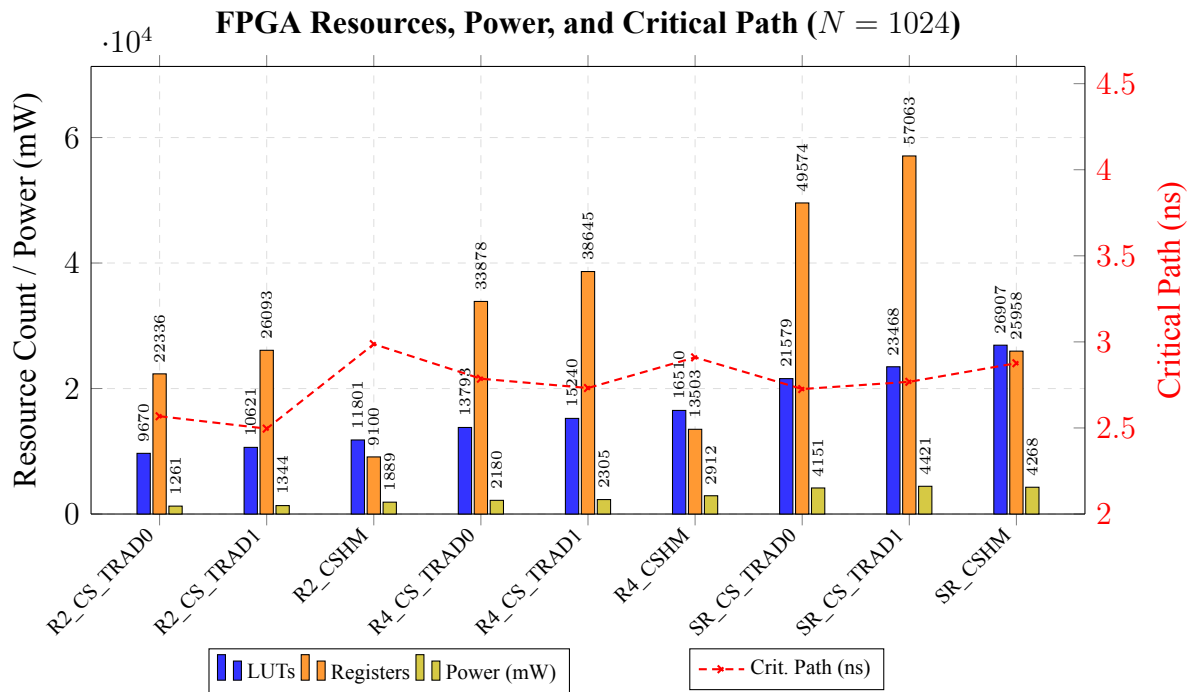
Figure 6.3: FPGA results 9-bit Unopt vs Opt ($N = 4096$).

These results follow the same trends as the ISO results in Chapter 3. Radix-2 and Radix-4 architectures use a similar amount of area. The main exception is for the 4096-point FFT, where using BRAMs saves a significant amount of regular FPGA fabric memory. On the other hand, the Split-Radix architecture uses more area across all FFT sizes.

Regarding timing, the traditional multiplier achieves better results. Furthermore, across all tests, the optimized versions show almost a 1 ns timing improvement compared to the unoptimized versions.

6.2 9-Bit Carry-Save vs. CSHM Multipliers

This section evaluates the performance of the Computation Sharing Multiplier (CSHM) in comparison to the Carry-Save multiplier. Because the design utilizes a 9-bit twiddle factor width, the CSHM-1 implementation was selected for this analysis. Also, no Brams are used for the designs with custom multipliers in order to evaluate the effects of not using dedicated FPGA silicon resources.

Figure 6.4: FPGA resources 9-Bit Carry-Save vs CSHM ($N = 256$).Figure 6.5: FPGA resources 9-Bit Carry-Save vs CSHM ($N = 1024$).

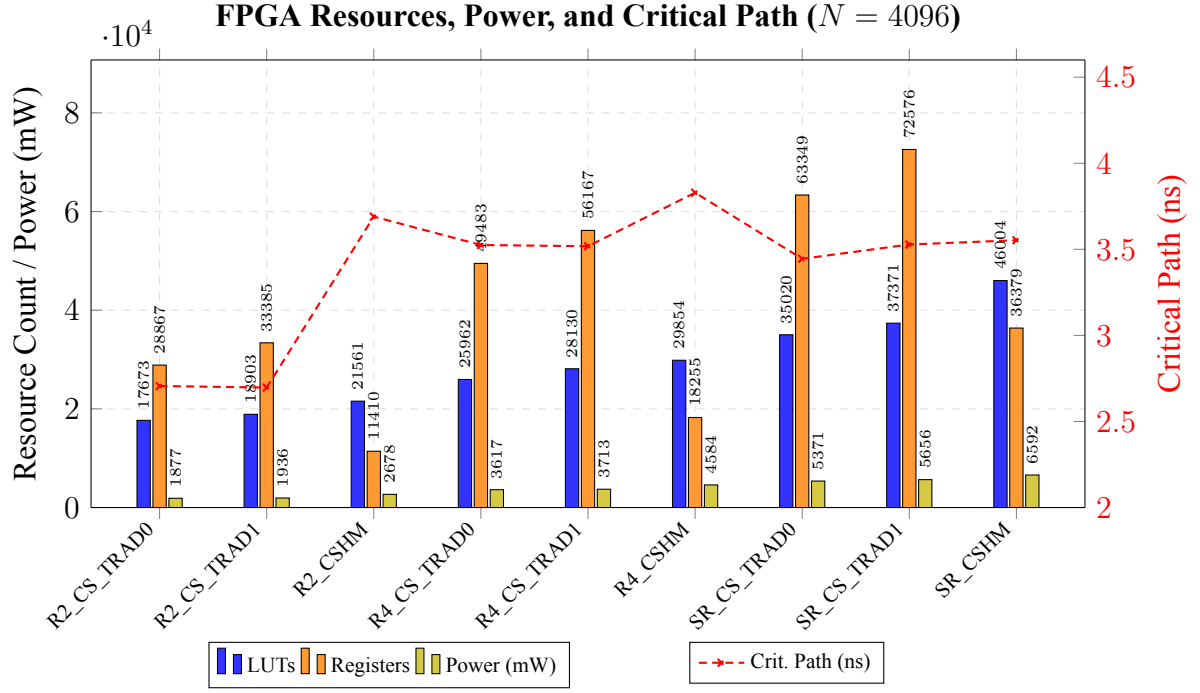


Figure 6.6: FPGA resources 9-Bit Carry-Save vs CSHM ($N = 4096$).

As demonstrated in the results, the CSHM architecture consistently outperforms the Carry-Save multiplier in terms of hardware utilization. The primary trade-off is a slight increase in critical path delay compared to the Carry-Save designs. Furthermore, all implementations achieve a critical path of under 3 ns, with the exception of the 4096-point FFT. These timing results are comparable to the DSP and BRAM optimized architectures discussed previously, however, a notable distinction is that the DSP/BRAM configurations successfully maintain these sub-3 ns speeds even at the 4096-point size. Furthermore, the performance of the traditional and Gauss multipliers in carry-save implementations is almost identical, even though the Gauss architecture uses only three-quarters of the carry-save multipliers.

6.3 13-Bit and 16-Bit Implementations

Similar results to those above are presented here for twiddle factor widths of 13 and 16, without the unoptimized versions. This comparison illustrates how the additional bits increase the overall area and how this extra area subsequently impacts the critical path.

6.3.1 DSP and BRAM Optimized Versions

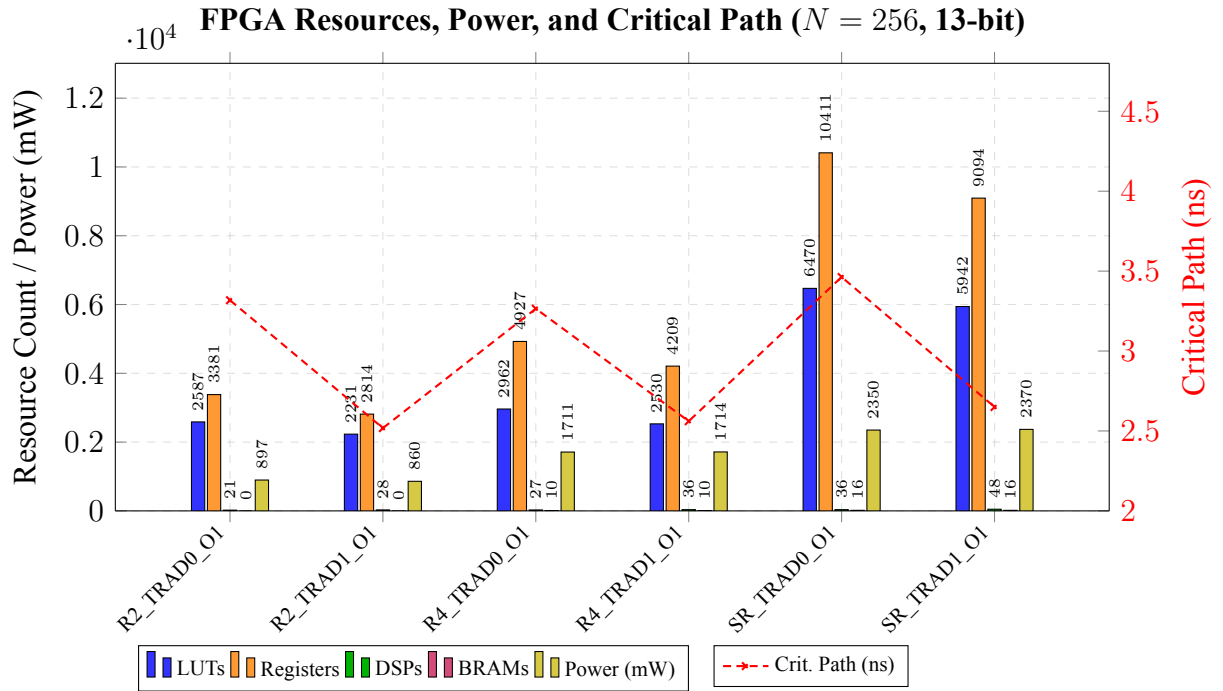


Figure 6.7: FPGA resources 13-bit Opt ($N = 256$).

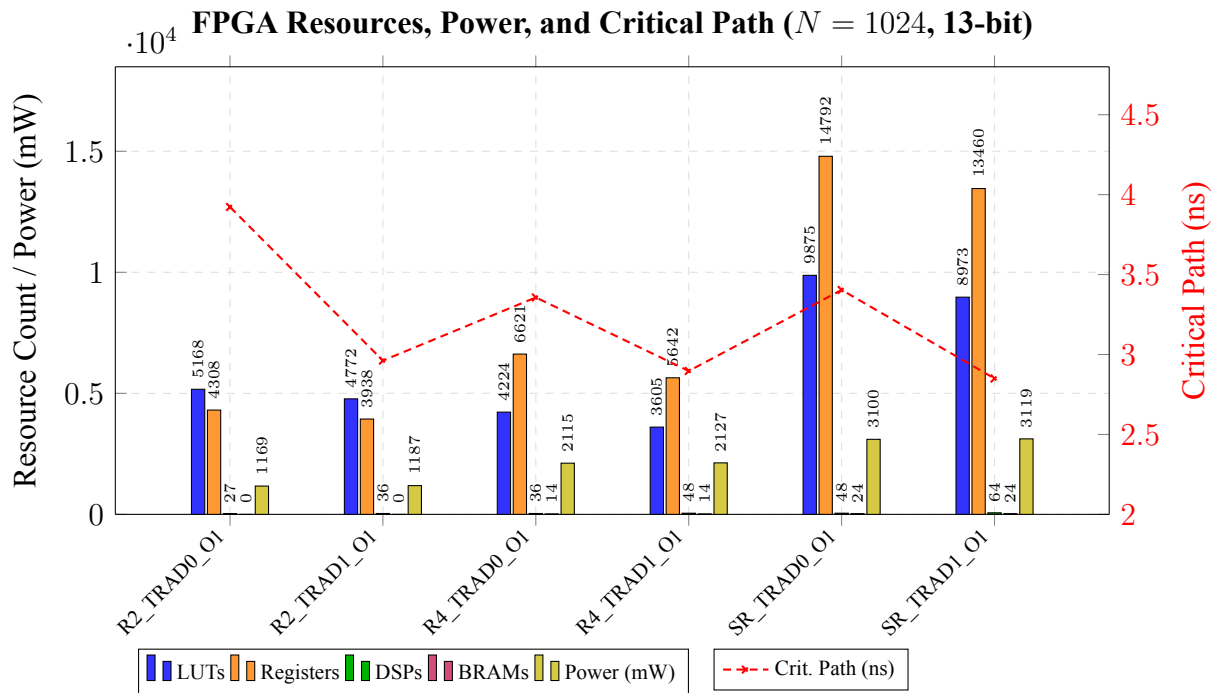


Figure 6.8: FPGA resources 13-bit Opt ($N = 1024$).

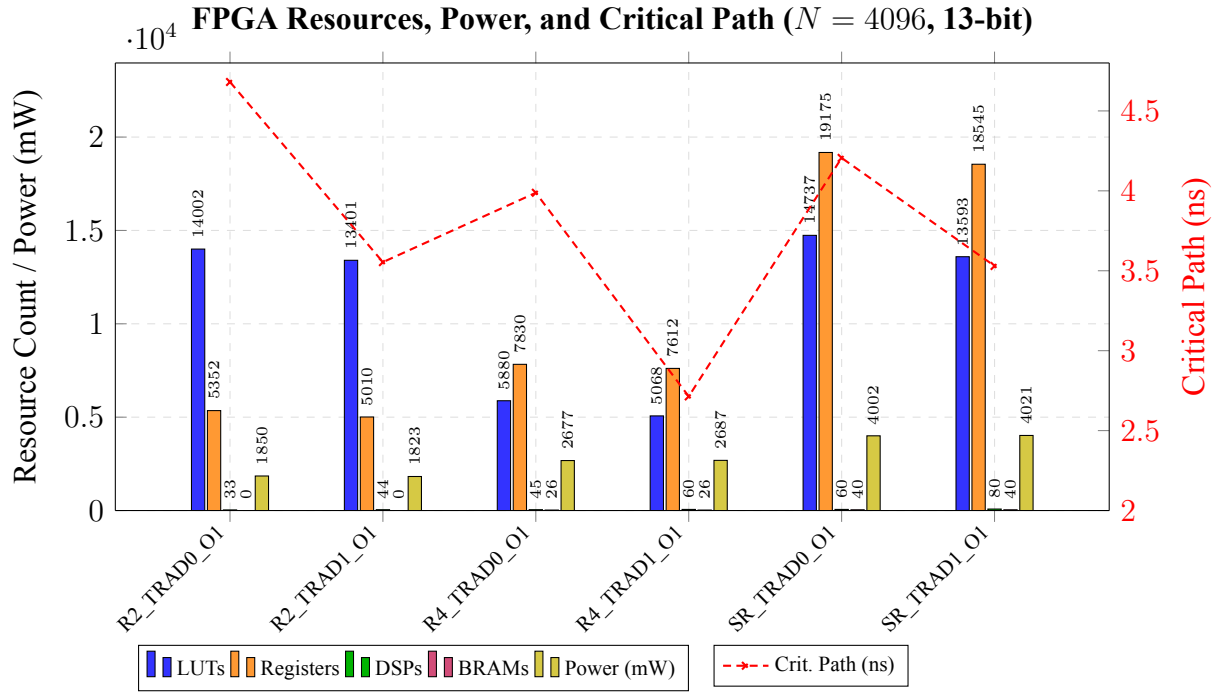


Figure 6.9: FPGA resources 13-bit Opt ($N = 4096$).

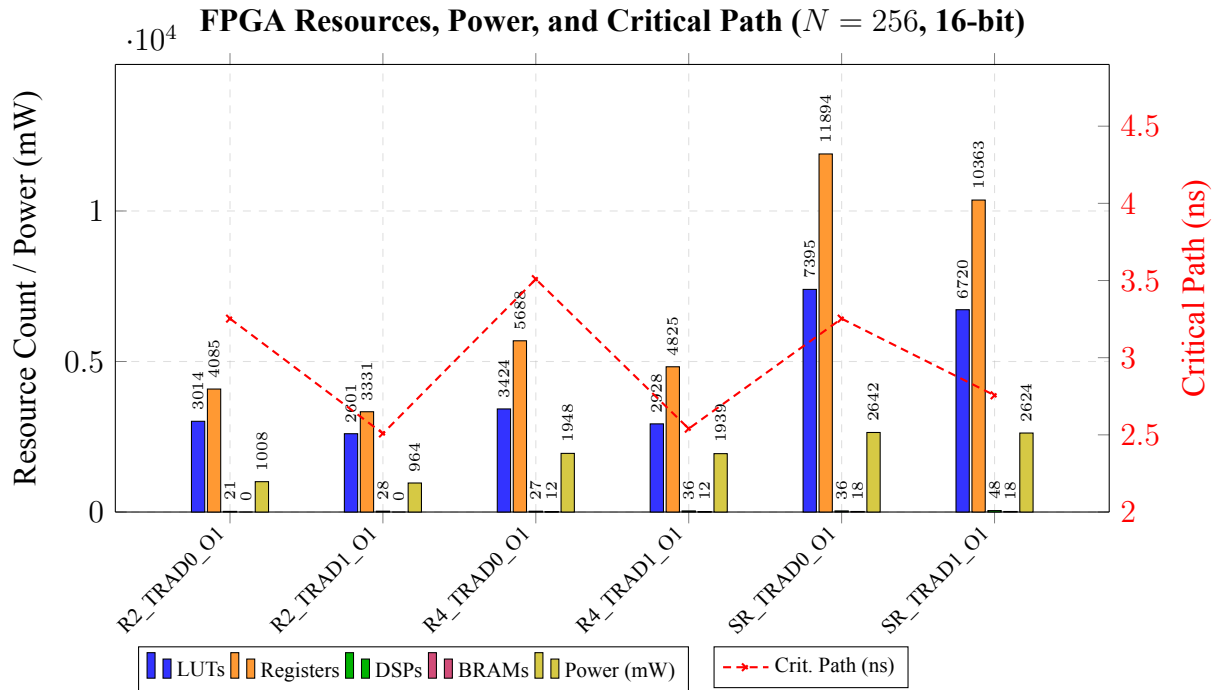
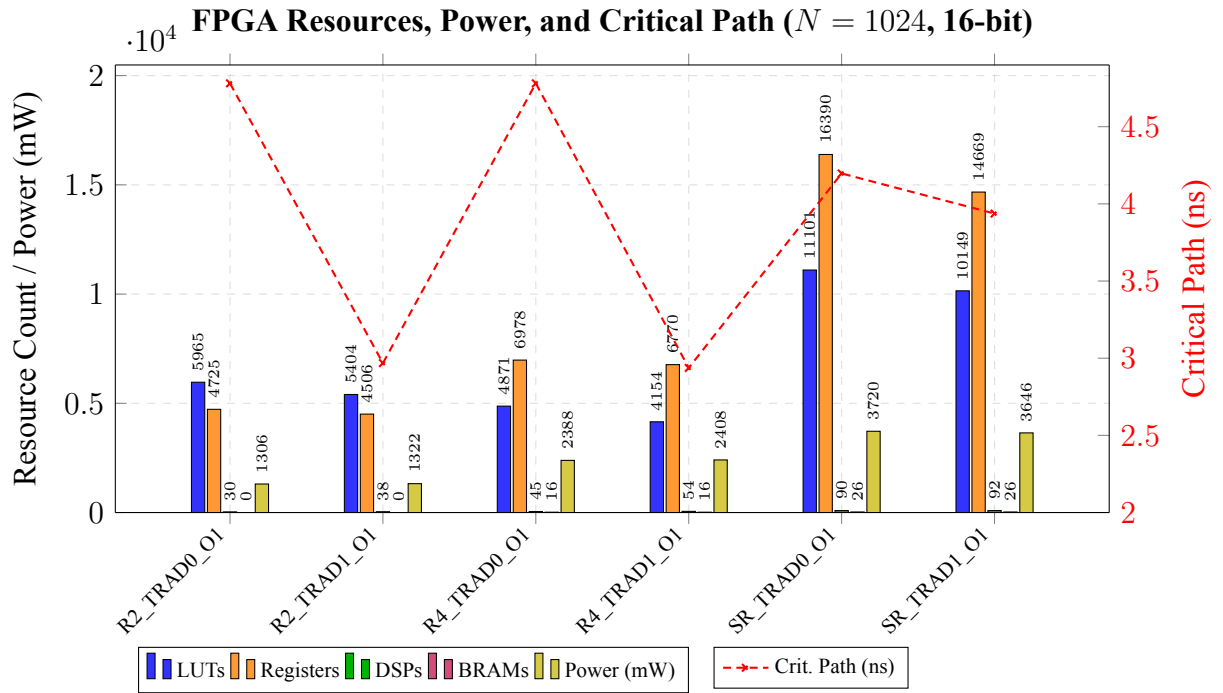
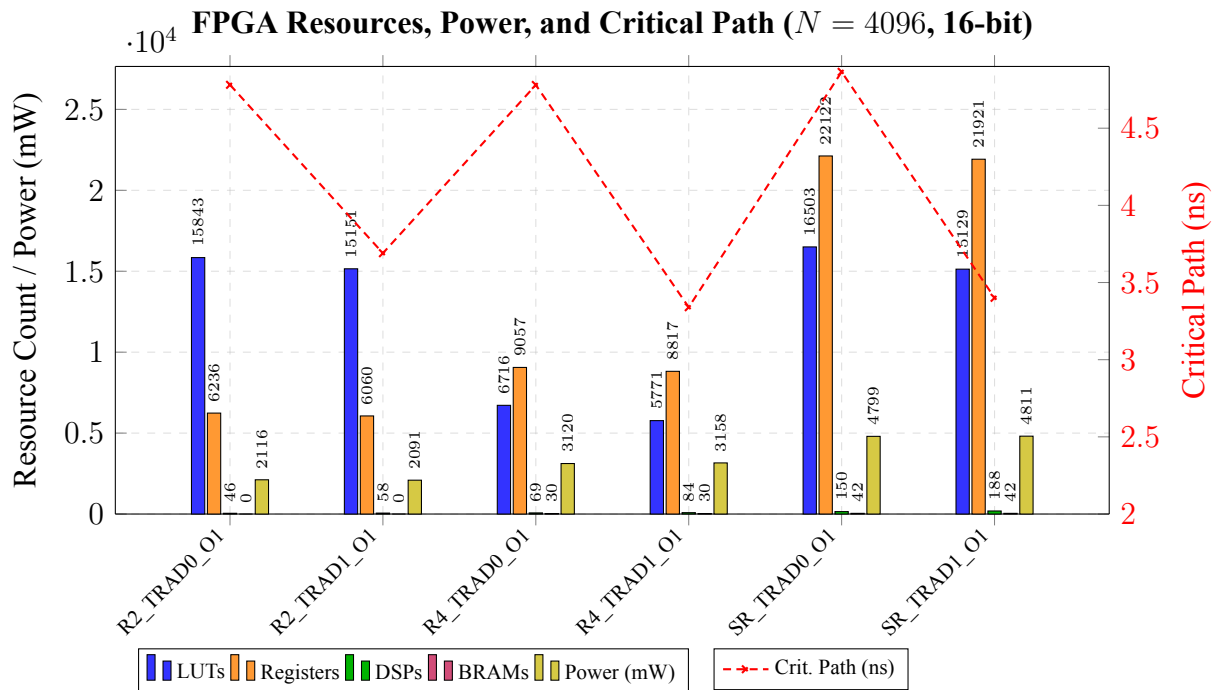


Figure 6.10: FPGA resources 16-bit Opt ($N = 256$).

Figure 6.11: FPGA resources 16-bit Opt ($N = 1024$).Figure 6.12: FPGA resources 16-bit Opt ($N = 4096$).

The pattern observed in the 9-bit implementation continues, with the critical path lengthening uniformly.

6.3.2 Carry-Save and CSHM Versions

For the custom multiplier, the implementation was limited to 13 bits because obtaining the results for 16 bits took over 10 hours and could not be fully completed. For the CSHM, since a 13-bit width is used, the second CSHM implementation is applied.

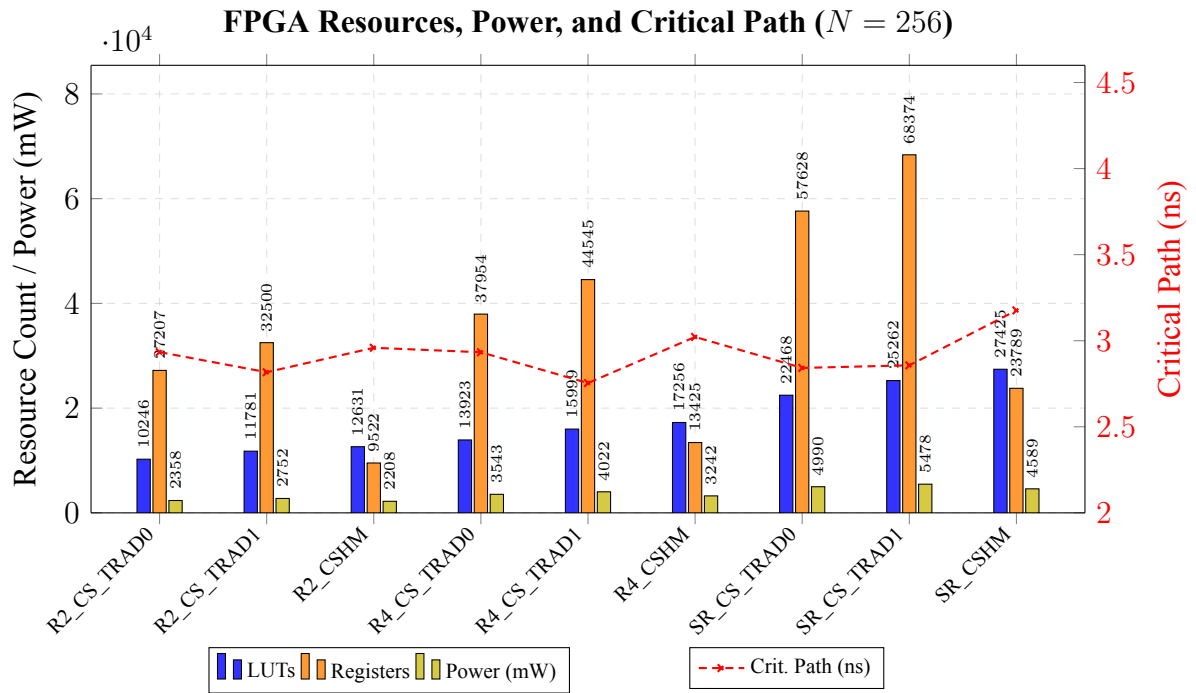
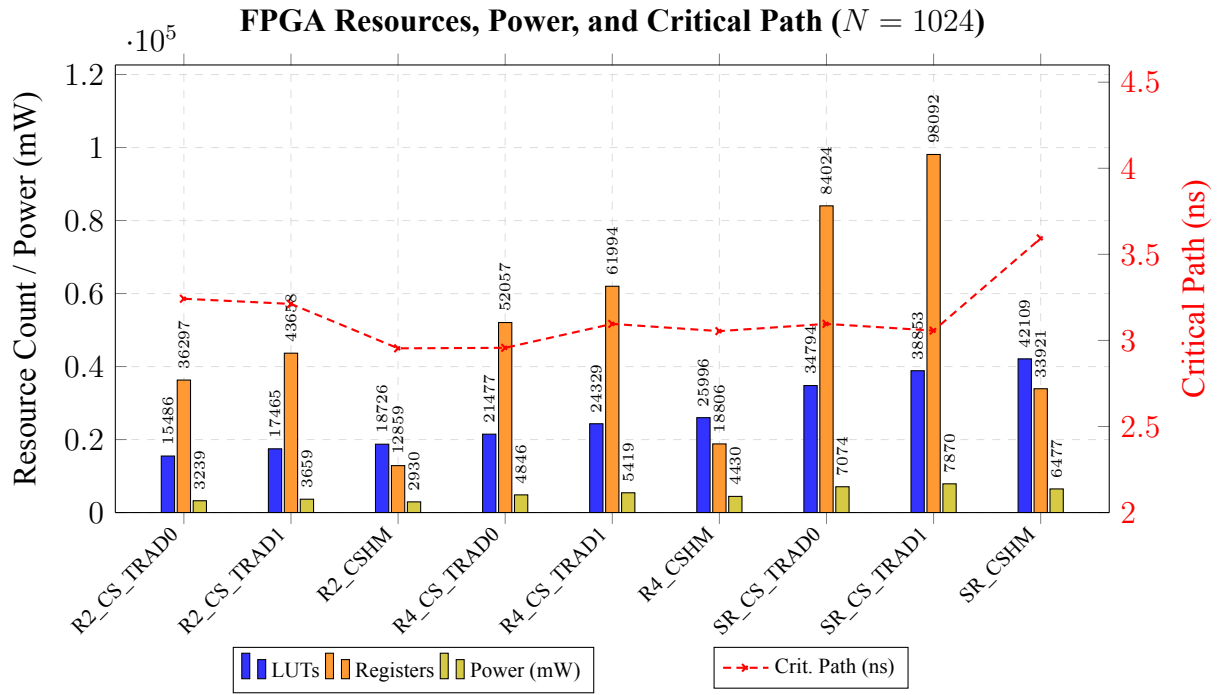
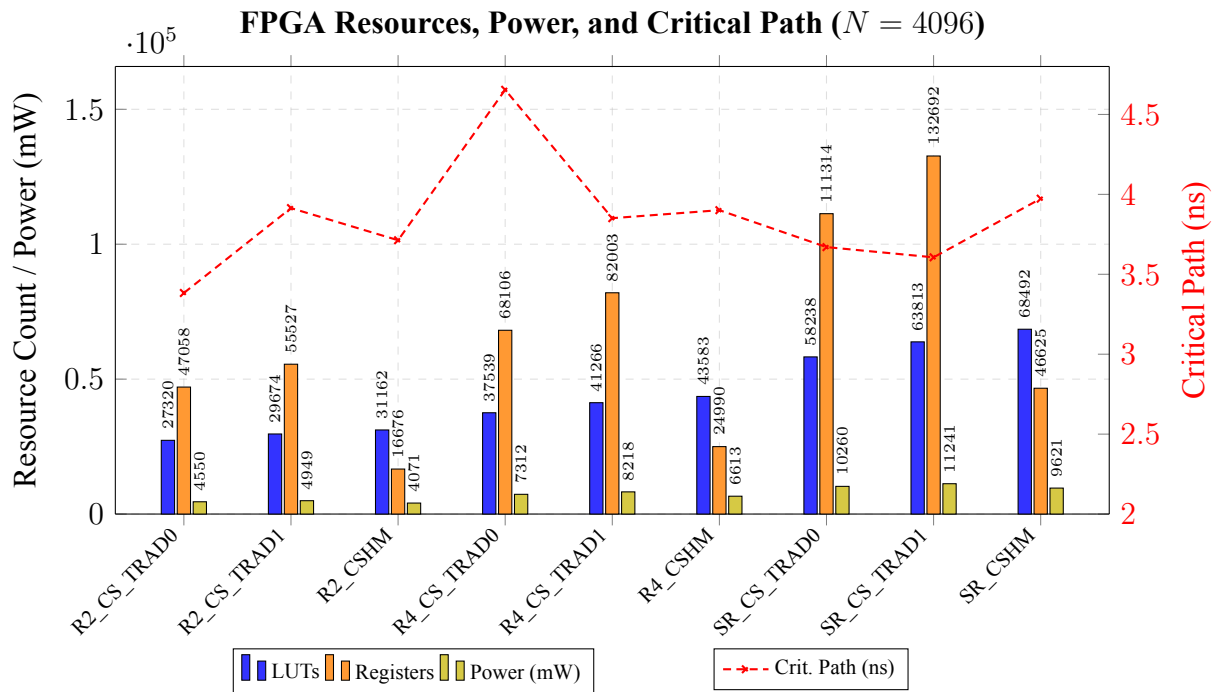


Figure 6.13: FPGA resources 13-Bit Carry-Save vs CSHM ($N = 256$).

Figure 6.14: FPGA resources 13-Bit Carry-Save vs CSHM ($N = 1024$).Figure 6.15: FPGA resources 13-Bit Carry-Save vs CSHM ($N = 4096$).

The results show the same pattern as the 9-bit version, but with very high power con-

sumption and longer critical paths.

6.4 Latency and Throughput Results

The following section presents the FIFO latency and throughput of selected configurations to evaluate their performance for DSP applications.

6.4.1 DSP and BRAM Optimized Versions

The latencies for the best 9-bit implementations per FFT type and FFT size will be evaluated, since the twiddle factor size does not algorithmically affect the latency of these designs. The only difference would be through the reduction of the maximum frequency caused by an increase in the critical path. The best versions are the ones using the traditional multiplier.

Table 6.2: Performance metrics for $N = 256$ (9-bit Opt)

Architecture	Latency (Cycles)	Frequency (MHz)	Latency (μs)	Throughput (MSamples/s)
Radix-2	179	409.3	0.437	818.6
Radix-4	104	390.4	0.266	1561.6
Split-Radix	172	381.0	0.451	1524.0

Table 6.3: Performance metrics for $N = 1024$ (9-bit Opt)

Architecture	Latency (Cycles)	Frequency (MHz)	Latency (μs)	Throughput (MSamples/s)
Radix-2	577	390.3	1.478	780.6
Radix-4	308	392.9	0.784	1571.6
Split-Radix	530	388.3	1.365	1553.2

Table 6.4: Performance metrics for $N = 4096$ (9-bit Opt)

Architecture	Latency (Cycles)	Frequency (MHz)	Latency (μ s)	Throughput (MSamples/s)
Radix-2	2127	370.5	5.741	741.0
Radix-4	1088	354.2	3.072	1416.8
Split-Radix	1896	372.8	5.086	1491.2

Overall, the Radix-4 design is best across the tested implementations, having the lowest latency and the highest processing throughput. Because it consumes four samples per clock cycle, it cuts the execution cycle count and overall runtime roughly in half compared to the Radix-2 design, while effectively doubling data throughput past 1400 MSamples/s.

The Split-Radix has high throughput—ranging between 1491.2 and 1553.2 MSamples/s. Its deeper algorithmic pipeline keeps its latency high and close to that of the Radix-2 architecture. Split-Radix therefore offers a throughput advantage over Radix-2, but Radix-4 remains the most well-rounded option whenever low response time and high data rates are equally essential.

6.4.2 Carry-Save and CSHM Versions

For this version, instead of testing different transform sizes and FFT types, only the 256-point Radix-2 architecture is examined at 9 and 13 bits. This is done because carry-save delay depends directly on the width of the data. Additionally, the previous section has already shown how transform size and FFT type affect latency.

Table 6.5: Performance metrics for $N = 256$ Radix-2 (9-bit)

Multiplier	Latency (Cycles)	Frequency (MHz)	Latency (μ s)	Throughput (MSamples/s)
Carry-Save	258	420.8	0.613	841.6
CSHM	186	363.2	0.512	726.4

Table 6.6: Performance metrics for $N = 256$ Radix-2 (13-bit)

Multiplier	Latency (Cycles)	Frequency (MHz)	Latency (μ s)	Throughput (MSamples/s)
Carry-Save	293	354.9	0.826	709.8
CSHM	186	337.9	0.550	675.8

Overall, the CSHM multiplier provides the lowest execution latency, keeping a fixed runtime of 186 cycles across both bit widths. Because its pipeline depth remains constant regardless of word length, widening the data to 13 bits does not increase algorithmic latency.

The standard Carry-Save design has a higher operating frequency. The increase in data width results in higher overall latency. Carry-Save is better when sample rate is important, while CSHM is the correct choice when latency and scaling are required.

6.5 Evaluation Against Existing Architectures

The work presented in [5] implements a split-radix processor utilizing a 16-bit twiddle factor width. However, unlike the progressive bit-growth strategy employed in this thesis, their design scales down the datapath after each complex multiplication to maintain a constant word length. To ensure a fair architectural comparison that accounts for the resource overhead of bit growth, their 16-bit design will be compared against the 13-bit implementation proposed in this work. Furthermore, since the referenced paper does not report SQNR metrics, this comparative evaluation will be strictly based on hardware utilization and timing performance. Unfortunately, a perfectly direct comparison is not feasible, as the proposed architectures cannot be implemented on the specific FPGA devices utilized in the referenced paper. It is also important to mention that this paper calculates latency using LIFO.

Table 6.7: Comparison of FFT Processors

Processor	Altera Core	Xilinx Core	Their Design	Radix-2	Radix-4	Split-radix
FFT size	256	256	256	256	256	256
Twiddle Size	16	16	16	13	13	13
LUTs	2144	2027	6702	2231	2530	5942
Regs	3758	—	6498	2814	4209	9094
DSPs	12	30	48	28	36	48
BRAMs	19 (9K)	3 (36K)	5 (4K)	0	10 (36K)	16 (36K)
Latency(μs)	0.69	0.59	0.123	0.128	0.102	0.286
Frequency (Mhz)	370	432	350	397	390	377
FPGA family	Stratix III	Virtex 5	Stratix II	Artix-7	Artix-7	Artix-7

These results show that the radix-2 and radix-4 designs are superior based on the evaluated metrics, demonstrating similar or lower latencies and reduced utilization of LUTs, registers, and DSPs. In contrast, the referenced split-radix implementation exhibits higher LUT utilization, requires fewer registers, and uses an identical number of DSPs. Furthermore, it utilizes fewer BRAMs—allocating BRAM for twiddle factors rather than internal stage memory—and achieves a lower latency compared to the proposed split-radix design from this Thesis.

Chapter 7

Conclusions and Future Work

This thesis presented a thorough study of design options for FPGA-based FFT processors. It explored key choices such as pipeline style (SDF vs. MDF), FFT algorithms (Radix-2, Radix-4, and Split-Radix), data bit-widths, complex multiplication methods, and memory setups. It also examined how to optimize FPGA resources like BRAMs and DSPs, while comparing two custom multipliers (Carry-Save and CSHM) to highlight the pros and cons of each approach.

Overall, this work serves as a practical guide for building a flexible and scalable FFT processor that can meet different design needs, such as low area, high accuracy, or high speed. It demonstrated how each choice directly affects energy use, hardware cost, latency, and throughput. Finally, it introduced automated scripts to test these designs and offered valuable insights into how the CSHM multiplier performs with FFT twiddle factors.

7.1 Summary of Results

The results indicate that the Radix-4 architecture represents the most efficient design option whenever feasible. When Radix-4 cannot be deployed, the choice between Radix-2 and Split-Radix depends primarily on the required throughput. Although the Split-Radix algorithm achieves the highest numerical accuracy, this marginal gain does not outweigh its substantial logic area and power consumption penalties.

Utilizing Block RAMs (BRAMs) for delay path storage proved to be the optimal approach, preserving FPGA fabric resources and enabling straightforward scalability to larger transform lengths. Furthermore, reducing the twiddle-factor bit width provides notable area

savings at the expense of degraded accuracy, hence, designs should adopt the minimum word length that satisfies application accuracy constraints. The strategic integration and optimization of DSP blocks and BRAMs proved decisive, yielding an approximate 1 ns reduction in critical path delay across all architectures. Finally, the CSHM multiplier demonstrated clear advantages over the Carry-Save design by consuming significantly fewer logic resources and achieving lower execution latency, with only a marginal reduction in peak operating frequency.

7.2 Future Work and Extensions

- **Datapath and Pipeline Optimization:** Further refine the pipeline depth of Stage 3 in the Split-Radix architecture to reduce excessive register utilization and shorten total execution latency.
- **CSHM Exploration:** Expand the design space exploration of the Computation Sharing Multiplier (CSHM) in FFT applications by investigating more hardware implementations and optimizing the multiplier design.
- **SoC Integration and Real-Time Verification:** Integrate the proposed FFT cores into a complete FPGA-based System-on-Chip (SoC) using standard on-chip interconnects (such as AXI4-Lite or AXI4-Stream) to interface with an embedded processor, enabling empirical validation of the design.
- **RISC-V Coprocessor and Dynamic Scalability:** Adapt the FFT architecture as a hardware accelerator or tightly coupled coprocessor within an open-source RISC-V ecosystem, supporting run-time reconfigurability for variable transform sizes alongside on-the-fly twiddle factor computation.

Bibliography

- [1] J. W. Cooley and J. W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, April 1965. doi:10.2307/2003354.
- [2] P. Duhamel and H. Hollmann. Split radix FFT algorithm. *Electronics Letters*, 20(1):14–16, January 1984. doi:10.1049/el:19840012.
- [3] Xilinx, Inc. 7 series DSP48E1 slice user guide. Technical Report UG479 (v1.10), Xilinx, Inc., San Jose, CA, March 2018. https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1.
- [4] Xilinx, Inc. 7 series FPGAs memory resources user guide. Technical Report UG473 (v1.14), Xilinx, Inc., San Jose, CA, July 2019. https://docs.amd.com/v/u/en-US/ug473_7Series_Memory_Resources.
- [5] J. García, J. A. Michell, G. Ruiz, and A. M. Burón. FPGA realization of a split radix FFT processor. In *VLSI Circuits and Systems III*, volume 6590 of *Proc. SPIE 6590*, page 65900P, May 2007. doi:10.1117/12.721867.
- [6] P. Gupta. Accurate performance analysis of a fixed point FFT. In *22nd National Conference on Communication (NCC)*, pages 1–6, 2016. doi:10.1109/NCC.2016.7561147.
- [7] B. P. Lathi. *Modern Digital and Analog Communication Systems*. Oxford University Press, New York, 3rd edition, 1998.
- [8] P. Duhamel and M. Vetterli. Fast Fourier transforms: A tutorial review and a state of the art. *Signal Processing*, 19(4):259–299, April 1990. doi:10.1016/0165-1684(90)90158-U.

- [9] J. Park, W. Jeong, H. Mahmoodi-Meimand, Y. Wang, H. Choo, and K. Roy. Computation sharing programmable FIR filter for low-power and high-performance applications. *IEEE Journal of Solid-State Circuits*, 39(2):348–357, February 2004. doi:10.1109/JSSC.2003.821785.
- [10] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020. doi:10.1038/s41586-020-2649-2.
- [11] S. Chowdhury, B. K. Pandit, and A. Banerjee. Computation-efficient and multiplierless hardware realization of decimation in time FFT. In *2022 IEEE Region 10 Symposium (TENSYP)*, pages 1–6, Mumbai, India, 2022. doi:10.1109/TENSYP54529.2022.9864458.
- [12] W. Snyder. Verilator: Open-source SystemVerilog simulator and lint system. <https://www.veripool.org/verilator/>. Accessed: 10-08-2026.
- [13] Xilinx, Inc. Vivado design suite, version 2018.3. <https://www.xilinx.com/products/design-tools/vivado.html>, 2018. Accessed: 12-08-2026.
- [14] F. Skarman et al. Surfer: An extensible waveform viewer, version 0.7.0. <https://surfer-project.org/>, 2024. doi:10.5281/zenodo.10653540.
- [15] Signal-to-quantization-noise ratio. https://en.wikipedia.org/wiki/Signal-to-quantization-noise_ratio. Accessed: 13-08-2026.
- [16] Cooley–Tukey FFT algorithm. https://en.wikipedia.org/wiki/Cooley-Tukey_FFT_algorithm. Accessed: 01-09-2026.
- [17] Fourier transform. https://en.wikipedia.org/wiki/Fourier_transform. Accessed: 01-09-2026.